

TRAMA

Technical Documentation

News Portal and Editorial Management System

Technologies

Laravel 13 · Vue 3 · Inertia.js 2 · MySQL

Project Technical Documentation

Author: Gastón Rizzo

Table of Contents

1. Overview: What TRAMA Is	3
2. Technologies, Development Environment, and Libraries	4
3. General Architecture	12
4. Project Structure	14
5. Configuration Files and Environment Variables	18
6. MySQL Database: Tables and Relationships	24
7. How Laravel, Inertia, and Vue Connect	29
8. The Date Used by TRAMA and TramaClock	36
9. Server-Side Rendering (SSR) and Hydration	39
10. Public Homepage: How Articles Are Selected and Organized	43
11. Registration, Authentication, Account Recovery, and Sessions	45
12. Roles, Dashboards, and Permissions	52
13. CMS Dashboard and Role-Based Metrics	54
14. Creating an Article: Fields and Validation	56
15. Autosave and Change Protection	61
16. Workflow Between Journalist and Editor	63
17. Scheduling, Publishing, Archiving, and Homepage Placement	68
18. Article History, Revisions, and Restoration	70
19. Categories and Tags	72
20. Images: Validation, Processing, and Recovery	76
21. Comments and Registered-User Engagement	80
22. Automatic Comment Moderation	83
23. Manual Review of Comments and Accounts	87
24. Blocking and Deleting Users and Employees	92
25. Contact and Informational Pages	98
26. Advertising: Banners, Impressions, Clicks, and Reporting	100
27. Views, Profiles, and Search	106
28. Queues, Scheduled Tasks, Transactions, and Concurrency	113
29. Error Handling, Logging, and Fallback Pages	119
30. Local Installation of TRAMA	127
31. Running TRAMA in Development	135
32. Deploying TRAMA to a Server	138
33. Test Data and Seeders	142
34. Automated Tests and Technical Checks	149

1. Overview: What TRAMA Is

TRAMA is a Spanish-language news portal organized into two major areas within the same application. The first is the public site, where any visitor can read articles and registered users can participate. The second is the internal area, where journalists, editors, and administrators work with content and site management.

The project implements a complete editorial workflow. An article can begin as a draft, be sent to an editor, returned to the journalist with feedback, scheduled, published, or archived. The system also preserves previous revisions, processes images, moderates comments, records advertising metrics, and runs background tasks.

The code includes an internal reader role. That value identifies accounts created through the portal. In the interface and throughout this document, the term registered user is used because it better describes this type of account.

How This Document Is Written

Each section first explains what happens in TRAMA through a practical example. It then introduces the classes, tables, methods, and technical names that make that behavior possible. Names taken from the code are preserved exactly as they appear in the project.

How Step-by-Step Examples Are Used in This Documentation

To keep the technical explanations grounded, the chapters follow specific people. The visible action is presented first, followed by an explanation of what happens inside Vue, Inertia, Laravel, MySQL, Node.js, or the relevant supporting processes.

- Carlos represents the developer who installs, runs, configures, deploys, and tests TRAMA.
- Candela represents a registered user who browses the portal, searches for news, and participates.
- Valentina represents a journalist who creates and edits articles and submits them for review.
- Mateo represents an editor who reviews, schedules, publishes, and moderates.
- Marina represents an administrator who manages categories, tags, advertising, and accounts.

Step by Step: An Article Through TRAMA from Start to Finish

This walkthrough provides a mental map of the project before moving into specific classes and files.

Step	What the Person Does and What Happens Internally
1	Valentina enters the CMS and creates an article as a draft. Vue keeps the form values while she writes.
2	When she saves, Inertia sends the request to Laravel. A Form Request validates the data, and the controller delegates the editorial rules to the appropriate services.
3	Eloquent stores the article and its relationships in MySQL. The draft may be associated with a category, tags, a cover image, and revisions.
4	Valentina submits the article for review. Mateo opens it from his dashboard and can return it for changes, schedule it, or publish it depending on its state and his permissions.
5	If the article is scheduled, Laravel Scheduler periodically checks whether its publication time has arrived and changes its state when appropriate.
6	Candela opens the article from the portal. Laravel queries the public data, records the view, and provides Vue with the article, visible comments, related articles, and advertising.

Step	What the Person Does and What Happens Internally
7	If Candela posts a comment, the request finishes quickly and automatic moderation continues through a queue and a worker. In this way, the portal, CMS, and background processes operate as a single system.

2. Technologies, Development Environment, and Libraries

TRAMA uses several technologies, but each one solves a specific problem. This section explains what each technology does before its technical name is used throughout the rest of the document.

Technology	What It Does in TRAMA
Laravel 13 / PHP	This is the part that runs on the server. It receives requests, validates data, applies permissions, queries MySQL, manages sessions, runs tasks, and prepares responses.
Vue 3	Builds the interface users see in the browser and updates parts of the screen when data changes.
Inertia.js 2	Allows Laravel to deliver data directly to Vue screens without creating a separate data-route layer for every screen.
MySQL	Permanently stores users, articles, comments, revisions, advertising, sessions, and the rest of the system's data.
Laravel Fortify	Provides the foundation for signing in and recovering passwords. TRAMA adds its own account and verification rules on top of that foundation.
Laravel Queue	Allows jobs to run in the background. TRAMA uses it to process automatic comment moderation without blocking the browser request.
Laravel Scheduler	Allows Laravel to define tasks that must be checked periodically, such as publishing scheduled articles.
Node.js	Allows JavaScript to run outside the browser. TRAMA uses it to process images, run local models, and generate the initial HTML for the homepage.

What Node.js Is and How TRAMA Uses It

Node.js is an environment that allows JavaScript to run outside the browser. In TRAMA, it does not replace Laravel: PHP and Laravel remain the main server, receiving requests, querying MySQL, applying permissions, and preparing responses. Node.js is used when the project needs to run server-side tools or processes written in JavaScript.

Node.js and NPM serve different purposes. Node.js is the program that runs JavaScript; NPM is the package manager that downloads and installs the JavaScript libraries defined in package.json. As a result, npm install installs the dependencies, after which Node.js can run Vite, Sharp, ONNX Runtime, franc, and TRAMA's own scripts.

In this project, Node.js is involved in four specific tasks:

Use	What Node.js Does in TRAMA
Vite	npm run dev and npm run build run Vite with Node.js to process the frontend JavaScript, Vue, and CSS and generate the development or production assets.

Use	What Node.js Does in TRAMA
SSR	php artisan inertia:start-ssr uses the SSR bundle generated from resources/js/ssr.js. Node.js runs Vue on the server to produce the homepage's initial HTML.
Comments	The ModerateComment Job starts a Node.js process using scripts/comments/moderate-comment.mjs. That process analyzes a comment, returns a JSON result to Laravel, and then exits.
Images	Laravel starts scripts/images/process-image.mjs and scripts/images/classify-image-safety.mjs. Sharp processes the images, and ONNX Runtime runs the local safety classification.

Files with the .mjs extension are JavaScript modules. The files inside scripts/ are not Vue components or code that runs in the browser; they are helper programs that Node.js executes when Laravel needs to perform a specific task. For processes started from PHP, the executable path is obtained from TRAMA_NODE_BINARY and made available as config('trama.node_binary').

Composer, NPM, and Libraries

Composer is the tool that installs and maintains the PHP libraries Laravel needs. A library is ready-made code that solves a specific problem and can be reused by the project instead of being implemented from scratch.

NPM is the JavaScript package manager used by TRAMA. It reads package.json and downloads, installs, and maintains the libraries needed by the frontend and by processes executed with Node.js. Those dependencies include Chart.js, Axios, Sharp, franc, Hugging Face Tokenizers, and ONNX Runtime.

Library	Actual Use in TRAMA
tightenco/ziggy + ziggy-js	Allow Vue and SSR to use the named routes defined by Laravel without manually writing URLs.
@lucide/vue	Provides reusable icons for layouts, buttons, editorial actions, searches, and navigation.
@tanstack/vue-table	Manages the structure and sorting of the Articles, Comments, Users, Categories, and Tags grids.
@vuepic/vue-datepicker	Provides the date and time pickers used in filters, article scheduling, and advertising active periods.
Chart.js + vue-chartjs	Display the daily impressions-and-clicks chart in the Advertising module.
Axios	Sends requests from JavaScript to the server, for example during autosave.
Sharp	Rotates, crops, resizes, and converts images.
ONNX Runtime	Runs text- and image-classification models locally.
Hugging Face Tokenizers	Prepares text for some moderation models.
Franc	Automatically detects the language of comments to support the moderation process.

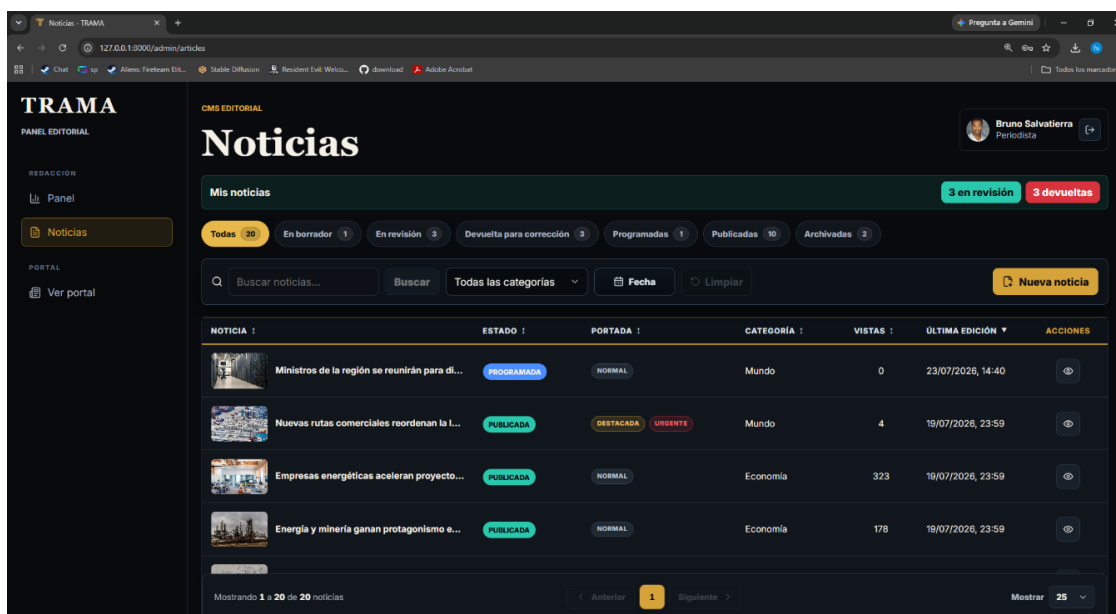
Interface Libraries and Vue Integrations

The table above summarizes the main dependencies, but some of them are directly involved in building CMS screens. TRAMA uses TanStack Vue Table for several administrative grids, Vue DatePicker for dates and times, Chart.js together with vue-chartjs for advertising charts, and Ziggy/Inertia to connect Vue with Laravel routes and navigation.

These tools do not replace server-side logic. Laravel continues to validate data, apply permissions, query MySQL, paginate results, and decide what information is delivered. Vue libraries are mainly responsible for interaction and for presenting that data in the browser. Lucide Vue, for example, provides reusable icons, but the HTML, CSS, and business rules still belong to TRAMA.

TanStack Vue Table: CMS Grids

TRAMA uses @tanstack/vue-table in the Articles, Comments, Users, Categories, and Tags listings. It is a headless library, meaning that it handles the logic required to organize columns, rows, cells, and sorting, but does not provide a predefined visual design. TRAMA therefore continues to build the table with its own HTML and apply the editorial dashboard's CSS styles.



Articles Grid in the Editorial Dashboard.

The TanStack examples in this subsection are taken from resources/js/Pages/Admin/Articles/Index.vue. To show how sorting connects to Laravel, app/Http/Requests/Editorial/ArticleIndexRequest.php is also used. The excerpts preserve the project's real names and structures while omitting lines that are unnecessary for explaining this integration.

In resources/js/Pages/Admin/Articles/Index.vue, the TanStack pieces required to build the grid are imported and the sorting functionality used by this screen is enabled:

```
import {
  FlexRender,
  rowSortingFeature,
  tableFeatures,
  useTable
} from '@tanstack/vue-table';

const features = tableFeatures({
  rowSortingFeature,
});
```

In the same file, each column is defined with an object. id identifies the column, accessorFn obtains the value from the article received from Laravel, header defines the heading, and enableSorting indicates whether that column can sort the listing. For sortable columns, those identifiers match the values accepted by app/Http/Requests/Editorial/ArticleIndexRequest.php in the sort parameter.

```
{
  id: 'category',
  accessorFn: (article) =>
    article.category?.name || 'Sin categoría',
  header: 'Categoría',
  enableSorting: true,
  cell: (info) => info.getValue(),
},
```

The table instance is also created in `resources/js/Pages/Admin/Articles/Index.vue` and receives the articles on the current page through `props.articles.data`. The `manualSorting: true` option is important: TanStack must not sort only the visible rows, because pagination and full-list sorting are handled by Laravel.

```
const tableData = computed(() => props.articles.data || []);

const table = useTable({
  key: 'editorial-articles-table',
  features,
  columns,
  data: tableData,
  manualSorting: true,
  enableMultiSort: false,
  enableSortingRemoval: false,
  state: {
    get sorting() {
      return sorting.value;
    },
  },
  onSortingChange: changeSorting,
});
```

When the user clicks a column heading, TanStack reports which column and direction were selected. `changeSorting()` converts that choice into the sort and direction parameters, and TRAMA requests the listing again through Inertia. Laravel applies the ordering to the full query and returns the corresponding page.

```
function changeSorting(updater) {
  const nextSorting = typeof updater === 'function'
    ? updater(sorting.value)
    : updater;

  const selected = nextSorting[0] || {
    id: 'updated_at',
    desc: true,
  };

  sorting.value = [selected];

  visitList({
    sort: selected.id,
    direction: selected.desc ? 'desc' : 'asc',
  });
}
```

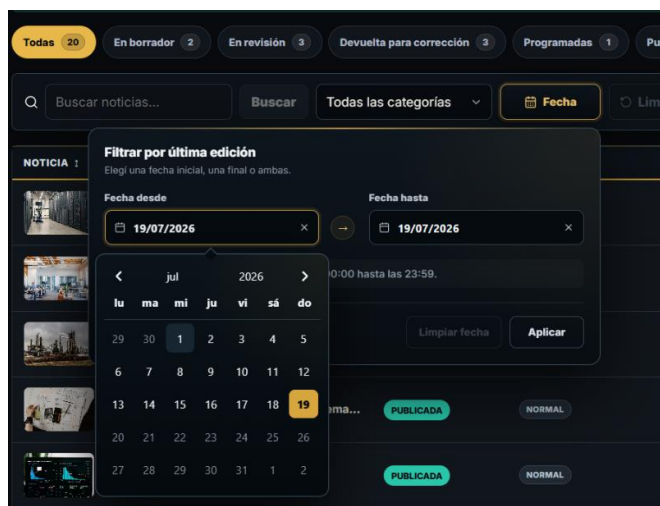
When rendering the table, `table.getRowModel().rows` provides the rows for the current page. Within each row, `row.getAllCells()` iterates through its cells and FlexRender resolves the content. TanStack organizes the grid model and state; Vue continues to generate the HTML, and TRAMA retains its own CSS styles.

```
<tr
  v-for="row in table.getRowModel().rows"
  :key="row.id"
>
```

```
<td
  v-for="cell in row.getAllCells()"
  :key="cell.id"
>
  <FlexRender :cell="cell" />
</td>
```

Vue DatePicker: Filters, Scheduling, and Active Periods

TRAMA uses @vuepic/vue-datepicker whenever a screen needs to select a date or a date-and-time combination. The following screenshot and code are taken from `resources/js/Pages/Admin/Articles/Index.vue`, where the component is used to filter articles by date. The same library is reused in `resources/js/Pages/Admin/Articles/Form.vue` to schedule articles and in `resources/js/Pages/Admin/Advertisements/Index.vue` to define campaign active periods. `resources/css/app.css` imports the component's base styles.



Date picker used to filter the Articles listing.

In this filter, the user sees the date in DD/MM/YYYY format, while `model-type` keeps the value received by Vue as YYYY-MM-DD. The Spanish locale is imported from `date-fns/locale`.

```
const datePickerFormats = {
  input: 'dd/MM/yyyy',
};

<VueDatePicker
  v-model="form.date_from"
  model-type="yyyy-MM-dd"
  :formats="datePickerFormats"
  :locale="es"
  :max-date="dateFromMaximum"
  :time-config="{ enableTimePicker: false }"
/>
```

`v-model` binds the calendar to Vue state; `formats` controls the visible representation; `model-type` defines the stable value that can be sent to the server; and `min-date` or `max-date` limit the options available through the interface. These restrictions improve the user experience, but Laravel validates the received values again.

In `resources/js/Pages/Admin/Articles/Form.vue`, the same component uses `model-type="yyyy-MM-dd'T'HH:mm"`, displays DD/MM/YYYY HH:mm, and enables time selection in 24-hour format. In `resources/js/Pages/Admin/Advertisements/Index.vue`, it is reused for `starts_at` and `ends_at`. The library handles the calendar interaction, but Laravel validates the dates it receives again.

Chart.js and vue-chartjs: Advertising Metrics

When the administrator opens Advertising, Laravel provides the metrics for the selected period and Vue receives the dates, impressions, and clicks used by the chart. The following screenshot and excerpts are taken from resources/js/Pages/Admin/Advertisements/Index.vue. In that file, Chart.js is the engine that builds the chart, while vue-chartjs provides the Vue Line component that integrates it into the screen. TRAMA registers only the modules required for that line chart. The screenshot comes from a local database that had already been used, so its cumulative values may differ from those produced by a fresh seed run documented in section 33.



Impressions and clicks chart in the Advertising module.

```
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  LineElement,
  PointElement,
  Tooltip,
  Legend
} from 'chart.js';
import { Line } from 'vue-chartjs';

ChartJS.register(
  CategoryScale,
  LinearScale,
  LineElement,
  PointElement,
  Tooltip,
  Legend,
);
```

Laravel provides the period labels, impressions, and clicks in props.chart. buildChartData() transforms those series into the format expected by Chart.js and assigns impressions and clicks to separate vertical axes so both magnitudes can be read clearly.

```
function buildChartData(chart) {
  return {
    labels: chart?.labels || [],
    datasets: [
      {
        label: 'Impresiones',
        data: chart?.impressions || [],
        yAxisID: 'yImpressions',
      },
      {
        label: 'Clicks',
        data: chart?.clicks || [],
        yAxisID: 'yClicks',
      },
    ],
  };
}
```

The template ultimately renders the chart using the Line component. TRAMA's options use two independent scales because impressions and clicks have different magnitudes. The screen also retains a signature of `props.chart` to avoid redrawing the chart when a new Inertia response only changes the grid order and the metrics remain the same.

```
<Line
  :data="chartData"
  :options="chartOptions"
/>
```

Vue Integrations with Laravel: Inertia and Ziggy

`createInertiaApp()` passes the Inertia plugin to setup. TRAMA registers it on the Vue instance through `.use(plugin)`. It then registers `ZiggyVue`, allowing the frontend to use the named routes defined by Laravel.

```
return createVueApp({
  render: () => h(App, props),
})
.use(plugin)
.use(ZiggyVue)
.mount(el);
```

Thanks to Ziggy, a screen can generate a URL from a route name and its parameters. For example, the article form's autosave uses `route('admin.articles.autosave', props.article.id)` instead of manually writing a URL containing the article identifier.

The same integration exists in `resources/js/ssr.js`. There, Vue is created with `createSSRApp()`, the Inertia plugin is registered, and `ZiggyVue` also receives the routes shared by Laravel. This allows `route()` to be resolved while Node.js generates the homepage's initial HTML.

Plugins Used During the Vite Build

`vite.config.js` registers the plugins involved during development and compilation. `laravel-vite-plugin` connects Vite to the `resources/css/app.css` and `resources/js/app.js` entry points and also declares `resources/js/ssr.js` as the SSR entry point. `@vitejs/plugin-vue` allows Vite to process `.vue` components.

```
plugins: [
  laravel({
    input: ['resources/css/app.css', 'resources/js/app.js'],
    ssr: 'resources/js/ssr.js',
    refresh: true,
  }),
  vue(),
  tailwindcss(),
],
```

The file also retains `@tailwindcss/vite` and `tailwind.config.js` with `@tailwindcss/forms`. However, the current interface is written with project-specific CSS: `resources/css/app.css` contains no `@tailwind` directives and does not import `tailwindcss`. For that reason, Tailwind is documented as configuration that is present in the project, but not as a styling technology on which the current interface depends.

Development Environment Used

TRAMA was developed primarily in Visual Studio Code, which was used to edit, organize, and debug the project's code. `composer.json` declares PHP ^8.3, but the current `composer.lock` pins Symfony 8.1 dependencies that require PHP 8.4.1 or later. The documented local environment uses PHP 8.5.8, which is the recommended version for reproducing this installation. Composer installs the PHP dependencies; Node.js and NPM are used for Vue, Vite, and supporting processes; and MySQL stores the application's data.

Item	Use in the TRAMA Environment
Visual Studio Code	Editor used to write, organize, and debug the code during development.
PHP 8.5.8	Version used and recommended for reproducing the documented environment. Although <code>composer.json</code> declares PHP ^8.3, the current <code>composer.lock</code> contains packages that require PHP 8.4.1 or later.
Composer	Installs and maintains the PHP dependencies for the Laravel backend.
Node.js + NPM	Node.js runs Vite, SSR, and the supporting scripts for comments and images. NPM installs the JavaScript dependencies defined in <code>package.json</code> , such as Vue, Sharp, franc, and ONNX Runtime.
MySQL	Stores the application's persistent data.

Artificial Intelligence Models in TRAMA

TRAMA is prepared to use artificial intelligence models for two specific system functions: automatic comment moderation and image validation. For comments, the models provide signals for detecting spam, threats, and toxicity. For images, they enable a safety classification before an article cover image or employee photo is accepted.

An artificial intelligence model is a set of operations and parameters obtained during training. When it receives a new input, such as a comment or an image, it generates a prediction or one or more scores. The process of using an already trained model to analyze an input is called inference.

The models expected by TRAMA's scripts use the ONNX (Open Neural Network Exchange) format. ONNX provides a way to represent machine-learning models so they can be run by different tools. When those files are installed, TRAMA uses ONNX Runtime from Node.js to load the `.onnx` files and obtain their results locally.

```
Comment -> tokenization -> ONNX model -> scores -> TRAMA rules -> approved / pending / rejected
```

Models that analyze text also require a tokenizer, which converts the comment into a numerical representation the model can process. After inference, TRAMA's scripts interpret the scores and combine them with project-specific rules. The models provide signals; the project's logic makes the final decision. The scripts expect to find the models and their supporting files inside `resources/models`. To run inference, those folders must contain the model files and their corresponding supporting files.

Visual Styles

CSS is the language that defines the visual appearance of a page. The interface uses CSS written specifically for the project. The main stylesheet is `resources/css/app.css`, and the dashboard has dedicated files inside `resources/css/admin/`, including `article-form.css`, `comments.css`, `advertisements.css`, `categories.css`, `tags.css`, `users.css`, and `editorial-listing.css`.

Step by Step: Carlos Changes a Screen and Each Technology Does Its Job

The simplest way to understand the stack is to follow a small change from the code to the browser.

Step	What the Person Does and What Happens Internally
1	Carlos opens the repository. Composer describes Laravel's PHP dependencies, while NPM describes the JavaScript dependencies used by Vue, Inertia, Vite, and the Node.js scripts.
2	He starts Laravel and Vite. Laravel handles the routes, and Vite serves the frontend assets during development.
3	Carlos modifies a .vue component. Vite recompiles the affected module and the browser receives the change without manually rebuilding the entire project.
4	When a screen needs data, Inertia allows Laravel to respond with props instead of splitting the system into an independent API and a separate application.
5	Eloquent queries or modifies MySQL when an operation requires persistent data.
6	Node.js is used for specific tasks: SSR, image processing, and local execution of moderation or safety models.
7	The visible result is a single application, but each technology handles a different responsibility.

3. General Architecture

TRAMA is organized so that each part of the system has a clear responsibility. Laravel controls routing, access, validation, and server-side logic; MySQL stores the data; Inertia connects the backend to the interface; and Vue renders the screens used by the user. This separation prevents routes, permissions, queries, and presentation from being concentrated in a single file.

For example, when an editor opens Articles, the request enters through a Laravel route. Middleware checks that the account may access the dashboard, the controller prepares the listing, MySQL returns the articles, and Inertia passes that data to Vue to build the grid. If the editor then performs an operation that requires several steps, the controller can delegate it to a Service instead of concentrating all the logic in a single class.

That path is repeated with variations across many screens. routes/web.php identifies the requested route; middleware applies shared rules; Requests validate input data; Controllers and Services coordinate the operation; Models/Eloquent work with MySQL; and Resources prepare the information that must reach the interface. The table summarizes the role of each piece.

Not all work finishes within the request the browser is waiting for. Comment moderation is sent to a queue so that a worker can process it in the background; Laravel Scheduler checks periodic tasks such as scheduled publications; and Node.js is involved in specific processes such as SSR, image processing, and local model execution. The following table summarizes the main pieces of this architecture.

Before Continuing: What an HTTP Request, a Route, and a Response Are

When Candela clicks an article, the browser needs to ask the server for something. That communication takes place through HTTP. An HTTP request is the message the browser sends to Laravel to request a page or perform an action. The request includes a URL and a method indicating what kind of operation is being attempted.

Laravel compares that method-and-URL combination with the routes defined by the project. A route is the rule that associates a URL pattern with the code that must handle it. If a match is found, the request continues through the middleware and reaches the corresponding controller. The result Laravel returns to the browser is called an HTTP response.

Example: GET /noticias/congreso-acelera-el-debate-por-la-reforma-digital means “retrieve this article.” By contrast, POST /noticias/{article}/comentarios represents sending data to create a user contribution, and PATCH is used when TRAMA partially updates an existing resource, as happens during autosave.

Method	Main Purpose in TRAMA
GET	Retrieve a page or data without creating a new record. Example: opening an article.
POST	Send data to create something or perform an action. Example: registering an account or creating a comment.
PATCH	Update part of an existing record. Example: autosaving an article.
DELETE	Request deletion of a record when the operation is permitted.

The term endpoint also appears in this documentation. In this context, it simply means an HTTP access point in the system: a route prepared to receive a specific request. The term payload refers to the set of data carried within that request.

Form Request and Resource: Controlling What Comes In and What Goes Out

Once the request reaches Laravel, TRAMA does not automatically pass all data to the controller. Many operations use classes that extend FormRequest. A Form Request is a Laravel class designed to authorize, prepare, and validate input data before the Controller works with it.

For example, ArticleRequest can check whether the account has permission to edit the article, normalize text, and apply different rules depending on the requested editorial state. If validation fails, the controller does not continue with the operation and Laravel returns the errors to the Vue form.

Another piece appears on output: Laravel Resources. A Resource receives data that has already been retrieved by Eloquent and prepares the exact shape that should reach the interface. It does not represent another table or replace the Model; it transforms and limits the information that is sent, for example so that a public card receives only the fields it needs.

Step	What the Person Does and What Happens Internally
1	Candela requests an article through a URL. routes/web.php recognizes the route.
2	Middleware applies shared rules before the request reaches the controller.
3	If the operation receives data, a Form Request can authorize, normalize, and validate it.
4	The Controller coordinates the operation and can delegate queries or rules to Services.
5	Eloquent queries or modifies MySQL.
6	A Resource can transform the data before passing it to Inertia and Vue.
7	Laravel returns the HTTP response, and the browser displays the result.
Project Component	Responsibility
routes/web.php	Associates each site URL with the controller that must handle it and with the access controls that run beforehand.
Middleware	Classes that Laravel runs before or after the controller to apply shared rules, such as checking that an account is still active.

Step	What the Person Does and What Happens Internally
Controllers	Receive the request and decide which operation must be performed. If the task requires several steps, they call other classes responsible for that operation.
Validation Requests	Check the received data: lengths, formats, required fields, permissions, and form conditions.
Services	Group operations that require several steps or several tables, such as an article's editorial workflow.
Models / Eloquent	Represent MySQL data and its relationships from PHP.
Resources	Prepare the information sent to the interface so that fields a screen does not need are not exposed.
Jobs / Commands	Represent work that can be executed later or from the console.

Step by Step: Candela Opens an Article and Moves Through the Architecture

This walkthrough shows how the layers work together without turning the controller into a class that does everything.

Step	What the Person Does and What Happens Internally
1	Candela selects an article and the browser requests a public Laravel URL.
2	routes/web.php identifies the route and applicable middleware before running the controller.
3	ArticleController receives the request and uses query services to obtain a publicly visible article.
4	Eloquent queries MySQL and loads relationships such as author, category, and tags.
5	Resources prepare only the fields required by the public interface.
6	TramaBridge converts that result into an Inertia response and adds any applicable shared data.
7	Trama.vue resolves the specific screen and ArticleShow.vue renders the article. The same architecture is repeated with variations in other project features.

4. Project Structure

TRAMA starts from Laravel's standard structure and adds project-specific folders for the editorial workflow, the Vue interface, Node.js processes, and the intended location of the artificial intelligence models used for moderation and image safety. The following tree shows the folders that matter for understanding the project and leaves out installed dependencies and temporary files that do not help explain how it works.

General Organization

```

TRAMA/
├── app/
│   ├── Actions/Fortify/
│   ├── Console/Commands/
│   └── Http/

```

```

├── Controllers/{Admin, Auth, Public}/
├── Middleware/
├── Requests/{Admin, Editorial}/
├── Resources/
├── Responses/
├── Jobs/
├── Logging/
├── Mail/
├── Models/
├── Notifications/
├── Observers/
├── Providers/
├── Rules/
├── Services/{Categories, Editorial, Images}/
├── Support/
├── config/
├── database/
│   ├── migrations/
│   ├── seeders/
│   │   ├── data/
│   │   └── Support/
├── public/images/{articles, banners, brand, categories, team}/
├── resources/
│   ├── css/admin/
│   ├── js/
│   │   ├── Components/
│   │   ├── Layouts/
│   │   ├── Pages/{Admin, Auth, Public}/
│   │   ├── Support/
│   │   ├── app.js
│   │   └── ssr.js
│   ├── models/
│   │   ├── comment-spam/
│   │   ├── comment-threat-detoxify/
│   │   ├── comment-toxicity/
│   │   └── image-safety/
│   └── views/{emails, errors}/
├── routes/
├── scripts/{comments, images}/
├── storage/
├── tests/{Feature, Unit}/
├── composer.json
├── package.json
└── vite.config.js

```

How to Navigate the Project

Area	What It Contains and When to Look There
Laravel Backend	app/Http, app/Services, and app/Models contain entry routes, validation, operation coordination, and data access.
Vue Interface	resources/js contains the layouts, components, and screens that Inertia displays in the browser.
Styles	resources/css contains the general stylesheet and the styles specific to the editorial dashboard.
Node.js Processes	scripts/comments and scripts/images run moderation and image processing outside the browser.

Area	What It Contains and When to Look There
Local Models	resources/models contains the folders where the scripts expect to find ONNX models and their supporting files.
Database and Test Data	database/migrations defines the structure; database/seeder loads and verifies the test-data scenario.
Tests	tests contains the automated checks for TRAMA's behavior.

Components That Reappear Throughout the Project

You do not need to memorize every class in the repository. These are the components that reappear in several sections because they connect important parts of the system.

Component	Location	Why It Matters
ArticleWorkflowService	app/Services/Editorial	Coordinates state changes and editorial-workflow operations.
ModerateComment	app/Jobs	Takes comments from the queue and coordinates their automatic moderation.
TramaBridge	app/Support	Connects Laravel to Inertia's root page.
TramaClock	app/Support	Maintains the editorial date for the test scenario.
OperationFailureHandler	app/Support	Records unexpected failures and returns safe messages to the interface.
Trama.vue	resources/js/Pages	Selects the Vue screen specified by Laravel through Inertia.
ssr.js	resources/js	Starts Vue SSR rendering for the homepage.

Intended Location of Local Models

Inside resources/models, TRAMA reserves four folders for the models involved in automatic comment review and image safety. Each folder corresponds to a different function, and the Node.js scripts look there for the model they need when a check is performed. To run these checks, the folders must contain the ONNX files and supporting files required by each model.

Folder	Intended Function in TRAMA
comment-spam	Analyzes comments for signs of spam, promotions, offers, and calls to action.
comment-threat-detoxify	Analyzes possible threats and provides an additional signal for the moderation context rules.
comment-toxicity	Evaluates toxicity, attacks, and insults in comments.

Folder	Intended Function in TRAMA
image-safety	Analyzes images to determine whether their content meets the safety rules before they are accepted.

Nombre	Fecha de modificación	Tipo	Tamaño
comment-spam	20/8/2026 15:54	Carpeta de archivos	
comment-threat-detoxify	20/8/2026 15:54	Carpeta de archivos	
comment-toxicity	20/8/2026 15:54	Carpeta de archivos	
image-safety	20/8/2026 15:54	Carpeta de archivos	

Folders reserved for local models inside resources/models.

To work, a text model needs the ONNX file and supporting files that describe its configuration and how text is converted into tokens. For example, comment-threat-detoxify expects a structure like the following when its files are installed:

Required File	Function
model.quant.onnx	Quantized ONNX model loaded by ONNX Runtime to perform inference.
tokenizer.json	Defines how text is split and transformed into tokens that the model can process.
tokenizer_config.json	Stores additional parameters used when initializing the tokenizer.
config.json	Describes general parameters and metadata required to interpret the model.

Nombre	Fecha de modificación	Tipo	Tamaño
config.json	18/8/2026 17:44	Archivo JSON	2 KB
model.quant.onnx	18/8/2026 17:44	Archivo ONNX	272.272 KB
tokenizer.json	18/8/2026 17:44	Archivo JSON	16.683 KB
tokenizer_config.json	18/8/2026 17:44	Archivo JSON	2 KB

Example of files required by comment-threat-detoxify when the model is installed.

Frontend Entry and Navigation Files

In addition to the main folders, four components help explain how a Laravel response enters the Vue application and how the public interface remains in place during Inertia navigation.

File	Function in TRAMA
resources/views/app.blade.php	Base HTML document where Inertia inserts the application. On the homepage it also receives the HTML generated by SSR.
resources/js/inertia-pages.js	Inertia resolver. It accepts only the Trama root page and assigns it the persistent layout.

File	Function in TRAMA
resources/js/Layouts/PersistentInertiaLayout.vue	Keeps the public layout mounted while navigating between Public/ screens and allows Admin/, Auth/, and mock advertiser pages to pass through without that layout.
lang/es	Contains language strings used by Laravel for messages and validation when applicable.

Step by Step: Carlos Wants to Change a Rule and Knows Where to Look

The folder structure stops being an abstract list when it is used to solve a specific task.

Step	What the Person Does and What Happens Internally
1	Carlos wants to change a rule in the article form. He first looks at resources/js/Pages/Admin/Articles/Form.vue to understand what the interface shows and sends.
2	If the rule must prevent invalid data from reaching the system, he checks app/Http/Requests/Admin/ArticleRequest.php.
3	If the state workflow or the way an article is stored changes, he looks for ArticleWorkflowService inside app/Services/Editorial.
4	If he needs to understand columns or relationships, he checks the Article model and the migrations in database/migrations.
5	If the operation has its own route, he looks for its definition in routes/web.php and the corresponding controller in app/Http/Controllers.
6	If the change must be checked automatically, he locates or adds a test in tests/Feature or tests/Unit.
7	This means the correct folder is chosen by responsibility: interface, HTTP input, business rule, persistence, or testing.

5. Configuration Files and Environment Variables

TRAMA separates configuration from code so that the same project codebase can run on different computers. Values that vary by environment—such as the MySQL connection, SMTP mail, file paths, the Node.js executable, or the SSR server—are defined outside the classes and then organized by Laravel through its config/ files.

.env and .env.example

The .env file contains the values TRAMA uses on the computer where it runs. It defines, for example, how to connect to MySQL, which URL the application uses, how mail is sent, which file paths the project uses, where the Node.js executable is located, and how Laravel connects to the SSR server. It also contains sensitive values such as APP_KEY, DB_PASSWORD, and MAIL_PASSWORD.

The .env.example file is not the active configuration. It is the project's reference template: it shows which variables TRAMA expects and their format without including real passwords or keys. Each computer's .env can use different values for fields such as DB_PORT, DB_DATABASE, DB_USERNAME, SMTP credentials, or TRAMA_NODE_BINARY.

```

APP_NAME=TRAMA
APP_ENV=local
APP_KEY=
APP_DEBUG=true
APP_URL=http://127.0.0.1:8000
APP_TIMEZONE=America/Argentina/Buenos_Aires

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=<mysql-port>
DB_DATABASE=<database-name>
DB_USERNAME=<username>
DB_PASSWORD=<password>

SESSION_DRIVER=database
SESSION_LIFETIME=120
QUEUE_CONNECTION=database
CACHE_STORE=database

MAIL_MAILER=smtp
MAIL_HOST=<smtp-server>
MAIL_PORT=<smtp-port>
MAIL_USERNAME=<smtp-username>
MAIL_PASSWORD=<smtp-password>
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS="noticias@trama.local"
MAIL_FROM_NAME="TRAMA"
TRAMA_CONTACT_EMAIL=<contact-email>

TRAMA_REFERENCE_DATE=2026-07-19
TRAMA_REFERENCE_TIME=15:30:00
TRAMA_LOG_PATH=storage/logs
TRAMA_ARTICLE_COVERS_PATH=public/images/articles
TRAMA_BANNERS_PATH=public/images/banners
TRAMA_NODE_BINARY=<path-to-node-executable>

INERTIA_SSR_ENABLED=true
INERTIA_SSR_URL=http://127.0.0.1:13714
INERTIA_SSR_ENSURE_BUNDLE_EXISTS=true
INERTIA_SSR_BUNDLE=bootstrap/ssr/ssr.js
VITE_APP_NAME="{APP_NAME}"

```

Values written between `<>` in the example represent data that varies by computer. In the documented environment, sessions, queues, and cache use the database; article cover images and banners are stored inside the project; and `TRAMA_NODE_BINARY` must point to the Node.js executable available on that computer.

What Each Main Group of Variables Does

The previous block brings the main configuration together, but each variable has a different responsibility. The following are the groups that actually affect TRAMA's documented behavior. Redis, Memcached, and AWS options that also appear in `.env.example` are general Laravel alternatives and are not part of this project's primary local environment.

Group	Main Variables	What They Control in TRAMA
Application	APP_NAME, APP_ENV, APP_DEBUG, APP_URL, APP_TIMEZONE, APP_KEY	Identify the application and environment, define the URL and time zone, and control diagnostics and cryptographic features. APP_KEY is secret and is generated for each installation.

Group	Main Variables	What They Control in TRAMA
MySQL	DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD	Specify the engine, server, port, database, username, and password. config/database.php uses these values to build the connection used by Eloquent and Query Builder.
Sessions	SESSION_DRIVER, SESSION_LIFETIME	With SESSION_DRIVER=database, Laravel stores sessions in the sessions table. SESSION_LIFETIME defines how many minutes an inactive session may remain valid according to the framework configuration.
Queues	QUEUE_CONNECTION	With database, pending Jobs are stored in MySQL's queue infrastructure. ModerateComment uses this connection, and a worker consumes those jobs later.
Cache	CACHE_STORE	With database, Laravel uses the cache tables for temporary values and locks that do not need to be stored as permanent editorial information.
Email	MAIL_MAILER, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_ENCRYPTION, MAIL_FROM_ADDRESS, MAIL_FROM_NAME, TRAMA_CONTACT_EMAIL	Configure SMTP, the sender, and the internal Contact recipient. Registration, password recovery, and Contact use the configured mailer.
Editorial Clock	TRAMA_REFERENCE_DATE, TRAMA_REFERENCE_TIME	Set the starting point for TramaClock so the demonstration scenario remains consistent. They do not replace APP_TIMEZONE; they serve a different purpose.
Logs	LOG_CHANNEL, LOG_STACK, LOG_LEVEL, TRAMA_LOG_PATH	Control the logging infrastructure. TRAMA_LOG_PATH is used by the daily trama_unexpected channel explained in section 29.
Files	TRAMA_ARTICLE_COVERS_PATH, TRAMA_BANNERS_PATH, FILESYSTEM_DISK	Define the physical locations managed by the project and Laravel's default disk. config/trama.php normalizes the article-cover and banner folders.
Node.js	TRAMA_NODE_BINARY	Specifies the Node executable that PHP starts with Process::run() for comment moderation and image processing/classification.

Group	Main Variables	What They Control in TRAMA
SSR	INERTIA_SSR_ENABLED, INERTIA_SSR_URL, INERTIA_SSR_ENSURE_BUNDL E_EXISTS, INERTIA_SSR_BUNDLE	Enable SSR, specify where the Node process listens, and indicate which compiled bundle must exist. TRAMA applies SSR only to the homepage.
Frontend	VITE_APP_NAME	Vite exposes this value to client-side JavaScript and to the SSR entry point. VITE_* variables can end up in the browser and must never contain secrets.

A variable does not “do” anything simply by existing in `.env`. Laravel must read it from a `config/` file and turn it into an option that is then consumed by Controllers, Services, Jobs, Middleware, or the frontend. For that reason, it is better to follow the complete path than to memorize isolated names.

How Variables Reach the Code

Laravel loads environment variables when the application starts. In TRAMA, the files in `config/` use `env()` to turn those values into application configuration. The rest of the code then reads `config()` instead of depending directly on the `.env` file. This separation makes it possible to manage those values from configuration files, define defaults, and use Laravel's configuration cache.

```
.env
↓
config/*.php -> env('VARIABLE_NAME')
↓
config('group.key')
↓
Controllers / Services / Jobs / Middleware
```

For example, `config/trama.php` reads `TRAMA_NODE_BINARY` and creates the `trama.node_binary` key. `ModerateComment` and the image services then read `config('trama.node_binary')` without depending directly on the `.env` file. If configuration is cached, Laravel continues using the values stored in that cache until it is updated.

A cache is temporary storage that keeps a prepared result so unnecessary work does not have to be repeated. In this case, the configuration cache allows Laravel to reuse processed configuration. Therefore, if Carlos changes `.env` while that cache is active, it may be necessary to clear or rebuild it so the new value is read.

Configuration Files Worth Locating

File	What It Connects	Specific Use in TRAMA
<code>config/trama.php</code>	Editorial clock, image paths, Node.js, and SSR policy	Defines the project's own configuration.
<code>config/inertia.php</code>	<code>INERTIA_SSR_*</code>	Configures the URL and bundle used by the SSR process.
<code>config/database.php</code>	<code>DB_*</code>	Builds the MySQL connection used by Eloquent and Query Builder.
<code>config/mail.php</code>	<code>MAIL_*</code> and <code>TRAMA_CONTACT_EMAIL</code>	Configures SMTP and the internal Contact recipient.

File	What It Connects	Specific Use in TRAMA
config/logging.php	LOG_* and TRAMA_LOG_PATH	Configures general logs and the trama_unexpected channel.
config/fortify.php	guard, broker, and features	Defines the authentication foundation that TRAMA extends with its own flows.
vite.config.js	Client and SSR entry points	Connects Laravel, Vue, and the Vite build process.

DB_*, MAIL_*, SESSION_*, QUEUE_*, and other general options follow Laravel's standard configuration. TRAMA adds its own variables for the clock, physical paths, Node.js, Contact, and SSR. The fact that .env.example includes options such as Redis, Memcached, or AWS does not mean they are part of the current local environment; they remain available as framework alternatives.

Concrete Example: How TRAMA Builds the MySQL Connection

Carlos does not write the port, username, or password inside each Model. He defines them once in .env. config/database.php reads DB_CONNECTION and DB_* and prepares the mysql connection that is then reused by the entire persistence layer.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=<mysql-port>
DB_DATABASE=<database-name>
DB_USERNAME=<username>
DB_PASSWORD=<password>
↓
config/database.php
↓
mysql connection
↓
Eloquent / Query Builder
↓
MySQL
```

DB_HOST identifies the server where MySQL is listening; DB_PORT specifies that service's port; DB_DATABASE the database name; and DB_USERNAME/DB_PASSWORD the credentials. If any of those values does not match the actual installation, Laravel cannot open the connection even if the Models' code is correct.

Concrete Example: How Mail Configuration Reaches Contact

config/mail.php reads the MAIL_* variables to build the SMTP mailer. MAIL_FROM_ADDRESS and MAIL_FROM_NAME define the general sender, while TRAMA_CONTACT_EMAIL feeds the mail.contact_address key used by the public Contact form as its internal recipient.

```
MAIL_MAILER=smtp
MAIL_HOST=<smtp-server>
MAIL_PORT=<smtp-port>
MAIL_USERNAME=<smtp-username>
MAIL_PASSWORD=<smtp-password>
MAIL_ENCRYPTION=tls
TRAMA_CONTACT_EMAIL=<contact-email>
↓
config/mail.php
↓
SMTP mailer + mail.contact_address
↓
registration / recovery / Contact
```

Concrete Example: What the SSR Configuration Controls

The `INERTIA_SSR_*` variables do not start a server by themselves either. `config/inertia.php` uses them to determine whether SSR is enabled, where the Node process is located, and which compiled bundle must exist. The process is then started with `php artisan inertia:start-ssr`, as explained in sections 9 and 31.

```
INERTIA_SSR_ENABLED=true
INERTIA_SSR_URL=http://127.0.0.1:13714
INERTIA_SSR_ENSURE_BUNDLE_EXISTS=true
INERTIA_SSR_BUNDLE=bootstrap/ssr/ssr.js
↓
config/inertia.php
↓
Laravel knows the SSR status, URL, and bundle
↓
php artisan inertia:start-ssr
```

Concrete Example: How TRAMA Finds Node.js

`TRAMA_NODE_BINARY` shows the complete path without having to repeat the same pattern for every variable. The value is defined in `.env`, `config/trama.php` turns it into `trama.node_binary`, and the PHP classes read it when they need to launch the internal moderation or image scripts. This variable does not configure Vite or the SSR server; those processes have their own configuration.

```
TRAMA_NODE_BINARY=<path-to-node-executable>
↓
config/trama.php
↓
config('trama.node_binary')
↓
ModerateComment / image services
↓
Node.js scripts
```

Variables That Reach the Browser

Vite can expose variables whose names begin with `VITE_` to the frontend. In TRAMA, `VITE_APP_NAME` is read from `resources/js/app.js` and `resources/js/ssr.js` through `import.meta.env.VITE_APP_NAME`. This rule is important for security: a `VITE_` variable can end up inside the JavaScript delivered to the browser, so it must never contain passwords, `APP_KEY`, SMTP credentials, or database secrets.

Configuration Security

- `APP_KEY`, `DB_PASSWORD`, and `MAIL_PASSWORD` belong to server-side configuration and are not sent to the frontend.
- `APP_KEY` is the key Laravel uses for its cryptographic features. The code obtains it from configuration rather than hard-coding it inside a class.
- `APP_DEBUG` must remain disabled in production so that an exception does not expose internal information to a visitor.
- `VITE_*` variables can be included in the JavaScript received by the browser; therefore, they must contain only data that may safely be visible on the client.
- Paths such as `TRAMA_NODE_BINARY`, `TRAMA_ARTICLE_COVERS_PATH`, and `TRAMA_BANNERS_PATH` are adjusted through environment configuration rather than being written inside controllers or services.

The idea is simple: `.env` stores the configuration TRAMA uses in that environment, `config/` organizes it, and the application code reads it through `config()`. This allows the same codebase to run with different connections, paths, or services without modifying controllers, services, or jobs.

Checking That the SSR Bundle Exists

TRAMA includes an additional SSR safeguard. `INERTIA_SSR_ENSURE_BUNDLE_EXISTS` tells Inertia to verify that the compiled server file exists before trying to use it. This prevents requests from being sent to Node.js for rendering before the bundle has been generated.

```
INERTIA_SSR_ENSURE_BUNDLE_EXISTS=true
```

The source file remains `resources/js/ssr.js`. The bundle configured in `INERTIA_SSR_BUNDLE` is the compiled output that Node uses during server-side rendering.

Step by Step: Carlos Changes a Variable and Follows It into the Code

Environment variables are useful only when you understand how they move from `.env` into the classes that ultimately consume them.

Step	What the Person Does and What Happens Internally
1	Carlos defines a value in <code>.env</code> , for example <code>TRAMA_NODE_BINARY</code> or a MySQL credential. That file is not committed to Git.
2	<code>config/trama.php</code> , or the corresponding configuration file, reads the value through <code>env()</code> and turns it into a Laravel configuration option.
3	The rest of the code reads <code>config(...)</code> instead of reading <code>.env</code> directly again in every class.
4	When a class needs to start Node.js, it uses the already normalized configuration path. When Eloquent needs the database, Laravel uses the options from <code>config/database.php</code> .
5	If a variable is shared with the browser, it must pass explicitly through configuration or a prop designed to be public; secrets remain on the server.
6	In production, changing <code>.env</code> may require clearing or rebuilding the configuration cache so Laravel reads the correct values again.

6. MySQL Database: Tables and Relationships

TRAMA stores persistent information in MySQL. MySQL is a relational database: information is divided into tables, and those tables are connected through identifiers. This allows a user, an article, a category, or a comment to be stored separately without losing the relationships between them.

Basic Example: An Article, Its Author, and Its Category

```
users
id | name
2  | Mateo Ledesma

categories
id | name
1  | Política

articles
id | author_id | category_id | title
1  | 2         | 1           | Congreso acelera el debate por la reforma digital
```

Article 1 has `author_id = 2`, so its author is Mateo Ledesma, the row where `users.id = 2`. It also has `category_id = 1`, so it belongs to `Política`, the row where `categories.id = 1`. This example corresponds to data loaded by the project's seeders.

id is the primary key: it uniquely identifies a row. author_id and category_id are foreign keys: they store identifiers that point to rows in other tables. MySQL can verify that those references are valid.

Migrations: How the Structure Is Created

Laravel calls the files that describe how to create or modify the database structure migrations. They are located in database/migrations. A migration can create a table, add a column, create an index, or define a relationship. This makes it possible to rebuild the structure without creating every table manually in MySQL.

```
$table->foreignId('author_id')
    ->constrained('users')
    ->restrictOnDelete();
```

That fragment creates author_id, specifies that it must point to users, and establishes what should happen if someone attempts to delete the related user.

Current Tables

Group	Tables and Main Function
Users and Access	users · password_reset_tokens · sessions
Articles and Classification	articles · categories · tags · article_tag
Editorial History	article_revisions · article_review_feedback · media_assets
Comments and Moderation	comments · comment_likes · comment_reports · user_moderation_reviews
Article Views	article_views
Advertising	advertisements · advertisement_daily_metrics
Contact	contact_messages
TRAMA Clock	trama_clock_state
Background Jobs	jobs · failed_jobs · job_batches
Cache	cache · cache_locks
Migration Tracking	migrations

Internal Laravel Infrastructure

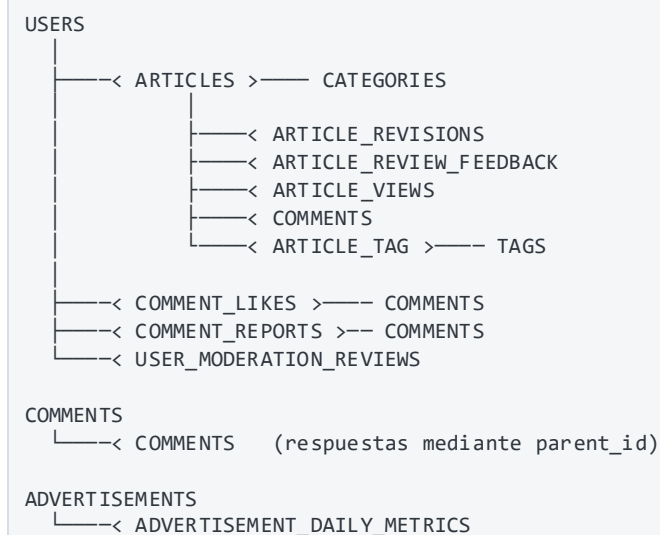
In addition to the tables that represent TRAMA's own content, Laravel maintains internal tables for sessions (sessions), password recovery (password_reset_tokens), queues (jobs, failed_jobs, and job_batches), cache (cache and cache_locks), and migration tracking (migrations). They support the site's operation but do not represent editorial information from the portal.

Most Important Domain Tables

Table	What It Stores
users	Public and internal accounts, role, profile, access status, and current administrative data.

Table	What It Stores
articles	The current state of each article: author, category, content, cover image, state, flags, views, and dates.
categories	News sections and their public data.
tags	Reusable topics and the reference to a primary tag after a merge.
article_tag	Each combination between an article and a tag.
article_revisions	Historical versions and actions recorded for an article.
article_review_feedback	Feedback left by an editor when returning an article for changes.
media_assets	Visual files related to articles and information about who uploaded them.
comments	Top-level comments and replies, author, and moderation state.
comment_likes	User likes on comments.
comment_reports	Comment reports and their resolution.
user_moderation_reviews	Cases in which an editor referred an account to an administrator.
article_views	Each recorded load of an article.
advertisements	Advertising campaigns.
advertisement_daily_metrics	Impressions and clicks by advertisement and date.
contact_messages	Messages submitted through Contact.
trama_clock_state	State required to continue TramaClock between sessions.

Simplified Relationship Map



Main Editorial Relationships

Relationship	What It Represents	On Delete
articles.author_id → users.id	Article author	RESTRICT
articles.category_id → categories.id	Article category	RESTRICT
article_tag.article_id → articles.id	Article associated with a tag	CASCADE
article_tag.tag_id → tags.id	Tag associated with an article	CASCADE
media_assets.article_id → articles.id	Visual file associated with an article	SET NULL
media_assets.uploaded_by → users.id	Account that uploaded the file	SET NULL
article_revisions.article_id → articles.id	Revisions of an article	CASCADE
article_revisions.user_id → users.id	Account that created the revision	RESTRICT
article_revisions.restored_from_revision_id → article_revisions.id	Revision used as the source of a restoration	SET NULL
article_review_feedback.article_id → articles.id	Returns for changes on an article	CASCADE
article_review_feedback.returned_by → users.id	Editor who returned the article	RESTRICT
article_review_feedback.resolved_by → users.id	Account that resolved the return	SET NULL
article_views.article_id → articles.id	Recorded view of an article	CASCADE
tags.merged_into_id → tags.id	Primary tag after a merge	SET NULL

Comment and Administration Relationships

Relationship	What It Represents	On Delete
comments.article_id → articles.id	Comment on an article	CASCADE
comments.user_id → users.id	Account that wrote the comment	SET NULL
comments.parent_id → comments.id	Reply to a top-level comment	CASCADE
comment_likes.comment_id → comments.id	Like on a comment	CASCADE
comment_likes.user_id → users.id	Account that gave the Like	CASCADE
comment_reports.comment_id → comments.id	Report on a comment	CASCADE
comment_reports.user_id → users.id	Account that submitted the report	CASCADE
comment_reports.reviewed_by → users.id	Editor who reviewed the report	SET NULL

Relationship	What It Represents	On Delete
user_moderation_reviews.user_id → users.id	Account reviewed by administration	CASCADE
user_moderation_reviews.requested_by_id → users.id	Editor who requested the review	SET NULL
user_moderation_reviews.comment_id → comments.id	Comment used as evidence	SET NULL
user_moderation_reviews.resolved_by_id → users.id	Administrator who resolved the case	SET NULL
users.moderation_review_requested_by_id → users.id	Most recent editor who requested an account review	SET NULL
users.moderation_review_comment_id → comments.id	Comment associated with the current administrative status	SET NULL
users.moderation_review_resolved_by_id → users.id	Administrator who resolved the current status	SET NULL
advertisement_daily_metrics.advertisement_id → advertisements.id	Metrics for a campaign	CASCADE

What RESTRICT, CASCADE, and SET NULL Mean

RESTRICT prevents a record from being deleted when doing so would break a relationship TRAMA needs to preserve. For example, a category that has articles cannot be deleted.

CASCADE deletes data that no longer makes sense on its own. If a comment is deleted, its Likes and reports are deleted as well.

SET NULL preserves the record but clears the reference. This makes it possible, for example, to keep a comment even if the registered account that wrote it has been deleted.

One-to-Many and Many-to-Many Relationships

A journalist can write many articles, but each article has only one author. That is a one-to-many relationship. By contrast, an article can have several tags and a tag can be used by many articles. That is a many-to-many relationship and requires the article_tag junction table.

Eloquent: Working with Those Relationships from PHP

MySQL remains the place where the data is stored. Eloquent is Laravel's tool for working with tables as PHP models and relationships. For example, Article knows its author through the following method:

```
public function author(): BelongsTo
{
    return $this->belongsTo(User::class, 'author_id');
}
```

The technical term for this type of tool is ORM, short for Object-Relational Mapping. Eloquent does not replace MySQL: it generates the required queries and returns the results as objects Laravel can use.

Eloquent and Query Builder: Two Ways to Work with MySQL

TRAMA primarily uses Eloquent when it needs to work with Models and relationships, for example Article, User, or Category. Laravel also provides Query Builder, a more direct tool for building operations on tables without necessarily loading a complete Model.

An expression such as `DB::table('articles')->where(...)->update(...)` belongs to Query Builder. TRAMA uses it, for example, when it is preferable to update an auxiliary column such as `search_text` without saving the entire Article model again. Both tools ultimately generate queries for MySQL; the difference is the level of abstraction used by the PHP code.

Indexes and Unique Constraints

MySQL can also speed up searches and prevent invalid combinations. `article_tag` prevents the same article-tag combination from being repeated. `comment_likes` prevents the same account from having two Like rows for the same comment. `comment_reports` applies an equivalent rule to reports. FULLTEXT speeds up the internal article search.

Data Prepared for Fast Queries

TRAMA stores some calculated copies to avoid repeating work in frequent queries. For example, `articles.views` stores the total needed by the interface while `article_views` keeps the detail of each load; `articles` stores the current version while `article_revisions` keeps previous versions; and `articles.search_text` stores text prepared for searching. In database design, deliberately storing a copy of this kind to solve a specific query is called controlled denormalization.

Step by Step: Valentina Creates an Article and MySQL Connects the Relationships

This example turns foreign keys and junction tables into an everyday CMS operation.

Step	What the Person Does and What Happens Internally
1	Valentina saves an article. The main row is inserted into <code>articles</code> and contains, among other data, <code>author_id</code> and <code>category_id</code> .
2	<code>author_id</code> points to <code>users.id</code> and identifies which internal account signed the article. <code>category_id</code> points to <code>categories.id</code> and determines its section.
3	If Valentina selects several tags, Laravel does not store a list inside <code>articles</code> ; it creates relationships in <code>article_tag</code> .
4	When a revision is recorded, <code>article_revisions</code> preserves the change without duplicating the project's entire structure in a single table.
5	If the article receives comments or views, those rows store <code>article_id</code> to link them to the same article.
6	Foreign keys prevent impossible references, while CASCADE, RESTRICT, and SET NULL rules define what happens when a related record is deleted or modified.
7	Eloquent allows PHP to traverse those relationships as objects and collections, even though MySQL still works with identifiers and keys underneath.

7. How Laravel, Inertia, and Vue Connect

Laravel and Vue perform different tasks within TRAMA. Laravel receives the request, checks permissions, validates data, queries MySQL, and decides what information the screen needs. Vue takes that information and builds the interface shown in the browser. Inertia connects the two: it allows a Laravel controller to deliver data to a Vue screen without creating a separate API for every page.

The Role of Each Component

Vue Concepts Worth Knowing Before Continuing

TRAMA uses Vue 3's Composition API. You do not need to know all of Vue to follow the project, but it helps to recognize several elements that repeatedly appear on screens: props, reactive state, computed values, watchers, and form bindings. The following definitions describe how they are actually used in this repository.

Component	What It Means in TRAMA	Real Example
<code>defineProps()</code>	Declares the data a component receives from outside.	<code>Trama.vue</code> receives <code>screen</code> and <code>screenProps</code> ; <code>Form.vue</code> receives <code>article</code> , <code>categories</code> , <code>tags</code> , and <code>permissions</code> .
<code>ref()</code>	Stores an individual reactive value that can change during interaction.	<code>Form.vue</code> uses <code>ref()</code> for the tag search field, modals, and autosave state.
<code>computed()</code>	Calculates a value from other reactive data and updates when its dependencies change.	<code>Trama.vue</code> computes <code>ActiveScreen</code> and <code>screenKey</code> ; <code>Login.vue</code> computes <code>submitDisabled</code> .
<code>watch()</code>	Runs a reaction when an observed value changes.	<code>Form.vue</code> watches <code>form.isDirty</code> to hide the previous saved message and also controls scroll locking when modals are open.
<code>v-model</code>	Keeps an HTML control synchronized with a reactive property.	<code>v-model="form.email"</code> writes directly to the login form; the article form does the same with the title, subtitle, and other fields.
<code>usePage()</code>	Provides access to Inertia's page object. <code>page.props</code> contains shared data and <code>page.url</code> contains the current URL.	<code>Trama.vue</code> reads <code>page.url</code> ; <code>Contact.vue</code> reads <code>page.props.flash</code> to detect the successful-send message.

```
const page = usePage();
const ActiveScreen = computed(() => screens[props.screen] || HomeScreen);
const screenKey = computed(() => `${props.screen}:${page.url}`);

<component :is="ActiveScreen" :key="screenKey" v-bind="screenProps" />
```

Do Not Confuse `screenProps` with Component Props

`screenProps` is the object `TramaBridge` passes to Inertia's root page. `Trama.vue` uses `v-bind="screenProps"` to turn its keys into props for the specific component. A prop is therefore the data ultimately received by that component; `screenProps` is the package that carries those values within TRAMA's own architecture.

TramaBridge.php: Preparing the Handoff from Laravel to Inertia

`TramaBridge` is a PHP backend class. It is not a Vue component and does not display anything by itself. A controller uses it once it already knows which screen it wants to open and what data that screen should receive.

```
return TramaBridge::render('Public/ArticleShow', [
    'article' => $articleData,
]);
```

In this example, `Public/ArticleShow` indicates which Vue screen should be displayed. The `$articleData` value contains the article and the other data Laravel has already retrieved and prepared. The call can be read as: “show the public article screen and give it this data.”

Trama.vue: The Component That Selects the Specific Screen

`resources/js/Pages/Trama.vue` is a Vue component. It acts as a common entry point for TRAMA's screens: it receives from Inertia the name of the screen requested by Laravel and selects the corresponding Vue component.

```
screen = Public/ArticleShow
```

```
Trama.vue looks for:
Public/ArticleShow.vue
```

Trama.vue does not itself represent an article, a form, or a listing. Its job is to decide which component to load for each response and pass it the received data.

Public/ArticleShow.vue: The Screen the Visitor Ultimately Sees

Public/ArticleShow.vue is the specific component that displays an article's details. It receives data prepared by Laravel and uses it to show the title, subtitle, body, author, category, tags, comments, views, and applicable advertising spaces.

```
TramaBridge.php
backend: indicates which screen and which data to send
    ↓
Inertia
transports that information between Laravel and Vue
    ↓
Trama.vue
frontend: decides which component to load
    ↓
Public/ArticleShow.vue
displays the article itself
```

Complete Example: From a Public URL to ArticleShow.vue

Suppose a visitor opens the article “Congress Speeds Up Debate on Digital Reform” at this URL:

```
/noticias/congreso-acelera-el-debate-por-la-reforma-digital
```

- 1. Laravel recognizes the URL.** `routes/web.php` recognizes the `/noticias/{slug}` pattern. The value received as slug is `congreso-acelera-el-debate-por-la-reforma-digital`, and the route sends the request to `ArticleController::show()`.
- 2. The controller delegates the lookup.** `ArticleController::show()` receives `ArticleQueryService` and calls `findPublishedBySlug()`. That service builds the query with Eloquent, applies public-visibility rules, looks up the slug in MySQL, and loads the main relationships—author, category, tags, and approved comments. This allows the controller to coordinate the request without concentrating the public query in the same class.

```
articles
├── author_id  ───> users
├── category_id ───> categories
├── article_tag ───> tags
└── comments
```

- 3. Laravel prepares the data.** After querying the database, the controller gathers the information it will deliver to the screen. To understand the flow, you can imagine content like the following:

```
$articleData = [
    'title' => 'Congreso acelera el debate por la reforma digital',
    'category' => 'Política',
    'author' => 'Mateo Ledesma',
    'views' => 325,
    // otros datos de la noticia
];
```

- 4. ArticleController calls TramaBridge.** The controller does not need to know how Trama.vue selects components. It only specifies the logical screen and the data it should receive.

```
return TramaBridge::render('Public/ArticleShow', [
    'article' => $articleData,
]);
```

What TramaBridge Does with That Information

TramaBridge transforms the previous call into the structure used by Inertia. TRAMA uses a single Inertia root page named Trama. The response therefore includes two properties: `screen` indicates which specific screen should be displayed, and `screenProps` contains the data that will be delivered to that screen.

```
Inertia::render('Trama', [
    'screen' => 'Public/ArticleShow',
    'screenProps' => [
        'article' => $articleData,
    ],
]);
```

`screen` is `Public/ArticleShow` and means “display the public details of an article.” `screenProps.article` contains the article data prepared by Laravel.

What Trama.vue Does When It Receives the Response

Trama.vue receives `screen` and `screenProps`. When `screen` is `Public/ArticleShow`, it selects `Public/ArticleShow.vue` and passes it the values contained in `screenProps`. Conceptually, the component performs an operation equivalent to the following:

```
<component
  :is="ActiveScreen"
  v-bind="screenProps"
/>
```

In simple terms: “show the component that corresponds to this screen and give it the data received from Laravel.”

Complete Flow

```
Visitor opens the URL
/noticias/congreso-acelera-el-debate-por-la-reforma-digital
↓
routes/web.php
↓
ArticleController::show()
↓
ArticleQueryService::findPublishedBySlug()
↓
Eloquent queries MySQL
↓
Laravel prepares $articleData
↓
TramaBridge.php
↓
Inertia sends screen + screenProps
↓
Trama.vue selects the component
↓
Public/ArticleShow.vue
↓
Article visible in the browser
```

Why TramaBridge Exists

Without TramaBridge, different controllers would have to repeat the same Inertia structure every time they wanted to display a screen. TRAMA uses this class so that the controller only specifies which screen it needs and what data it should receive; TramaBridge then builds the response in the format expected by Trama.vue.

Loading Screens Only When They Are Needed

TRAMA has many screens, and there is no need to download the JavaScript code for all of them when opening an article. For example, a visitor reading the portal does not need to load the administration dashboard code at that moment. Vue allows some components to be loaded only when they are going to be used; TRAMA performs that loading with `defineAsyncComponent()`.

Laravel Still Controls Routes and Permissions

TRAMA does not use Vue Router to define the site's routes. Routes remain defined in Laravel. If someone manually enters a private URL, the request still passes through `routes/web.php`, middleware, and the corresponding authorization checks. Hiding a button in Vue improves the interface, but it does not replace server-side permission checks.

Data Many Screens Receive Automatically

`HandleInertiaRequests` shares information that different screens routinely need, such as the authenticated account, temporary messages after an action, public categories, header banners, the routes Ziggy makes available to Vue, and the date TRAMA is using. Data specific to each screen remains inside `screenProps`.

Initial Load and Inertia Navigation

When someone enters the site through a full page load, the browser requests the document from Laravel. Laravel resolves the route, prepares the Inertia response, and delivers the base document that starts the Vue application. From that point on, `app.js` mounts or hydrates the application depending on the type of response received.

Subsequent navigation performed through Inertia does not need to rebuild the entire HTML document from scratch. Laravel remains responsible for routes, permissions, and the data for each screen, but Inertia retrieves the new response and Vue changes the active component within the application that is already loaded.

This distinction is important for understanding the next section: on most screens, Vue builds the interface in the browser; on the homepage, TRAMA adds SSR so that Vue can also generate the initial HTML from Node.js before the response reaches the browser.

Real Source Code for `TramaBridge::render()`

The bridge is not just an architectural idea: the `app/Support/TramaBridge.php` class centralizes the exact way Inertia responses are built. The method receives the logical screen and its data, limits SSR to the configured screen, and sends everything to the single Trama root page.

```
public static function render(string $screen, array $props = []): Response
{
    $ssrScreen = (string) config('trama.ssr.screen', 'Public/Home');

    config([
        'inertia.ssr.enabled' => (bool) config('inertia.ssr.enabled')
        && $screen === $ssrScreen,
    ]);

    return Inertia::render('Trama', [
        'screen' => $screen,
        'screenProps' => $props,
    ]);
}
```

`screen` is not a URL or a Vue Router route. It is a logical key agreed upon by Laravel and Trama.vue. `screenProps` contains only the screen-specific data already prepared by the controller.

The Single Inertia Resolver

resources/js/inertia-pages.js reinforces this architecture: Inertia accepts only the Trama root page. Public/, Admin/, and Auth/ screens are not registered as independent Inertia pages; Trama.vue resolves them internally.

```
export async function resolveInertiaPage(name) {
  if (name !== 'Trama') {
    throw new Error(`Página Inertia no reconocida: ${name}`);
  }

  const page = await import('./Pages/Trama.vue');
  page.default.layout = PersistentInertiaLayout;

  return page;
}
```

How Trama.vue Turns screen into a Component

Trama.vue maintains a closed map of screens. The lazy() function wraps defineAsyncComponent() to lazy-load screens that do not need to be part of the initial load. ActiveScreen then takes the component associated with screen, and screenKey forces a fresh mount when the URL changes.

```
const lazy = (loader) => defineAsyncComponent(loader);

const AdminEditorialLayout = lazy(() => import('@Layouts/AdminEditorialLayout.vue'));
const AuthNewsLayout = lazy(() => import('@Layouts/AuthNewsLayout.vue'));

const screens = Object.freeze({
  'Public/Home': Home,
  'Public/ArticleShow': lazy(() => import('./Public/ArticleShow.vue')),
  'Public/CategoryShow': CategoryShow,
  'Public/JournalistShow': lazy(() => import('./Public/JournalistShow.vue')),
  'Public/Search': lazy(() => import('./Public/Search.vue')),
});
```

Once the map has been defined, ActiveScreen selects the component associated with screen and screenKey changes with the URL. The following excerpt shows that selection without repeating the entire map.

```
const ActiveScreen = computed(
  () => screens[props.screen] || HomeScreen
);

const screenKey = computed(
  () => `${props.screen}:${page.url}`
);

<component
  :is="ActiveScreen"
  :key="screenKey"
  v-bind="screenProps"
/>
```

Persistent Public Layout

PersistentInertiaLayout.vue wraps Public/ screens with PublicNewsLayout and remains mounted during Inertia navigation that uses the same root page. This means the public header, navigation, and footer do not have to restart on every screen change. Admin/ and Auth/ screens are wrapped inside Trama.vue with their own layouts.

useForm: Vue Forms Connected to Laravel

useForm() is a function provided by Inertia's Vue adapter for working with forms. Its purpose is to bring together in a single object the data entered by the user and all the state required to send that data to the server. Instead of creating a separate variable for each field and manually managing the HTTP request, errors, and loading state, useForm() makes it possible to handle all of this from one object.

For example, a form can store values such as title, excerpt, or body, while also reporting whether it is being submitted (processing), exposing the validation errors returned by Laravel (errors), and providing operations to clear, transform, or reset data after a successful action.

TRAMA uses it to connect Vue forms to Laravel through Inertia. The user fills in fields in the interface, Vue keeps those values inside the object created with `useForm()`, Inertia sends them to Laravel, and if the server detects a validation error, those errors return to the same form so Vue can display them next to the corresponding field.

```
const form = useForm({
  email: '',
  password: '',
  remember: false,
  redirect: props.redirect || '',
});

form.post(route('login'), {
  onFinish: () => form.reset('password'),
});
```

Actual Element	How TRAMA Uses It
<code>form.processing</code>	Indicates that Inertia is submitting the form. It is used to disable buttons and display states such as “Signing in...” or “Saving...”.
<code>form.errors</code>	Contains errors returned by Laravel as well as local errors added with <code>setError()</code> .
<code>form.setError()</code> / <code>clearErrors()</code>	Allow some fields to be validated immediately in Vue without replacing the server's definitive validation.
<code>form.reset()</code>	Resets one or more fields. <code>Login.vue</code> , for example, clears the password when the attempt finishes.
<code>form.isDirty</code>	Indicates that the current values differ from the last saved reference. The article form uses it to detect pending changes.
<code>form.defaults()</code>	Turns the current values into the new saved reference. After a manual save, <code>Form.vue</code> uses it so that <code>isDirty</code> returns to false.
<code>form.transform()</code>	Transforms the payload before sending it. Article editing sends a POST with <code>_method="put"</code> to properly support <code>FormData</code> and files.

Do Not Confuse `useForm`, router, and Axios

`useForm` manages a form and its state. `router` performs Inertia navigation or actions that do not need to preserve a complete form. `Axios` is used for auxiliary, silent requests, such as article autosave. All three can communicate with Laravel, but they solve different needs.

Step by Step: Candela Navigates with Inertia Without Turning TRAMA into Two Separate Applications

This walkthrough shows what happens from a click until Vue changes screens.

Step	What the Person Does and What Happens Internally
1	Candela clicks a link generated by Inertia. On subsequent navigation, the browser does not need to download the entire HTML template again.
2	Laravel receives the normal URL and runs routes, middleware, controllers, and services exactly as it would for a traditional web request.

Step	What the Person Does and What Happens Internally
3	The controller calls <code>TramaBridge::render()</code> , specifying a logical screen name and the data specific to that screen.
4	Inertia returns a response containing the root component and props in a format understood by its Vue client.
5	<code>Trama.vue</code> receives screen and screenProps. A computed value looks up which actual component corresponds to that logical name.
6	Vue replaces the content area with the correct screen and preserves the persistent parts defined by the application.
7	The URL and security remain Laravel's responsibility; Vue handles the interface experience rather than replacing the server router.

8. The Date Used by TRAMA and TramaClock

TRAMA does not record its actions using the computer's current date directly. The project has its own clock so that articles, comments, advertisements, and other test data keep the dates for which they were prepared, even if the application is opened again weeks or months later.

The Project's Initial Date

The `.env` file defines that TRAMA starts on 19/07/2026 at 15:30. The computer may have a completely different date.

```
TRAMA_REFERENCE_DATE=2026-07-19
TRAMA_REFERENCE_TIME=15:30:00
```

`config/trama.php` reads those values and makes them available to the code. The `TramaClock` class uses that date and time as its starting point.

Example: The Real Date May Be Different

```
Computer's real date:
01/09/2026 10:20
```

```
Time TRAMA starts using:
19/07/2026 15:30
```

If an article or comment is created at that moment, TRAMA uses its own project time. It does not store 01/09/2026 simply because that is the computer's real date.

What TramaClock Does

`TramaClock` is the class responsible for calculating which date and time TRAMA considers current. The initial time does not remain frozen at 15:30: while the site is being used, the clock advances at essentially the same rate as real time.

```
TRAMA starts: 19/07/2026 15:30
10 minutes of use: 19/07/2026 15:40
20 minutes of use: 19/07/2026 15:50
```

What Happens When TRAMA Is Not Being Used

The important difference appears when there is no activity. Suppose the last action took place at 15:42:18 and the project is then closed for two days. TRAMA does not add those 48 hours to the project clock.

Last time used by TRAMA:

19/07/2026 15:42:18

The project remains closed for two days.

When it is used again:

approximately 19/07/2026 15:42:19

This way, leaving the project closed does not cause the test data to jump forward by several days automatically.

Why 30 Minutes of Inactivity Matter

TRAMA considers a period of use active as long as no more than 30 minutes pass without web requests. If the user navigates, saves an article, or performs another action before that limit, the clock keeps advancing. If more than 30 minutes pass without activity, that period is considered paused and the next request continues from the last stored time instead of adding the entire inactive period.

How the Clock Is Activated During a Request

All web requests pass through `AdvanceTramaClock`. It is middleware, meaning a class that Laravel automatically runs around a request to apply a shared rule. At the beginning of the request, it activates the clock; when the response finishes, it records the end of that activity.

```

The user opens an article
↓
Laravel receives the request
↓
AdvanceTramaClock
↓
TramaClock::activate()
↓
the controller runs
↓
the response is generated
↓
TramaClock::finishWebRequest()

```

Thanks to this middleware, `ArticleController`, `UserController`, and the other controllers do not have to advance the clock manually in every method.

Where the Last Time Is Remembered

TRAMA needs to remember how far its clock had advanced even if the server stops. It uses the MySQL `trama_clock_state` table for this purpose. The table stores the project time already reached, a real-clock value used only to measure how much time has elapsed, and the most recently recorded activity.

```

trama_clock_state
↓
last time used by TRAMA
last activity
real-clock reference used to measure the interval

```

Which Records Use This Time

The project's main models use a behavior called `UsesTramaClock`. When Laravel fills automatic fields such as `created_at` and `updated_at`, those models use `TramaClock` time instead of requesting the system's current date directly.

Computer's real date:

03/09/2026 09:15:00

TramaClock time:

19/07/2026 16:03:27

New comment → created_at = 19/07/2026 16:03:27

This behavior applies to articles, comments, revisions, categories, tags, advertisements, and other records that need to preserve the project's timeline.

What Happens If Two Requests Arrive at the Same Time

Two requests may try to update the clock state at nearly the same time. To prevent both from calculating and writing the same value out of order, Laravel briefly locks the `trama_clock_state` row while one of them finishes the update. The method used is `lockForUpdate()`.

```

Request A reads the clock
      ↓
temporarily locks the row
      ↓
updates trama_clock_state
      ↓
releases the row
      ↓
Request B can continue

```

TramaClock and Scheduler Are Not the Same Thing

Important Difference

TramaClock determines which date and time TRAMA is using.

Laravel Scheduler runs periodic tasks, for example checking whether a scheduled article is due to be published.

TramaClock works whenever the site is used even if Scheduler is not active. Scheduler can read TramaClock time to decide whether a scheduled task is due, but it is not what advances the clock.

Current Clock Limit

The current version of TramaClock is designed to remain within 19/07/2026. If it reaches the end of that day, `TramaClock::clamp()` stops it at 23:59:59 and does not allow it to continue into 20/07/2026.

Dates After 19/07/2026

Scheduled-article validation allows dates up to 30 days after the base date, but TramaClock cannot advance to the following day. Therefore, an article scheduled for 20/07/2026 or a later date may be accepted by the form even though the current clock cannot automatically reach that point.

Step by Step: Carlos Reopens TRAMA and TramaClock Continues Without Jumping an Entire Day

The editorial clock exists so that the demonstration scenario maintains a consistent chronology even when the project is opened on different real-world dates.

Step	What the Person Does and What Happens Internally
1	Carlos opens the portal, and a web request runs the middleware that activates TramaClock.
2	On its first activation, TramaClock starts from the configured reference date and time and creates the persistent state in <code>trama_clock_state</code> .
3	While there is activity, it calculates how much real time has elapsed since the anchor and advances editorial time by the same number of seconds.

Step	What the Person Does and What Happens Internally
4	If more than 30 minutes pass without activity, the next session does not add the entire real-world pause. It continues one second after the last editorial instant reached.
5	The state update runs inside a transaction and uses <code>lockForUpdate()</code> so that two simultaneous requests cannot move the clock backward.
6	Controllers and commands read <code>TramaClock::now()</code> or <code>today()</code> so publications, views, and other time-based records use the same editorial timeline.

9. Server-Side Rendering (SSR) and Hydration

TRAMA's homepage uses Server-Side Rendering (SSR) so its initial HTML can be generated with Vue before it reaches the browser. To understand the implementation, it helps to separate three moments: Laravel prepares the data, Node.js generates the initial HTML, and finally Vue activates that interface in the browser through hydration.

What Server-Side Rendering (SSR) Is

Server-Side Rendering means rendering on the server. On a Vue screen without SSR, Laravel prepares the Inertia response and the browser downloads JavaScript; Vue then builds the interface there. With SSR, an additional step appears: before the final response is sent, Vue also runs in Node.js and turns the components into HTML.

In TRAMA, this is deliberately used only for Public/Home. Laravel remains the main server: it resolves GET /, queries MySQL, applies business rules, and prepares the homepage sections. Node.js receives an already assembled Inertia page and converts it to HTML; it does not select articles, query the database, or replace Laravel.

Normal Screen vs. Homepage with SSR

Normal Inertia/Vue Screen	Public/Home Homepage with SSR
Laravel prepares the data and the Inertia page.	Laravel prepares the same types of data and the Inertia page.
The browser receives the application without pre-rendered Vue HTML.	The page can first be sent to the Node.js SSR process.
<code>resources/js/app.js</code> uses <code>createApp()</code> .	<code>resources/js/ssr.js</code> uses <code>createSSRApp()</code> and <code>renderToString()</code> .
Vue builds the interface in the browser.	Node generates the initial HTML and Laravel incorporates it into the response.
There is no hydration of existing Vue HTML.	<code>app.js</code> recognizes the existing HTML and Vue hydrates it.

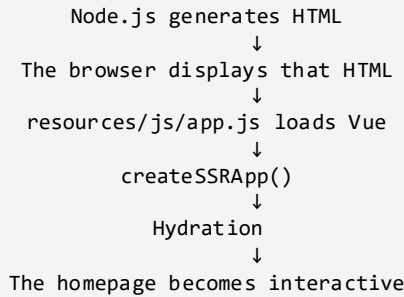
What Hydration Is

The HTML produced by SSR can display the header, navigation, main article, and other blocks from the first moment, but that HTML is not yet connected to all of Vue's reactive logic. Hydration is the process by which Vue takes that existing structure and associates it with its components, state, and events.

Key Idea

Hydration does not mean drawing the entire homepage again. Vue starts from the HTML it has already received and

connects it to the JavaScript application. A button may be visible before hydration; afterward, it is connected to its click event and the component's reactive state.



One Interface, Two Execution Moments

TRAMA does not maintain one homepage for the server and another for the browser. `Public/Home.vue` is part of the same application. During SSR it can run in Node.js to produce HTML, and afterward the browser continues working with that interface through hydration.

How `app.js` Chooses Between Mounting and Hydrating

The browser entry point inspects the container generated by Inertia. If it already contains HTML, the response was pre-rendered and `createSSRApp()` is used; if it is empty, a normal mount is performed with `createApp()`.

```

setup({ el, App, props, plugin }) {
  const createVueApp = el.innerHTML.trim()
    ? createSSRApp
    : createApp;

  return createVueApp({
    render: () => h(App, props),
  })
    .use(plugin)
    .use(ZiggyVue)
    .mount(el);
}

```

Complete Homepage Flow with SSR

Step 1. The browser requests the homepage. `GET /` is performed. `routes/web.php` sends the request to `HomeController`.

Step 2. Laravel prepares the data. `HomeController` queries the sections through `ArticleQueryService` and prepares the information required by `Public/Home`.

Step 3. The controller uses the bridge. `HomeController` calls `TramaBridge::render('Public/Home', ...)` with the already prepared data.

Step 4. SSR safeguards are applied. `HandleInertiaRequests` checks the home route, and `TramaBridge` checks again that the logical screen is `Public/Home`.

Step 5. Inertia builds the page. The root is `Trama`; screen is `Public/Home`, and `screenProps` contains the specific data. Shared props, including `Ziggy`, are also added.

Step 6. Node receives the page. The process started with `php artisan inertia:start-ssr` runs the compiled SSR bundle.

Step 7. Vue runs on the server. `resources/js/ssr.js` creates the application with `createSSRApp()`; `Trama.vue` resolves `Public/Home.vue`.

Step 8. HTML is generated. `renderToString()` converts the Vue tree into an HTML string.

Step 9. Laravel responds. The HTML is inserted into the base `resources/views/app.blade.php` document together with the Inertia props and JavaScript/CSS assets.

Step 10. Vue hydrates. When resources/js/app.js finishes loading, it detects the existing HTML and activates interaction through createSSRApp().

What ssr.js Does

resources/js/ssr.js is the Vue entry point used by Node. createServer() receives the Inertia page, renderToString() produces the HTML, and createSSRApp() creates the server application. mount() is not called because Node does not have a real DOM element.

```
createServer((page) => createInertiaApp({
  page,
  render: renderToString,
  resolve: resolveInertiaPage,
  setup({ App, props, plugin }) {
    return createSSRApp({ render: () => h(App, props) })
      .use(plugin)
      .use(ZiggyVue, ziggy);
  },
}));
```

Why Only the Homepage Uses SSR

The homepage is the site's main entry page and is where TRAMA takes advantage of server-generated initial HTML. Login, the editorial dashboard, and the other Inertia/Vue pages use normal mounting in the browser. Independent Blade views, such as the MySQL fallback page or mock advertiser pages, follow a different flow.

Two Safeguards That Limit SSR

INERTIA_SSR_ENABLED is not the only control. HandleInertiaRequests restricts SSR to the route named home. TramaBridge then compares the logical screen with Public/Home. The second check prevents a different response accidentally produced within / from inheriting SSR simply because it is on that route.

Request	Check	Result
GET /	home route + Public/Home screen	May use SSR if the general configuration is enabled.
GET /login	The route is not home	Normal Vue mounting in the browser.
GET /noticias/{slug}	The route is not home	An individual article does not use the homepage's selective SSR.
Another response within /	home route, but a different screen	TramaBridge disables SSR for that response.

The Role of app.blade.php

resources/views/app.blade.php is the base HTML document for the Inertia application. It does not query articles and does not replace Public/Home.vue. It contains the point where Inertia inserts the application and, when appropriate, receives the HTML generated by Vue before the response reaches the browser. It also includes critical CSS and preloads for important images to keep the first paint stable while the full assets finish loading.

Source Code, Bundle, and SSR Server

```
resources/js/ssr.js
  ↓ npm run build
bootstrap/ssr/ssr.js
  ↓ php artisan inertia:start-ssr
Inertia / Node.js SSR server
```

↓
Initial HTML for Public/Home

Building and running are separate operations. `npm run build` generates both the client assets and the server bundle. `php artisan inertia:start-ssr` starts the process that uses that bundle to handle rendering requests.

How SSR Is Started in TRAMA

Setting `INERTIA_SSR_ENABLED=true` allows Laravel to use SSR when the request corresponds to Public/Home, but that variable does not start any process by itself. For Node to render the homepage, the SSR server must be started from a terminal located at the project root.

```
php artisan inertia:start-ssr
```

The terminal must remain open while SSR is being used. This command runs the bundle configured in `INERTIA_SSR_BUNDLE`; it does not compile `resources/js/ssr.js`. If `bootstrap/ssr/ssr.js` does not exist or code involved in SSR changes, `npm run dev` does not rebuild that bundle: `npm run build:ssr` must be run and `php artisan inertia:start-ssr` restarted so Node uses the updated version.

Enabling, building, and running are different things

`INERTIA_SSR_ENABLED=true` enables SSR in the configuration. `npm run build:ssr` generates the bundle when it is missing or must be rebuilt. `php artisan inertia:start-ssr` is the command that actually starts the SSR server.

SSR Variables Used by TRAMA

```
INERTIA_SSR_ENABLED=true
INERTIA_SSR_URL=http://127.0.0.1:13714
INERTIA_SSR_ENSURE_BUNDLE_EXISTS=true
INERTIA_SSR_BUNDLE=bootstrap/ssr/ssr.js
```

Variable	Function
<code>INERTIA_SSR_ENABLED</code>	General SSR switch. Route and screen safeguards still limit its scope.
<code>INERTIA_SSR_URL</code>	Address where Laravel contacts the SSR process.
<code>INERTIA_SSR_ENSURE_BUNDLE_EXISTS</code>	Requires the bundle to exist before SSR is attempted.
<code>INERTIA_SSR_BUNDLE</code>	Path to the compiled file used by Node; this is not the <code>resources/js/ssr.js</code> source file.

Step by Step: Candela Opens the Homepage with SSR and Then Vue Takes Control

The first load and subsequent navigation use the same Vue screens, but they do not start from exactly the same place.

Step	What the Person Does and What Happens Internally
1	Candela enters the homepage URL. Laravel resolves Public/Home and prepares the page props.
2	Because the homepage is one of the screens enabled for SSR, Inertia sends that data to the Node process running the bundle generated from <code>resources/js/ssr.js</code> .
3	Node creates the Vue application on the server and returns pre-rendered HTML for the initial response.

Step	What the Person Does and What Happens Internally
4	The browser receives visible content before running the client-side JavaScript.
5	app.js detects that generated HTML already exists and uses hydration instead of mounting an empty interface from scratch.
6	Vue connects events, state, and components to that HTML without unnecessarily replacing the entire initial structure.
7	When Candela later navigates within the site, Inertia can resolve new screens through client-side navigation without always repeating the same SSR process.

10. Public Homepage: How Articles Are Selected and Organized

The homepage does not receive a single list of articles and leave Vue to decide what to do with it. Laravel builds each editorial block separately. GET / reaches HomeController, which requests the sections from ArticleQueryService, transforms the models through Resources, and passes Public/Home.vue separate properties for the main article, breaking articles, featured articles, latest articles, most-read articles, categories, and category blocks.

```

GET /
  ↓
HomeController
  ↓
ArticleQueryService::getHomePageSections()
  ↓
ArticleCardResource / CategoryResource
  ↓
TramaBridge::render('Public/Home')
  ↓
Public/Home.vue

```

Seven-Day Recency Window

To occupy homepage positions, an article must still be publicly visible and must have been published between the start of the day seven days ago and the current editorial time. This rule does not unpublish the article: once it moves beyond that window, it remains accessible through its URL, search, or other pages, but it no longer competes for the main article, breaking-news strip, featured articles, latest articles, most-read articles, or recent category blocks.

Block	How It Is Selected	Maximum Count
Main Article (Hero)	First recent article marked is_featured=true, ordered by published_at descending.	1
Breaking	Recent articles with is_breaking=true.	4
Featured	Recent articles with is_featured=true, excluding the main article.	6
Latest	Recent articles ordered by publication date.	8
Most Read	Same recent window, reordered by views from highest to lowest.	8

Block	How It Is Selected	Maximum Count
Category Blocks	Public categories with recent content; within each one, the newest articles are selected.	4 per category

Example: What Appears on the Homepage and What Does Not

Suppose TramaClock reads 19/07/2026 15:30. An article published on July 18 and marked Featured can become the main article. Another article from July 19 marked Breaking can appear in the top strip. An article published on July 10 is still a valid published article, but it is now outside the homepage window and is no longer eligible for these blocks. If the same article is breaking, featured, and also among the most read, Laravel may send it in more than one collection because each block follows a different editorial rule.

Public/Home.vue prevents duplicates within the moving strip by using a Set of IDs. The strip combines an Economy/Markets reference, breaking articles, latest articles, and most-read articles; if an article has already been added, it does not appear again in that same cycle. When an article has `is_breaking=true`, the Breaking label takes priority over any secondary label.

Real Source Code: How Laravel Builds the Homepage Blocks

The rules above are not applied in Vue. `app/Services/ArticleQueryService.php` builds a base query of visible, recent articles and then clones it for each block. The following excerpt is real project code; comments are omitted so the explanation can focus on the query:

```
$base = Article::query()
    ->publiclyVisible()
    ->where('published_at', '>=', $frontPageStartsAt)
    ->with(['author', 'category', 'tags'])
    ->latest('published_at');

return [
    'hero' => (clone $base)->where('is_featured', true)->first(),
    'breaking' => (clone $base)->where('is_breaking', true)->take(4)->get(),
    'featured' => (clone $base)->where('is_featured', true)->skip(1)->take(6)->get(),
    'latest' => (clone $base)->take(8)->get(),
    'mostRead' => (clone $base)->reorder()->orderByDesc('views')->take(8)->get(),
];
```

`publiclyVisible()` applies the conditions required for the article and its category to be publicly displayed. `published_at >= $frontPageStartsAt` introduces the seven-day recency window. `clone` makes it possible to reuse the same base query without mixing the conditions of one block with another. In Most Read, `reorder()` removes the base query's date ordering and replaces it with views descending.

Step by Step: Candela Opens the Homepage and Laravel Decides Which Article Occupies Each Block

The homepage is not a static list: it is assembled from editorial state, publication date, and the flags configured by the team.

Step	What the Person Does and What Happens Internally
1	Candela opens the homepage and HomeController requests the required public sections from ArticleQueryService.
2	The service uses TramaClock as its reference and calculates the start of the seven-day recency window.
3	The base query includes only published articles with a public category and a date within that recent window.

Step	What the Person Does and What Happens Internally
4	The main article comes from featured articles; the breaking strip uses <code>is_breaking</code> ; other blocks take the remaining featured articles, latest articles, and most-read articles.
5	The readership ranking orders by views, while recency blocks normally prioritize <code>published_at</code> .
6	Public categories are loaded in editorial order, and each can contribute a small sample of recent articles.
7	TramaBridge sends the complete set to <code>Public/Home.vue</code> , and the homepage renders each block without Vue having to decide the editorial rules itself.

11. Registration, Authentication, Account Recovery, and Sessions

Authentication means verifying who is trying to sign in. Authorization means deciding what that account may do after it has signed in. Laravel Fortify provides the foundation for login and password recovery; TRAMA adds registration, email verification, account blocking, and other project-specific rules.

Service Provider: Configuration Laravel Registers at Startup

A Service Provider is a class Laravel uses during application startup to register or configure behavior that must be available to the rest of the system. It does not represent a screen or a table; it prepares global services and rules before user requests arrive.

TRAMA uses `FortifyServiceProvider` to customize parts of authentication and password recovery. `AppServiceProvider`, in turn, registers general application configuration, including the Observer that keeps the editorial order of categories synchronized when relevant articles change.

Example: Registering an Account

Step by Step: Candela Creates an Account and Verifies Her Email

The screenshot shows a dark-themed registration page for TRAMA. On the left, the TRAMA logo is at the top, followed by the text "Noticias con criterio, datos y comunidad activa." and a link "Ingresá para comentar noticias." On the right, there's a "Crear cuenta" section. It starts with a link "← Volver a portada" and a heading "CUENTA TRAMA". Below is the "Crear cuenta" heading and a subtext "Creá tu cuenta para comentar noticias." The form has four fields: "Nombre" (with value "usuario_demo"), "Email" (with value "usuario@ejemplo.com"), "Contraseña" (with masked characters), and "Confirmar contraseña" (with masked characters). Each field has a small error message to its right: "El nombre debe tener entre 3 y 255 caracteres", "El email debe tener entre 3 y 255 caracteres", "La contraseña debe tener entre 8 y 255 caracteres", and "La contraseña debe tener entre 8 y 255 caracteres". At the bottom of the form is a yellow "Crear cuenta" button and a link "Ya tengo cuenta".

Registration form. The messages next to each field belong to the same useForm object.

`Register.vue` combines two layers of validation. Vue performs immediate checks with `form.setError()` for name, email, and password; when the form is submitted, Laravel validates everything again through `CreateNewUser`. Browser-side checks improve the experience but do not replace server-side rules.

```
const form = useForm({
  name: '',
  email: '',
  password: '',
  password_confirmation: '',
  redirect: props.redirect || '',
});

form.setError('email', 'Ingresa un correo electrónico válido.');
```

Step	Result
1	Candela enters her name, email, password, and confirmation.
2	Vue normalizes and checks the basic data. If it finds a problem, it uses setError() and does not submit yet.
3	useForm sends the POST request to /register.
4	RegisteredUserController and CreateNewUser validate again and create users with role=reader.
5	TRAMA attempts to send the verification email and ends the registration session.
6	Candela opens the signed link, verifies her email, and only then can she sign in normally.



Confirmation displayed after the account is created and before the email is verified.

Candela enters her name, email, password, and confirmation and clicks Create Account. Vue sends the form to Laravel. RegisteredUserController::store() uses CreateNewUser, which validates all data again on the server even though the browser has already performed preliminary checks.

The screenshot shows a web browser window with the TRAMA registration page. The page has a dark theme. On the left, there's a large text area with the TRAMA logo and the headline 'Noticias con criterio, datos y comunidad activa.' Below this, it says 'Ingresá para comentar noticias.' On the right, there's a registration form titled 'Crear cuenta'. The form includes a link to 'Volver a portada', a 'CUENTA TRAMA' link, and a heading 'Crear cuenta'. Below the heading, it says 'Creá tu cuenta para comentar noticias.' The form has four input fields: 'Nombre' (with 'gaston_rizzo' entered), 'Email' (with 'gaston_rizzo@hotmail.com' entered), 'Contraseña' (with masked characters), and 'Confirmar contraseña' (with masked characters). A yellow 'Crear cuenta' button is at the bottom of the form. Below the button, there's a link 'Ya tengo cuenta'.

Public registration form used in TRAMA.

Step	What Happens
1	Candela submits the form.
2	RegisteredUserController::store() receives the request.
3	CreateNewUser validates name, email, and password and creates users with role=reader and is_active=true.
4	Laravel attempts to send the verification email.
5	The session is closed: registering does not sign the user in automatically.
6	Candela opens the received link, verifies her email, and can then sign in.

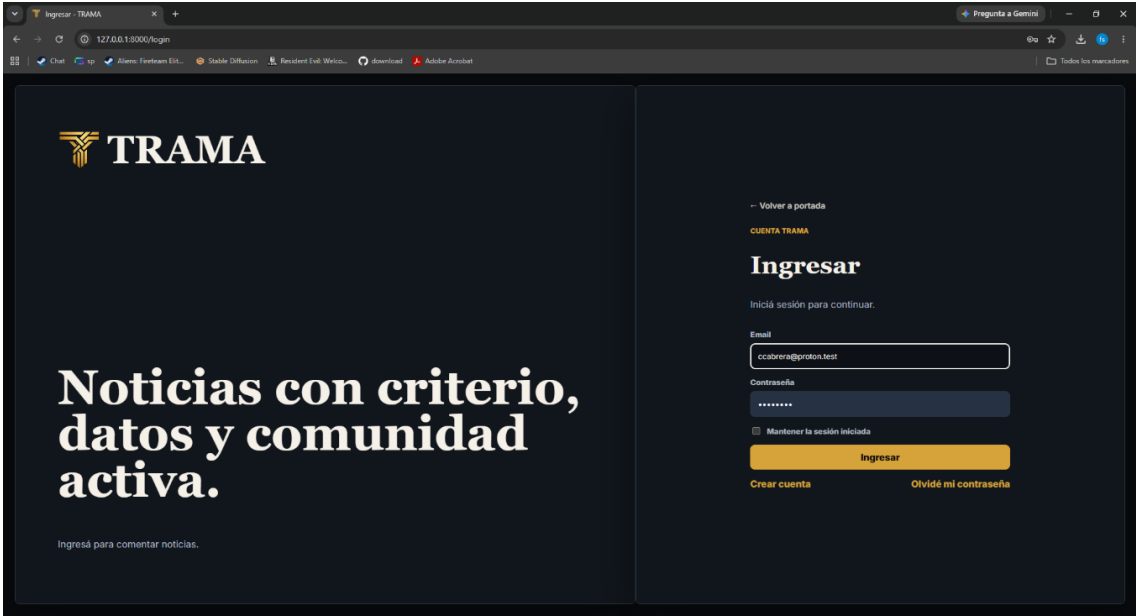
What SMTP Is and What Happens If It Fails

SMTP is the protocol an application uses to send email. If TRAMA creates the account but the mail service fails before sending the verification link, RegisteredUserController logs the error and deletes the newly created account. This prevents MySQL from retaining an account that the user cannot finish verifying.

Example: Signing In

Step by Step: Candela Signs In from Login.vue

Candela opens Sign In, enters her email and password, and clicks the button. On this screen, the form object was created with useForm(), so Vue knows the current values, Inertia knows whether the request is being processed, and Laravel can return errors associated with email or password.



Public sign-in screen. The screenshot uses test credentials; the technical flow is the same one Candela follows.

```
const form = useForm({
  email: '',
  password: '',
  remember: false,
  redirect: props.redirect || '',
});

form.post(route('login'), {
  onFinish: () => {
    form.reset('password');
    isSubmitting.value = false;
  },
});
```

Step	What Happens
1	v-model keeps form.email and form.password synchronized with the inputs.
2	submit() prevents double-click submission and calls form.post(route("login")).
3	The request reaches the authentication routes configured by Fortify.
4	Laravel validates credentials, rate limits, and account status.
5	If sign-in fails, Inertia returns errors and Login.vue reads them from form.errors.
6	If sign-in succeeds, Laravel regenerates the session and redirects to the appropriate destination.
7	onFinish clears the password and re-enables the button.

TRAMA does not store the password entered by the user exactly as typed. It stores a value produced by a security process called hashing. During login, Hash::check() can verify whether the entered password corresponds to that value without needing to store the original password.

Step	What Happens
1	Candela enters her email and password.
2	Fortify looks up the account by email.
3	Laravel checks the password.
4	TRAMA verifies that the account is active, the email is verified, and no mandatory password reset is pending.
5	If everything is correct, Laravel creates the session and allows access to continue.

Situation	What Happens
Incorrect email or password	No session is started, and an incorrect-credentials message is displayed.
Blocked account	Access is denied.
Email not verified	The user must complete verification first.
Mandatory password change pending	The link sent by email must be used before signing in.
Everything is correct	Laravel creates the session.

Login allows up to 5 attempts per minute for the normalized-email-and-IP combination. The IP is the network address from which the request arrives. The limit is enforced on the server, so it does not depend on JavaScript being enabled.

What a Session Is

After a successful login, Laravel stores the session in MySQL's sessions table. The browser retains a cookie—a small piece of data stored locally—with the identifier needed to recognize that session on subsequent requests. In Laravel, this configuration is known as the database session driver.

EnsureActiveAccount checks the account status again on subsequent requests. If Marina, the administrator, blocks an account that is already signed in, the screen currently open for that account may remain visible until the user performs another action. On the next request, the session is invalidated and the user is returned to the login page.

Password Recovery

Flow Concepts: Token, Password Broker, and Notification

A recovery token is a temporary value associated with the reset process. It is not the user's password; it proves that whoever opens the link has a valid reference issued by the system. Laravel stores the required information in password_reset_tokens and verifies the token before accepting a new password.

The Password Broker is the Laravel component that coordinates the creation and validation of those tokens. A Notification is a class used to represent a notification intended for a user; in this flow, TRAMA uses TramaResetPassword to build and send the email containing the recovery link.

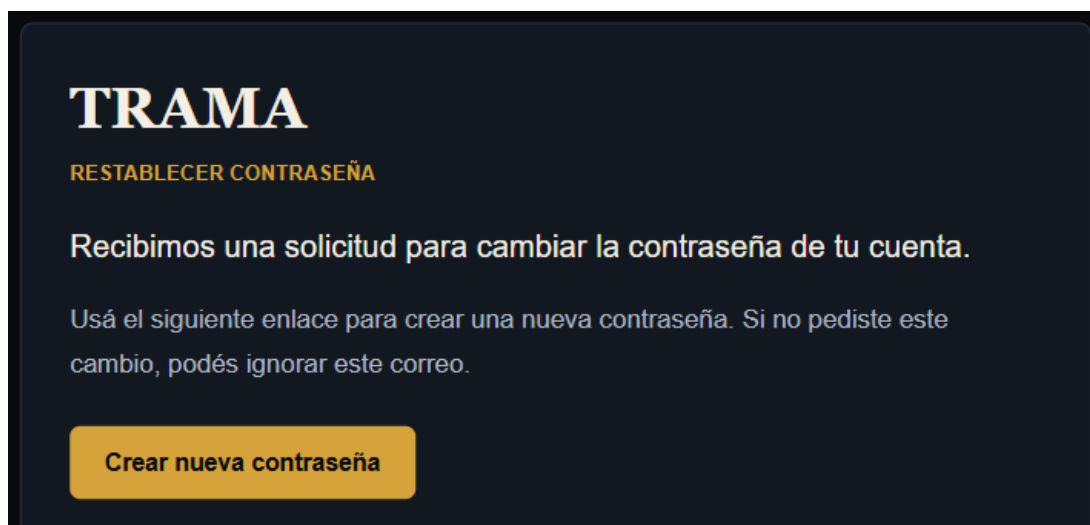
Step by Step: Candela Recovers Access

Step	What the Person Does and What Happens Internally
1	Candela opens “Forgot Password.” Laravel displays Auth/ForgotPassword.vue through Fortify and TramaBridge. The screen creates useForm({ email: "" }) to retain the email, errors, and processing state.
2	Candela enters her email and clicks Send Link. submit() runs form.post(route("password.email")), which corresponds to POST /forgot-password. Before Fortify, ValidateAuthInputLimits validates the format, normalizes the email, and applies the limits: one password-reset email attempt per email+IP every 5 minutes and up to 6 password-reset email attempts per IP in one hour.
3	If the request can continue, Fortify uses the Password Broker configured as users. When the account exists, Laravel generates the recovery token in password_reset_tokens and User::sendPasswordResetNotification() sends the custom TramaResetPassword notification. The visible response remains neutral whether or not the account exists.
4	Candela opens the received link. The /reset-password/{token} route displays Auth/ResetPassword.vue. FortifyServiceProvider provides email and token and also calculates resetLinkError with Password::broker(...)->tokenExists(), so an expired, invalid, or already used link is detected before submission is allowed.
5	ResetPassword.vue creates a useForm with token, email, password, and password_confirmation. Vue checks locally that the password is between 8 and 20 characters long, contains at least one letter and one number, and matches the confirmation. These checks help the user but do not replace server-side validation.
6	Candela clicks Update Password. form.post(route("password.update")) sends POST /reset-password. ValidateAuthInputLimits validates the email, password, and confirmation again in Laravel before Fortify processes the reset.
7	Fortify validates the token and email through the Password Broker and calls App\Actions\Fortify\ResetUserPassword. That action reapplies the password policy, stores the new hash with Hash::make(), and sets password_reset_required_at to null if the account had a pending mandatory change.
8	When the reset finishes successfully, Fortify's configuration redirects to /login and displays the status message. Candela can now sign in with the new password; the previous link is no longer valid as a recovery token.



Screen used to request the recovery link.

ForgotPassword.vue also uses useForm. The form sends only the email and remains disabled while form.processing is true. Laravel responds with a neutral message whether or not the account exists, so registered addresses are not disclosed.



Recovery email. The link leads to the ResetPassword.vue form.

On Forgot Password, the user enters an email address and requests a link. The screen displays the same message whether or not that email exists. This prevents someone from using the form to determine which addresses are registered.

For the same email and IP, at most one password-reset email attempt is made every 5 minutes. In addition, an IP can generate up to 6 password-reset email attempts per hour. If another request is made too soon, the screen keeps the same message but does not send another email. The recovery link remains valid for 60 minutes.

Real Source Code: Additional Checks During Login

Fortify provides the authentication mechanism, but TRAMA adds its own rules before accepting a session. app/Providers/FortifyServiceProvider.php registers an authentication function that checks the password and then reviews the account status. The following excerpt preserves the executable code from that function and omits only the explanatory comments from the source file:

```

Fortify::authenticateUsing(function (Request $request): ?User {
    $user = User::query()
        ->where('email', $request->input(Fortify::username()))
        ->first();

    if (! $user || ! Hash::check((string) $request->input('password'), $user->password)) {
        return null;
    }

    if (! $user->is_active) {
        throw ValidationException::withMessages([
            Fortify::username() => 'Tu cuenta se encuentra bloqueada y no puede acceder a TRAMA. Si
considerás que se trata de un error, contactá al equipo de TRAMA.',
        ]);
    }

    if ($user->password_reset_required_at !== null) {
        throw ValidationException::withMessages([
            Fortify::username() => 'Debés restablecer la contraseña desde el enlace enviado a tu correo.',
        ]);
    }

    if ($user instanceof MustVerifyEmail && ! $user->hasVerifiedEmail()) {
        throw ValidationException::withMessages([
            Fortify::username() => __('auth.email_not_verified'),
        ]);
    }

    if (Hash::needsRehash($user->password)) {
        $user->forceFill(['password' => Hash::make((string) $request->input('password'))])>save();
    }

    return $user;
});

```

Returning null allows Fortify to produce the normal incorrect-credentials message. The other situations become controlled validation errors: a correct password is not enough if the account is blocked, has a mandatory reset pending, or has not yet verified its email. Finally, `Hash::needsRehash()` checks whether the stored hash uses an older configuration; if necessary, Laravel generates it again with the current configuration after the user has been successfully authenticated. These checks run in Laravel and do not depend on the Vue interface hiding buttons.

12. Roles, Dashboards, and Permissions

TRAMA has four account types with different permissions: registered user, journalist, editor, and administrator. The menu changes according to the role, but important permissions are also checked in Laravel. Knowing a private URL does not allow those rules to be bypassed.

JOURNALIST	EDITOR	ADMINISTRATOR
Dashboard	Dashboard	Dashboard
Articles	Articles	Categories · Tags
View portal	Comments	Advertising
	View portal	Employees · Users
		View portal

Journalist

The journalist creates drafts, completes articles, submits them for review, reads the editor's feedback, and corrects returned articles. A journalist cannot publish an article directly or open the general moderation module.

Editor

The editor can also write articles. In addition, the editor reviews articles submitted by journalists, can return them with feedback, schedule them, publish them, archive them, change homepage flags, and moderate comments.

Administrator

The administrator uses a separate dashboard for categories, tags, advertising, employees, and registered users. The administrator does not replace the editor: administrators do not review or publish articles and do not write public comments.

Main Permissions

Action	Registered User	Journalist	Editor	Administrator
Write a top-level comment	Yes	No	No	No
Reply to a comment	Yes	Only on their own articles	Yes, as TRAMA Team	No
Like a comment	Yes	No	No	No
Report a comment	Yes	No	No	No
Create articles	No	Yes	Yes	No
Review journalists' articles	No	No	Yes	No
Publish or schedule	No	No	Yes	No
Moderate comments	No	No	Yes	No
Categories, tags, and advertising	No	No	No	Yes
Employees and registered users	No	No	No	Yes

Example of a Permission Checked on the Server

If a journalist manually enters a URL reserved for the administrator, Laravel checks the role again before running the controller. Hiding the option from the menu improves the interface, but the actual security is enforced on the server.

Employees and Registered Users Are Separate Modules

Employees contains the internal journalist, editor, and administrator accounts. Registered Users contains accounts created through the portal. The administrator can block them, reactivate them, initiate an access reset, and resolve reviews referred from Moderation.

Real Source Code: Permission Is Checked Before the Controller Runs

`routes/web.php` groups the structure and account modules under `EnsureAdminRole`. Manually knowing a URL such as `/admin/users/employees` does not bypass this middleware:

```
Route::middleware(EnsureAdminRole::class)->group(function (): void {
    Route::get('users/employees', [AdminUserController::class, 'employees'])
        ->name('users.employees');

    Route::get('users/readers', [AdminUserController::class, 'readers'])
        ->name('users.readers');

    Route::get('categories', [AdminCategoryController::class, 'index'])
        ->name('categories.index');
});
```

The `app/Http/Middleware/EnsureAdminRole.php` middleware checks the authenticated account before allowing the request to continue:

```
public function handle(Request $request, Closure $next): Response
{
    if (! $request->user()->canManageSystem()) {
        abort(403, 'Solo un administrador puede acceder a esta sección.');
```

`abort(403, ...)` ends the request with HTTP status 403, meaning that the account was recognized but is not authorized for that action. Only if `canManageSystem()` returns `true` is `$next($request)` executed and Laravel allowed to continue to the route and its controller.

Step by Step: Valentina Tries to Open a Module Reserved for the Administrator

This case shows why hiding a button is not enough to enforce permissions.

Step	What the Person Does and What Happens Internally
1	Valentina signs in with the journalist role. Her menu does not show Categories, Tags, Advertising, or Registered Users.
2	Even if she manually enters an administration URL, the request still enters through <code>routes/web.php</code> .
3	The route group runs <code>EnsureAdminRole</code> before the controller.
4	The middleware checks the actual role stored in the session and on the authenticated account.
5	Because Valentina is not an administrator, Laravel stops the request and does not execute the protected action.
6	Marina, with the administrator role, passes through the same middleware successfully and only then reaches the module controller.

13. CMS Dashboard and Role-Based Metrics

After login, an internal account reaches the CMS dashboard. `DashboardController` does not send the same content to everyone: it checks the user's permissions and combines `EditorialStatsService` with recent-activity listings. The goal is for each role to see information related to the tasks it can actually perform.

Role	Main Indicators	Activity and Listings
Journalist	States of their articles and cumulative views of their own publications.	Their most-viewed articles, articles under review, latest revisions, and recent publications.

Role	Main Indicators	Activity and Listings
Editor	Overall editorial states, Views Today, pending comments, and active authors.	Ranking of up to six published articles available to the editor, articles under review, latest revisions, recent publications, and pending comments.
Administrator	Active employees, registered users, active categories, active tags, and active advertisements.	Latest registered users and recent days with banner clicks. Does not receive editorial activity or article rankings.

How the View Calculation Changes

For a journalist, EditorialStatsService adds up the views field from that journalist's own articles and displays the cumulative total. For an editor, the dashboard uses ArticleView and counts only views from the current editorial day. This gives the editor a view of the day's operational activity rather than a historical total inflated by all the test data.

Real Source Code: The Dashboard Chooses Metrics by Role

The difference between Valentina, Mateo, and Marina is not handled by hiding cards in Vue. app/Services/EditorialStatsService.php receives the authenticated User and builds different data sets. The following excerpt preserves the main branches of getDashboardIndicatorsForUser():

```

if ($user->canManageSystem()) {
    return [
        'Empleados activos' => User::query()
            ->whereIn('role', ['admin', 'editor', 'journalist'])
            ->where('is_active', true)
            ->count(),
        'Usuarios registrados' => User::query()
            ->where('role', 'reader')
            ->count(),
        'Categorías activas' => Category::query()
            ->where('is_active', true)
            ->count(),
        'Publicidades activas' => Advertisement::query()
            ->where('is_active', true)
            ->count(),
    ];
}

$articles = $this->visibleArticlesForUser($user);

$indicators = [
    'Publicadas' => (clone $articles)->where('status', 'published')->count(),
    'En revisión' => (clone $articles)->where('status', 'review')->count(),
    'Vistas' => $this->dashboardViewsForUser($user, $articles),
];

if ($user->canReviewArticles()) {
    $indicators['Comentarios pendientes'] = Comment::query()
        ->where('status', 'pending')
        ->count();
}

```

canManageSystem() separates the administrator, who receives structure and account metrics. For journalists and editors, visibleArticlesForUser() first limits which articles are included in calculations: Valentina is restricted to her own authorship, while Mateo can see the broader editorial workflow according to the service rules. DashboardController then passes stats, rankings, revisions, and permissions to Admin/Dashboard.vue through TramaBridge.

Comparative Example

User	What They See on Entry
Valentina, journalist	Drafts, articles submitted for review, returned articles, scheduled articles, published articles, and archived articles authored by her; cumulative views and her six most-viewed articles.
Mateo, editor	Newsroom status, pending comments, active authors, Views Today, up to six articles under review, and the latest visible editorial actions.
Marina, administrator	Counts of employees and readers, active categories, tags, and advertisements; latest accounts created and recent advertising-click activity.

Permissions are also passed to Admin/Dashboard.vue. Vue can hide buttons or blocks that do not apply, but actual enforcement remains in Laravel: a card being absent from the dashboard does not replace middleware or authorization checks on private routes.

Step by Step: Valentina and Mateo Open the Dashboard and Receive Different Metrics

The dashboard reuses the same interface concept, but Laravel prepares different data according to each role's responsibilities.

Step	What the Person Does and What Happens Internally
1	Valentina enters the dashboard as a journalist. Laravel identifies her account and calculates metrics centered on her own articles.
2	The queries count drafts, reviews, publications, and views associated with Valentina's work according to the dashboard rules.
3	Mateo enters with the editor role. The same dashboard entry point requires a newsroom-wide view, so the scope of the queries changes.
4	Mateo can receive counts of articles under review, scheduled articles, published articles, and other overall signals that do not depend solely on his authorship.
5	The controller prepares the appropriate props for each role, and Vue renders equivalent cards with different values.
6	The distinction is made on the server; it does not consist of downloading all the data and hiding with CSS what each person should not see.

14. Creating an Article: Fields and Validation

The article form has Content, Configuration, and History tabs. When an article is being created for the first time, History does not appear yet because there is no previous save to compare.

TRAMA does not require a draft to be complete. A journalist can save the work completed so far and continue later. The full rules apply when the journalist attempts to submit the article for review, schedule it, or publish it.

Main Fields

Visible Field	What It Contains	Rule When Completing the Article
Title	Main headline	20 to 90 characters.

Visible Field	What It Contains	Rule When Completing the Article
Subheading	Text accompanying the title	50 to 140 characters.
Summary	Short text used in listings	50 to 200 characters.
Body	Full article content	300 to 10,000 visible characters.
Category	Main news section	Required.
Tags	Related topics	Between 1 and 5.
Cover Image	Main image	Required when the article must be complete.

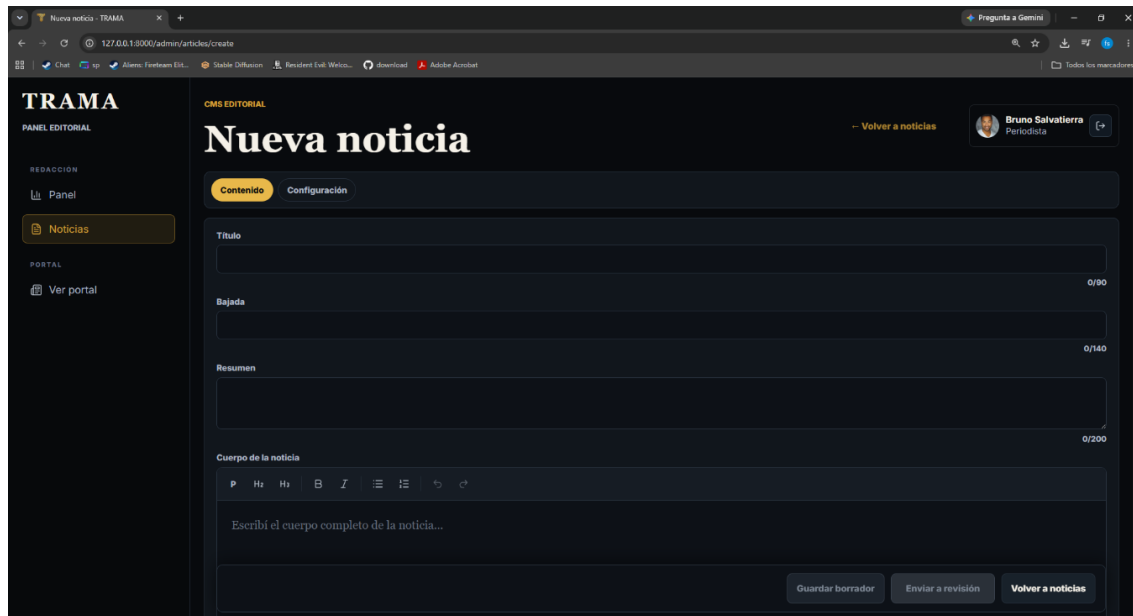
The interface uses the labels `Subtitle` and `Excerpt`. In the code and in MySQL, those same fields are named `subtitle` and `excerpt`.

How the Article Body Is Sanitized

The text editor uses HTML to represent elements such as paragraphs, bold text, or links. Before validating and storing the body, `HtmlSanitizer` removes tags or attributes that TRAMA does not allow. This prevents executable code embedded in the text from being stored.

Example: Saving a Draft and Submitting It for Review

Before Laravel: How Form.vue Preserves What Valentina Writes



Content tab of the editorial form while using a test journalist account.

The editorial form is one of TRAMA's most complete uses of `useForm`. The same object brings together content, category, cover image, editorial state, homepage flags, dates, and tags. When creating an article it starts with empty values; when editing, it takes the values received in `props.article`.

The screenshots in this subsection come from the visual guide and use a different test journalist account. The narrative example continues to use Valentina to preserve the instructional thread; both accounts use the same `Form.vue` and the same `Journalist-role` rules.

```
const form = useForm({
  category_id: props.article?.category_id || '',
  title: props.article?.title || '',
  subtitle: props.article?.subtitle || '',
  excerpt: props.article?.excerpt || '',
  body: props.article?.body || '',
  cover_image_file: null,
  remove_cover_image: false,
  status: props.article?.status || 'draft',
  is_breaking: props.article?.is_breaking || false,
  is_featured: props.article?.is_featured || false,
  published_at: props.article?.published_at || '',
  scheduled_at: props.article?.scheduled_at || '',
  tag_ids: props.article?.tag_ids || [],
});
```

Título
Nuevas rutas comerciales reordenan la logística regional 56/90

Bajada
La redacción compara proyectos logísticos y acuerdos recientes. 63/140

Resumen
El artículo aún necesita ordenar fuentes y verificar estimaciones antes de revisión. 84/200

Cuerpo de la noticia

Rich Text Editor: P H₂ H₃ B I [List Icons] [Undo] [Redo]

Nuevas rutas comerciales están modificando la planificación logística regional a medida que empresas prueban corredores alternativos y combinaciones distintas de puerto, ferrocarril y transporte por carretera. El borrador busca comparar qué cambios responden a costos coyunturales y cuáles pueden mantenerse en el tiempo.

La redacción analiza proyectos de infraestructura, acuerdos aduaneros y tiempos de tránsito, pero todavía necesita ordenar las fuentes para no mezclar obras anunciadas con capacidad ya disponible. Esa diferencia es clave porque una ruta potencial no puede evaluarse como si estuviera plenamente operativa.

El artículo también debe verificar estimaciones de ahorro y volúmenes antes de pasar a revisión. La versión final debería mostrar con claridad qué cargas están cambiando de recorrido, por qué lo hacen y qué cuellos de botella pueden limitar esa reorganización.

Buttons: Eliminar borrador, Guardar borrador, Enviar a revisión, Volver a noticias

Content entered during editing: title, subtitle, excerpt, and body. The screenshot belongs to a test journalist account.

Categoría
Mundo

Etiquetas 2 / 5
Geopolítica x Inversión x
Buscar etiqueta...
Podés elegir hasta 5. La categoría solo ordena las sugerencias y no limita la búsqueda.

ESTADO ACTUAL
En borrador

Imagen de portada
[Image of a port/warehouse]
Cambiar imagen
Quitar imagen
Imagen opcional mientras la noticia esté en borrador. JPG, JPEG, PNG o WEBP. Máximo 5 MB.
Previsualizar noticia

Buttons: Eliminar borrador, Guardar borrador, Enviar a revisión, Volver a noticias

Configuration for the same form: category, tags, and cover image.

Step by Step: Valentina Saves the First Draft

Step	What Happens in the Project
1	Valentina writes. Each v-model changes a property of the form object.
2	form.isDirty becomes true because the values no longer match the reference from the last save.
3	Valentina clicks Save Draft, and submit() first validates the checks the browser can perform.
4	During creation, form.post(route("admin.articles.store")) sends FormData to Laravel.
5	ArticleRequest normalizes the title, subtitle, excerpt, and body and applies the rules corresponding to the draft state.
6	ArticleController coordinates cover-image processing and delegates the appropriate editorial workflow.
7	MySQL stores the article and its relationships within the transactions defined by the backend.
8	The response returns through Inertia. onSuccess clears the temporary file and runs form.defaults(), causing isDirty to return to false.

What Happens If Valentina Tries to Complete an Action with Invalid Data

When Laravel returns validation errors, Inertia places them in form.errors. Form.vue checks which fields failed: if the problem is in the title, subtitle, excerpt, or body, it opens Content; for category, tags, cover image, or scheduling, it opens Configuration. This returns the user directly to the area that needs to be corrected.

```
onError: (errors) => {
  const hasContentErrors =
    errors.draft_content || errors.title || errors.subtitle ||
    errors.excerpt || errors.body;

  activeTab.value = hasContentErrors
    ? 'content'
    : 'settings';
},
```

Valentina starts an article and writes only the title and part of the body. She can click Save Draft even if the category, tags, cover image, or minimum lengths for some fields are still missing. Later she completes the article and clicks Submit for Review. At that point, ArticleRequest applies the full rules.

If, for example, the excerpt has only 35 characters, Laravel rejects the operation, keeps the article as a draft, and displays the error next to the corresponding field.

Step	What Happens
1	Form.vue gathers the values entered in the form.
2	ArticleRequest normalizes and validates the data according to the requested state.
3	If a new cover image was uploaded, ArticleCoverProcessor and SafeImageProcessor inspect it and generate the final file.
4	ArticleController calls ArticleWorkflowService to store the related changes.

Step	What Happens
5	MySQL updates the article, its tags, the search text, and the history within a transaction; in other words, as a group of changes that is committed or rolled back together.
6	The form receives the new state or the validation messages.

Cover Image

The cover image accepts JPG, JPEG, PNG, or WebP files up to 5 MB. The image must have sufficient resolution to generate 1600 × 900 without upscaling. It also goes through the local safety analysis explained in section 20.



Example of a cover image rejected by safety validation.

What an Article Slug Is

A published article needs a readable URL. TRAMA generates part of that URL from the title; that part is called the slug.

```
Title:
Nuevas rutas comerciales reordenan la logística regional

Slug:
nuevas-rutas-comerciales-reordenan-la-logistica-regional

URL:
/noticias/nuevas-rutas-comerciales-reordenan-la-logistica-regional
```

While the article remains a draft, it can use a provisional identifier such as borrador-UUID. When it is scheduled or published for the first time, it receives its final slug. It then keeps that slug even if the title changes so previously shared links are not broken.

Text Editor and History

Undo and redo inside the editor affect only the writing session currently open in the browser. History is different: it stores permanent revisions in MySQL and remains available even after the browser is closed.

Real Source Code: Rules Change According to the Requested State

`app/Http/Requests/Admin/ArticleRequest.php` does not apply exactly the same rules to a draft as it does to an article ready to enter the editorial workflow. It first determines whether the requested state requires a complete article:

```
private function requiresCompleteArticle(): bool
{
    return in_array(
        (string) $this->input('status'),
        ['review', 'scheduled', 'published'],
        true,
    );
}
```

That result is used to build the rules. A draft may leave fields incomplete, while review, scheduling, and publication require the editorial minimums:

```
$requiresCompleteArticle = $this->requiresCompleteArticle();

$titleRules = $requiresCompleteArticle
    ? ['required', 'string', 'min:20', 'max:90']
    : ['nullable', 'string', 'max:90'];

return [
    'category_id' => $requiresCompleteArticle
        ? ['required', $this->categoryExistsRule()]
        : ['nullable', $this->categoryExistsRule()],

    'title' => $titleRules,

    'tag_ids' => $requiresCompleteArticle
        ? ['required', 'array', 'min:1', 'max:5']
        : ['sometimes', 'array', 'max:5'],
];
```

Validation is decided on the server from the received state. Therefore, modifying the form's HTML or JavaScript does not allow an incomplete article to be submitted for review: `ArticleRequest` checks the rules again before the controller continues.

15. Autosave and Change Protection

After the first manual save, an editable article can be autosaved. Autosave reduces the risk of losing work, but it does not create a historical revision every time someone types a word.

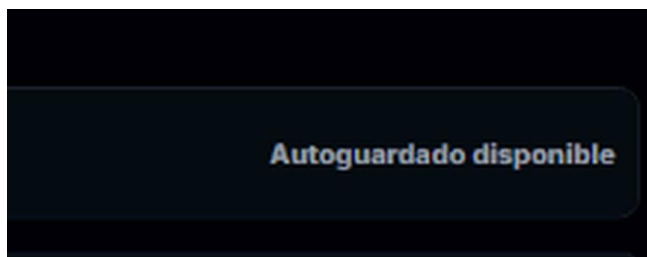
It is used only in Draft (draft) and Returned for Changes (needs_changes), which are the states in which the author can continue modifying the content.

Example of the 15-Second Timer

Do Not Confuse Manual Save with useForm and Autosave with Axios

Manual saving must handle the complete form, files, errors, and Inertia navigation. Autosave serves a different purpose: silently sending only a subset of fields for an article that already exists, without changing screens. That is why `Form.vue` builds `autosavePayload` and uses `window.axios.patch()`.

Manual Save	Autosave
useForm + Inertia	Direct Axios
Can create a new article or update the existing one	Only updates an existing article
Includes cover image, tags, state, and other fields depending on the action	Sends category, title, subtitle, excerpt, and body
Can return errors inside <code>form.errors</code>	Handles the result as an auxiliary request
Can run <code>form.defaults()</code> when finished	Updates the form's reference with the autosaved values



Visible state when autosave is available.



Warning displayed when there is work completed after the last save.

Valentina changes a sentence. Eight seconds pass and she types again, so the timer starts over. Five seconds later she corrects another word and the timer resets again. Only after 15 seconds pass with no changes does Vue send a single autosave request.

If the article has id 37, the request is sent to `/admin/articles/37/autosave`. The HTTP method used is PATCH, which in this case indicates that part of an existing record is being updated.

```
const autosavePayload = {
  category_id: form.category_id,
  title: form.title,
  subtitle: form.subtitle,
  excerpt: form.excerpt,
  body: form.body,
};

window.axios.patch(
  route('admin.articles.autosave', props.article.id),
  autosavePayload
);
```

What Autosave Stores

`AutosaveArticleRequest` validates `category`, `title`, `subtitle`, `excerpt`, and `body`. `ArticleWorkflowService::autosave()` updates those fields and recalculates reading time. It does not change the article state, does not change tags or the cover image, and does not create a new row in `article_revisions`.

What Happens If Two Saves Coincide

Autosave may begin just as the journalist clicks Save Changes. To prevent both operations from trying to write the same article at the same time, Laravel briefly locks that row while one of the operations finishes. The method used is `lockForUpdate()`.

Protection When Leaving the Screen

If there are changes made after the last save and the user tries to return to the listing, open another dashboard option, or sign out, TRAMA displays a confirmation. If the browser tab is closed or reloaded, it uses the browser's built-in warning; in JavaScript, that event is called `beforeunload`.

Step by Step: Valentina Writes, Waits, and Autosave Protects the Draft

Autosave is a good example of a silent action that does not need to replace the entire screen.

Step	What the Person Does and What Happens Internally
1	Valentina modifies the title, subtitle, excerpt, or body of an editable article. Vue detects that there are pending changes.
2	Each new change restarts the timer. While Valentina is still typing, a request is not sent for every keystroke.
3	When 15 seconds pass without changes and the form has the minimum required data, Vue builds <code>autosavePayload</code> .
4	Instead of the normal <code>useForm</code> submission, the component uses <code>axios.patch()</code> against the <code>admin.articles.autosave</code> route.
5	<code>AutosaveArticleRequest</code> validates only the fields allowed for this operation, and <code>ArticleWorkflowService::autosave()</code> locks the row with <code>lockForUpdate()</code> .
6	MySQL updates the work in progress and recalculates the reading time, but it does not change the status, tags, cover image, or create a historical revision.
7	Vue receives the response and changes the visual indicator to <code>Changes Autosaved</code> . If Valentina tries to leave earlier, the interface can warn that there are still pending changes.

16. Workflow Between Journalist and Editor

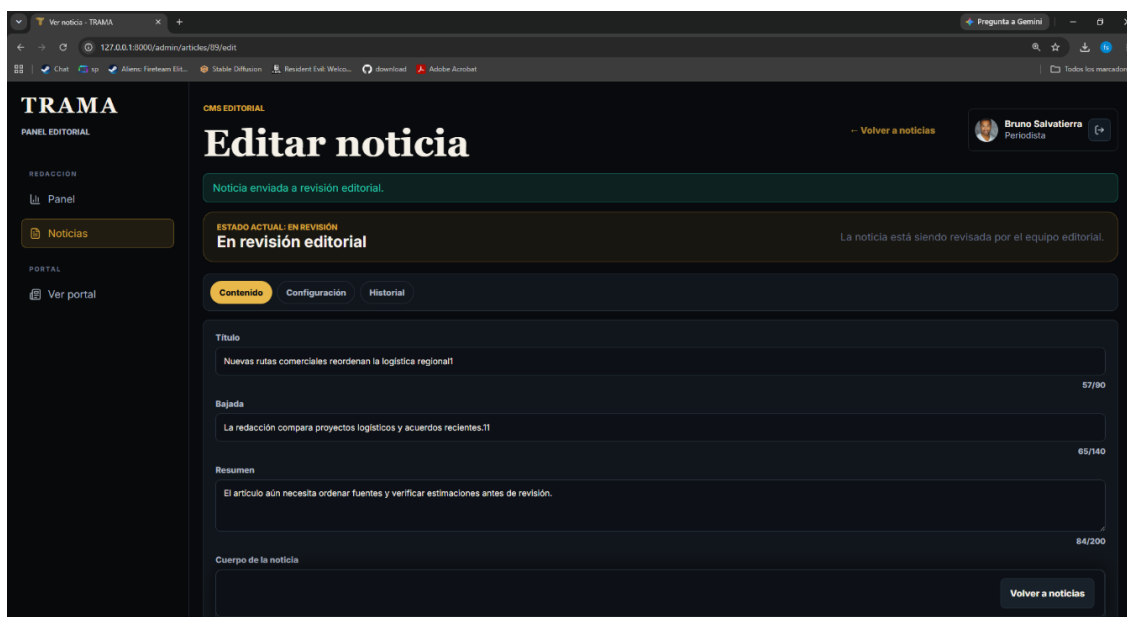
The editorial workflow clearly separates the journalist's work from the editor's. The journalist writes the article and makes the requested corrections. The editor reviews the content and decides whether it needs changes, should be published immediately, or should be scheduled for later.

Complete Example: From Draft to Publication

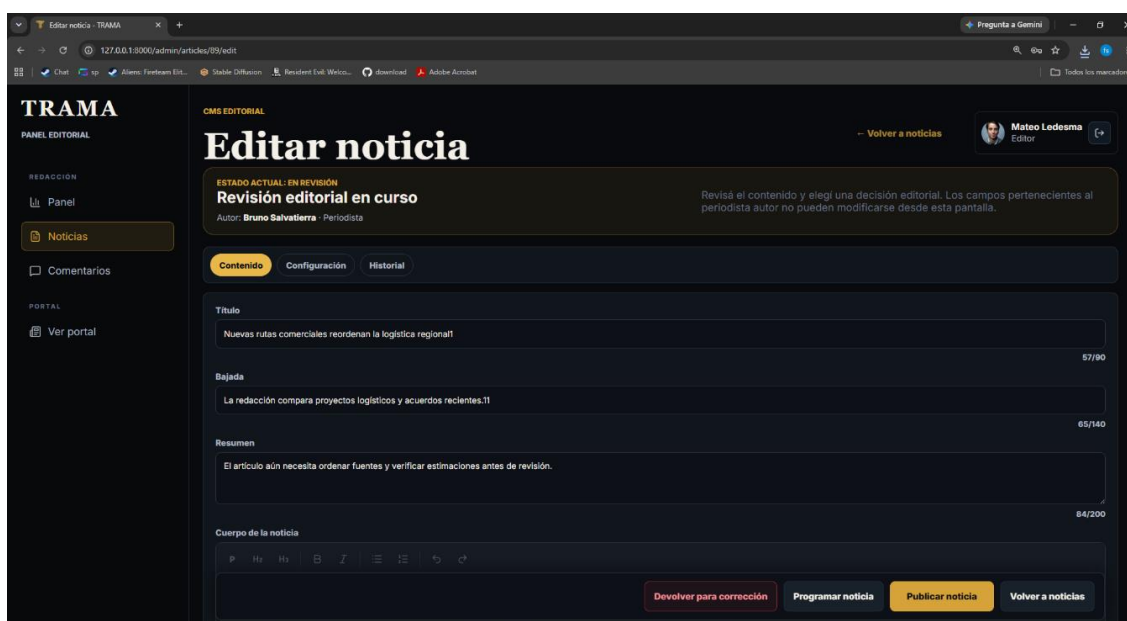
The Same Flow Seen from the Interface



Valentina confirms submission of the article into the editorial workflow.



After submission, the article is Under Review and its content is locked for the journalist.



Mateo opens the submitted article and makes the editorial decision.

Why This Example Matters

The same Form.vue screen changes its behavior according to author, role, and state. Vue can hide or disable controls, but definitive authorization remains in ArticleRequest, middleware, and ArticleWorkflowService. The interface explains the state; the server enforces it.

Valentina finishes an article and clicks Submit for Review. Before accepting it, TRAMA checks again that it contains the data required to enter the editorial workflow. If validation succeeds, the article stops being a draft and moves to Under Review.

From that moment on, Valentina can no longer modify it. The article becomes available to Mateo, the editor, who can open it from the articles pending review and inspect its content, configuration, and history.

If Mateo finds something that must be corrected, he writes feedback and uses Return for Changes. The article moves to Returned for Changes, and the feedback is recorded in its history so the journalist knows exactly what must be reviewed.



REVISIÓN EDITORIAL

Devolver para corrección

La noticia cambiará a “Devuelta para corrección”. La observación quedará registrada en su historial.

Observaciones

Ordenar y citar mejor las fuentes, diferenciando los proyectos anunciados de la infraestructura que ya está operativa. También verificar las cifras de ahorro, volúmenes y tiempos de tránsito, y agregar ejemplos concretos de rutas o cargas que estén cambiando de recorrido.

272/1200

Cancelar **Enviar observaciones**

The editor returns the article with feedback that is recorded in the history.

Valentina opens the editor again and sees the editor's request. She can correct the content and save those changes without resubmitting the article yet. When she finishes, she clicks Resubmit for Review and the article returns to Under Review.

Mateo receives it again. He can return it once more if he still finds problems or, if it is ready, schedule it for a future date or publish it. This keeps responsibility for correcting the content with the journalist while leaving the editorial decision to the editor.

Flow Summary



States That Appear During the Workflow

Text Shown to the User	Value Stored by TRAMA	What It Means
Draft	draft	The author is still writing.
Under Review	review	The journalist submitted it and is waiting for an editor's decision.
Returned for Changes	needs_changes	The editor left feedback and the journalist must make corrections.
Scheduled	scheduled	Prepared to be published at a specific date and time.
Published	published	May appear on the portal.
Archived	archived	Removed from the portal without deleting the article or its history.

What the Editor Can Change in Someone Else's Article

When Mateo reviews an article written by Valentina, he can decide its state, but he does not directly replace its title, subtitle, excerpt, body, category, tags, or cover image. If corrections are needed, he returns it with feedback so the author can make the changes.

The editor can write his own articles. In that case, he can save them as drafts, schedule them, or publish them without going through another editor's review. A journalist cannot publish directly.

Real Source Code: Returning an Article for Changes

`ArticleWorkflowService` centralizes transitions that require several coordinated changes. The `requestChanges()` method demonstrates this responsibility clearly: the article state, the editor's feedback, and the history must be stored as a single operation.

```
return DB::transaction(function () use ($article, $reviewer, $message): Article {
    $locked = Article::query()
        ->with('tags:id,name,slug')
        ->whereKey($article->id)
        ->lockForUpdate()
        ->firstOrFail();

    if ($locked->status !== 'review') {
        throw ValidationException::withMessages([
            'message' => 'Solo se puede devolver una noticia que está en revisión.',
        ]);
    }

    $previousSnapshot = $locked->revisionSnapshot();
    $previousStatus = $locked->status;

    $locked->update([
        'status' => 'needs_changes',
        'published_at' => null,
        'scheduled_at' => null,
        'is_breaking' => false,
        'is_featured' => false,
    ]);

    $locked->reviewFeedback()->create([
        'returned_by' => $reviewer->id,
        'message' => $message,
    ]);

    $this->revisions->record(
        $locked,
        $reviewer,
        'changes_requested',
        $previousStatus,
        $previousSnapshot,
    );

    return $locked->refresh();
}, attempts: 3);
```

`DB::transaction()` guarantees that the changes are committed together. `lockForUpdate()` rereads and locks the row before checking the state, preventing another operation from publishing or archiving the same article while the editor is returning it. The feedback is created after the state change within the same transaction, and `ArticleRevisionService` records the result in the history.

Step by Step: Valentina and Mateo Complete One Editorial Cycle

The same article changes hands without losing authorship, history, or editing rules.

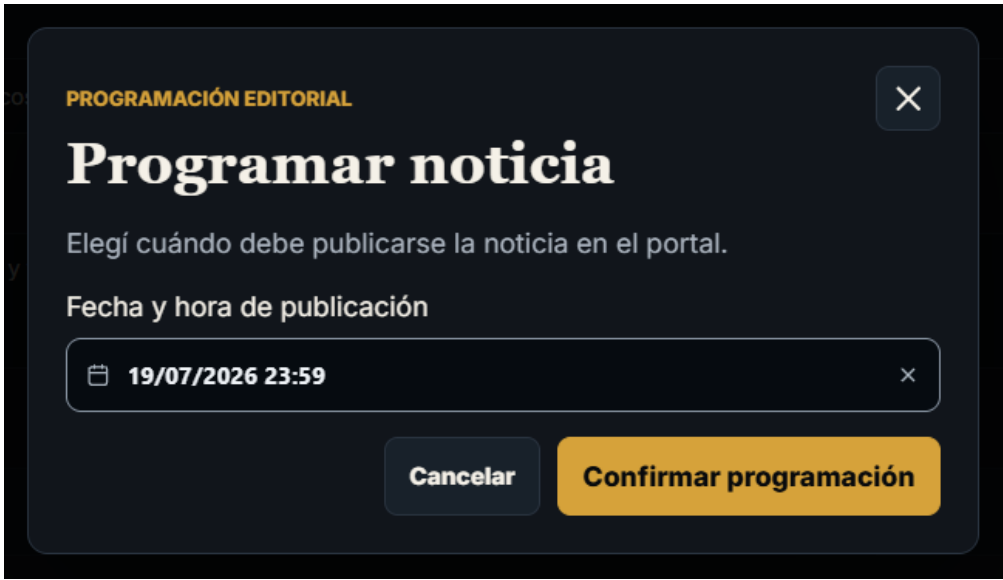
Step	What the Person Does and What Happens Internally
1	Valentina finishes the draft and clicks Submit for Review. Laravel validates the requirements for the requested state.
2	<code>ArticleWorkflowService</code> changes the article to Under Review and records the action in the history.
3	Valentina can no longer edit it while she waits for an editor's decision.

Step	What the Person Does and What Happens Internally
4	Mateo opens the article from his dashboard. He can review content, configuration, history, and the editorial controls enabled by his role.
5	If he finds a problem, he writes feedback and returns the article for changes. The article moves to Returned for Changes and Valentina regains editing access.
6	Valentina corrects the indicated issue and resubmits the article. The return remains associated with the workflow and can be marked as resolved.
7	Mateo can schedule or publish the valid version. The transition runs within the editorial service rules and is recorded as a revision.
8	When the article becomes Published, public portal queries can include it according to date, category, and homepage treatment.

17. Scheduling, Publishing, Archiving, and Homepage Placement

Scheduling an Article

Step by Step: Mateo Schedules a Publication



Scheduling modal used by the editor.

Step	What Happens
1	Mateo opens Schedule Article and chooses a date and time with Vue DatePicker.
2	The stable value is stored in form.scheduled_at and the requested state becomes scheduled.
3	ArticleRequest requires a valid date later than the current TRAMA time and within the 30-day window.
4	ArticleWorkflowService records the Scheduled state and the corresponding revision.
5	The article is locked for editing while it remains in that state.

Step	What Happens
6	Laravel Scheduler runs <code>articles:publish-scheduled</code> ; when the time arrives, automatic publication validates the current state again within a transaction.

Suppose the time TRAMA is using is 19/07/2026 16:35. Mateo reviews an article and decides that it should be published at 16:40. When he confirms, the article enters the Scheduled state and the date and time are stored in `scheduled_at`. As long as it remains scheduled, normal content is locked to prevent silent changes before publication.

Laravel has a tool called Scheduler that checks periodic tasks defined by the project. Every minute it can run the `articles:publish-scheduled` command. When an article has reached its `scheduled_at` time, the command checks it again inside a transaction and changes it to Published. That action is recorded in the history as `published_automatically`. To test this automatic publication locally without waiting for a server task scheduler, Scheduler can be kept running with `php artisan schedule:work`. This command only runs Laravel's periodic tasks: it does not advance TramaClock and is not required for browsing TRAMA.

To test automatic publication locally, keep another terminal open with:

```
php artisan schedule:work
```

The process checks scheduled tasks while it remains active. It does not advance TramaClock; it only allows Laravel to run `articles:publish-scheduled` when appropriate.

```
19/07/2026 16:35 → Scheduled
19/07/2026 16:39 → still Scheduled
19/07/2026 16:40 → the Scheduler can publish it
```

Current

For dates after 19/07/2026, the limitation explained in section 8 applies: the form can accept them, but TramaClock cannot advance to the following day with the current configuration.

Limitation

Canceling a Schedule

Only the editor can cancel a schedule. If the article was written by that same editor, it returns to Draft. If it belongs to a journalist, it returns to Under Review. TRAMA clears scheduling and publication dates that no longer apply and records `schedule_cancelled` in the history.

Publishing and Archiving

Publishing sets `published_at`, and the article can appear on the portal if its category also meets the visibility rules. Archiving changes the status to Archived and removes the article from the portal, while preserving the record and its previous revisions.

Breaking and Featured

The editor can mark a published or scheduled article as Breaking or Featured. In the code, these flags are stored as `is_breaking` and `is_featured`. The homepage allows at most 7 active featured articles at the same time.

TRAMA also limits the number of categories that can appear publicly at the same time to 12. If publishing an article would cause a thirteenth section to appear, the operation is rejected until another category stops appearing on the portal.

Real source code: from the Scheduler to automatic publishing

`routes/console.php` registers which command Laravel should check and how often. This configuration does not start the Scheduler by itself; it only defines the task:

```
Schedule::command('articles:publish-scheduled')
  ->everyMinute()
  ->withoutOverlapping();
```

`everyMinute()` sets the frequency. `withoutOverlapping()` prevents a second execution of the same command from starting while the previous one is still active. When the Scheduler runs it, `PublishScheduledArticles` looks for scheduled articles whose publication time has arrived. The first fragment shows the query and `chunkById(100, ...)`, which processes them in batches of up to 100 records:

```
Article::query()
  ->where('status', 'scheduled')
  ->whereNotNull('scheduled_at')
  ->where('scheduled_at', '<=', $editorialNow)
  ->orderBy('id')
  ->select('id')
  ->chunkById(100, function ($articles) use ($revisions, $categoryOrder, $editorialNow, &$amp;published):
void {
    foreach ($articles as $articleReference) {
```

Inside each batch, the `foreach` processes one article at a time. Publishing runs inside a transaction: before changing the status, the command reads the row again with `lockForUpdate()` and checks once more that it is still scheduled and that its publication time has arrived:

```
$wasPublished = DB::transaction(function () use ($articleReference, $revisions, $categoryOrder,
$editorialNow): bool {
    $article = Article::query()
        ->with('tags:id,name,slug')
        ->whereKey($articleReference->id)
        ->lockForUpdate()
        ->first();

    if (
        ! $article
        || $article->status !== 'scheduled'
        || $article->scheduled_at === null
        || $article->scheduled_at->gt($editorialNow)
    ) {
        return false;
    }
```

The lock prevents two executions from making a decision about the same article at the same time. Outside these fragments, the method validates the public category limit, saves the previous snapshot, publishes the article, and records `published_automatically` in the history. If a rule throws `ValidationException`, that article remains scheduled and the rest of the batch continues.

18. Article History, Revisions, and Restoration

Whenever an important save occurs, TRAMA can preserve a copy of the article as it stood at that moment. This makes it possible to see what it contained before, what changed, who made the change, and what status it had at the time. Those copies are stored in `article_revisions`.

Example of a revision

```
Previous version
Title: New Trade Routes Reshape Regional Logistics

Later version
Title: New Trade Corridors Reshape Regional Logistics
```

`articles` stores the article's current state. `article_revisions` also makes it possible to inspect previous versions. In the code, the copy of an article's fields at a specific point in time is called a snapshot.

Revision field	What it means
snapshot	Copy of the article's content and important data at that point in time.
changed_fields	List of fields that changed compared with the previous revision.
captured_at	Date and time when that copy was created.
status_from / status_to	Status before and after the action.
user_id	Account that performed the recorded action.

Actions that create a revision

- Initial creation.
- A manual save that changes information.
- Submitting or resubmitting for review.
- Returning an article for changes.
- Scheduling, publishing, or canceling a schedule.
- Automatic publishing.
- Changing homepage flags.
- Archiving.
- Restoring a previous version.

Autosave works differently: it protects work in progress, but it does not add a revision every 15 seconds. This keeps History from filling up with hundreds of minor changes.

Editor feedback

When Mateo returns an article to Valentina and writes “Verify the figure in the second paragraph,” the feedback is stored in `article_review_feedback` together with the account that sent it. When Valentina corrects the article and resubmits it, that feedback can be marked as resolved.

Restoring a version

Only the editor can restore from History, and cannot do so on someone else's draft. Restoring does not erase what happened afterward. TRAMA takes the selected revision and creates a new Draft based on it, clears public dates, restores the category, tags, and cover image, rebuilds `search_text`, and records a new `restored_as_draft` revision.

Real source code: how a revision is stored

`app/Services/Editorial/ArticleRevisionService.php` manages revision creation. The current article is converted into a snapshot and compared with the previous snapshot to identify which fields changed:

```
$article->load('tags:id,name,slug');
$snapshot = $article->revisionSnapshot();

return $article->revisions()->create([
    'user_id' => $user->id,
    'action' => $action,
    'status_from' => $statusFrom,
    'status_to' => $article->status,
    'snapshot' => $snapshot,
    'changed_fields' => $this->changedFields($previousSnapshot, $snapshot),
    'restored_from_revision_id' => $restoredFrom?->id,
]);
```

`snapshot` preserves the state that remains after the action. `previousSnapshot` represents the earlier state, and `changedFields()` determines which fields changed between the two states. `restored_from_revision_id` preserves which historical revision was used as the source of a restoration.

Real source code: restoring does not directly replace a published article

`ArticleWorkflowService::restore()` takes only the permitted editorial fields from the selected snapshot and forces the result to Draft. It then restores tags, rebuilds `search_text`, and creates a new revision:

```
$restorable = Arr::only($snapshot, [
    'title', 'subtitle', 'excerpt', 'body', 'category_id',
    'cover_image', 'cover_alt', 'is_breaking', 'is_featured',
]);

$locked->update([
    ...$restorable,
    'status' => 'draft',
    'published_at' => null,
    'scheduled_at' => null,
]);

$locked->tags()->sync(array_map('intval', $snapshot['tag_ids'] ?? []));
$this->searchIndexer->index($locked);
$this->revisions->record(
    $locked,
    $user,
    'restored_as_draft',
    $previousStatus,
    $previousSnapshot,
    $lockedRevision,
);
```

The historical version is not silently republished. A new editable state is created, and the restoration itself is recorded, so the history preserves both what originally happened and the later decision to restore it.

Step by step: Mateo restores a revision without deleting the previous history

Restoration does not magically turn an old publication into the current state: it creates a new controlled working point.

Step	What the person does and what happens internally
1	Mateo opens the History tab for an article and selects an earlier revision.
2	The interface shows which fields that version contained and which values changed compared with other revisions.
3	When he chooses Restore, Laravel checks permissions and the article's current status again.
4	<code>ArticleWorkflowService</code> takes the recoverable data from the revision and applies it inside a protected transaction.
5	The restored article returns as a new editable draft instead of silently overwriting a publication that is already visible.
6	The restoration itself creates a new revision, so the history preserves both the old version and the decision to recover it.
7	Mateo or the author can continue working from that draft and go through the normal editorial workflow again.

19. Categories and Tags

Categories and tags are both used to organize articles, but they serve different purposes. A category is a main newspaper section, such as Politics or Economy. A tag represents a topic that can appear in articles from different sections, such as Artificial Intelligence.

Categories

Practical example: Marina creates a category

New Category form in the administrator panel.

Category creation also uses `useForm` in `Admin/Categories/Index.vue`. `createForm` stores the name, description, color, and cover image; `form.processing` disables the button while the request is being processed, and `form.errors` displays the problems returned by Laravel. New categories are created as active by the backend, so status is not part of `createForm`.

A duplicate name is displayed next to the field that needs to be corrected.

Each article belongs to a category. The administrator can create and edit its name, description, color, cover image, and status. A category appears on the portal only when it is active, has all of its public data completed, and has at least one available published article.

`CategoryEditorialOrderService` maintains the public order 1, 2, 3... If a category stops being displayed, the others are renumbered so there are no gaps. TRAMA allows at most 12 public categories at the same time.

What an Observer is and why TRAMA uses one with Article

An Observer is a class that reacts automatically to events on a Model. Instead of requiring every Controller or Service to remember a secondary task, Laravel can notify the Observer when a record is saved, deleted, restored, or permanently deleted.

TRAMA registers `ArticleObserver` from `AppServiceProvider`. The Observer watches for changes that can affect a category's public availability—for example, status, publication date, or `category_id`—and, once the database change has been committed, calls `CategoryEditorialOrderService` to resynchronize the public order.

The phrase “after the commit” means that the Observer waits until the transaction is confirmed before acting. This prevents it from reorganizing categories based on a change that might ultimately be rolled back.

Step	What the person does and what happens internally
1	Valentina saves an article and the change is committed in MySQL.
2	Laravel detects the event on the Article Model.
3	<code>ArticleObserver</code> checks whether a value relevant to public visibility changed.
4	If appropriate, it calls <code>CategoryEditorialOrderService</code> .
5	The service recalculates the public category sequence to avoid gaps or inconsistent positions.

A category cannot be deleted while it still has an associated article. In that case it can be deactivated, but not removed from the database.

Public category page

When a reader opens `/seccion/{slug}`, `Public\CategoryController::show()` looks for a category that meets the public visibility rules. The section must be active and have published content available. It then loads only publicly visible articles from that category, ordered from newest to oldest, and displays 9 per page in `Public/CategoryShow.vue`. `CategoryResource` prepares the section data and `ArticleCardResource` prepares each article.

Example: Candela opens the Economy section. If Economy is active, TRAMA displays its header and the first 9 public articles; the rest appear on later pages. If the category is deactivated, `/seccion/economia` is no longer a valid public page and Laravel returns a 404 instead of displaying a section that the administrator removed from the portal.

Tags and the many-to-many relationship

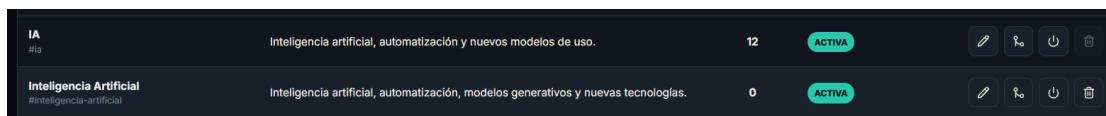
An article can have several tags, and the same tag can be used by many articles. For example, an Economy article can use AI, Companies, and United States; at the same time, AI can also appear in Technology or Politics articles.

In a database, this type of relationship is called many-to-many. TRAMA represents it through `article_tag`, a table that stores each combination of an article and a tag.

Merging duplicate tags

Practical example: Marina merges two tags

`Admin/Tags/Index.vue` maintains a separate `mergeForm` with `target_tag_id`. This prevents merge errors and processing state from being mixed with the create and edit forms. Laravel then performs the transactional operation that moves relationships and preserves the traceability of the merged tag.



IA	Inteligencia artificial, automatización y nuevos modelos de uso.	12	ACTIVA	   
Inteligencia Artificial	Inteligencia artificial, automatización, modelos generativos y nuevas tecnologías.	0	ACTIVA	   

Result of a tag merge in the grid.

Suppose the tags AI and Artificial Intelligence both exist and the administrator decides to keep AI as the primary tag. `TagMergeService` moves the relationships from articles that used Artificial Intelligence to AI, avoids creating duplicate combinations, and leaves the previous tag inactive with `merged_into_id` pointing to the primary tag.

```

BEFORE
Article 10 → Artificial Intelligence
Article 11 → AI

AFTER
Article 10 → AI
Article 11 → AI

Artificial Intelligence.merged_into_id → AI

```

A tag also cannot be deleted if it still has associated articles, if it was merged into another tag, or if other tags were merged into it.

Real source code: how tags are merged without duplicating relationships

The following fragment shows the core of the relationship move. `insertOrIgnore()` prevents a duplicate combination from being created when an article already uses the primary tag; the links to the secondary tag are then removed.

```

$articleIds = DB::table('article_tag')
    ->where('tag_id', $source->id)
    ->pluck('article_id');

foreach ($articleIds as $articleId) {
    DB::table('article_tag')->insertOrIgnore([
        'article_id' => $articleId,
        'tag_id' => $target->id,
    ]);
}

DB::table('article_tag')
    ->where('tag_id', $source->id)
    ->delete();

```

`insertOrIgnore()` prevents violation of the unique key on `article_tag` when an article already uses both tags. The links to the secondary tag are then removed. Deactivation of the source tag and its `merged_into_id` were already explained in the previous example, so they are not repeated in this fragment.

Step by step: Marina creates a category and later resolves a duplicate tag

Categories and tags serve different purposes, but both show how administration changes data that is later reused by articles and searches.

Step	What the person does and what happens internally
1	Marina opens Categories and fills in the name, description, color, and image. <code>useForm</code> keeps those values and disables the button while the request is being processed.
2	Laravel validates that the name is valid and not duplicated. If validation fails, Inertia returns the error to the corresponding field.
3	If the category is created successfully, <code>CategoryEditorialOrderService</code> adds it to the editorial order and MySQL stores the new row.
4	Articles can use that category through <code>category_id</code> , and the portal can display it when it is active and has public content.
5	Later, Marina notices two equivalent tags, for example AI and Artificial Intelligence, and opens the Merge action.
6	The merge service moves <code>article_tag</code> relationships to the primary tag without creating duplicate combinations.

Step	What the person does and what happens internally
7	The secondary tag is resolved according to the module's rules, and the articles retain their associated content.
8	ArticleSearchIndexer can rebuild search_text so that CMS search uses the final category and tag names.

20. Images: Validation, Processing, and Recovery

Images uploaded by the team to TRAMA go through a process before becoming available on the site. Laravel validates the received file, requests a local safety classification, and, if the content is accepted, Node.js and Sharp generate the final version in the format and dimensions required by each module. Only then is the processed file retained and the path used by the application recorded in MySQL.

What WebP is

WebP is an image format that can maintain good quality with a smaller file size. TRAMA converts article cover images, employee photos, category cover images, and processed advertisements to WebP.

Common process

Step	What happens
1	Laravel checks that the file uses a supported format, does not exceed the maximum size, and has sufficient dimensions.
2	SafedImageProcessor requests a local content classification before transforming the image.
3	ArticleImageSafetyPolicy compares the resulting scores with the limits configured by TRAMA and decides whether the file can proceed.
4	If the image is valid, Node.js runs scripts/images/process-image.mjs. Sharp corrects orientation, crops when necessary, resizes, and converts the result to WebP.
5	The final file is saved in the configured folder, and Laravel stores in MySQL the public path that the interface will use.

For an article cover image, the process can be read as follows: original file → Laravel validation → safety classification → Sharp processing → WebP 1600 × 900 → file saved → path recorded on the article.

EXIF and photo orientation

Some phones do not physically rotate the pixels of a portrait photo. Instead, they store data inside the file indicating how it should be displayed. That data is part of the EXIF metadata. Before cropping, Sharp applies that orientation so the photograph does not end up sideways.

Safety classification

The local model returns three values. SFW represents content considered safe; NSFW indicates sexual or adult content; NSFL is used for extreme graphic content. TRAMA blocks an image when the NSFW or NSFL probability reaches the 80% threshold configured for that category.

Use	Input file	Result
Article cover image	JPG/JPEG/PNG/WebP · max. 5 MB · sufficient resolution	WebP 1600 × 900 · quality 80

Use	Input file	Result
Employee photo	JPG/JPEG/PNG/WebP · max. 4 MB · minimum 600 × 600	WebP 600 × 600 · quality 82
Category cover image	Valid image with sufficient resolution	WebP 1600 × 900
Advertising	Valid image and exact size according to its placement	WebP at the required size

What Sharp is and what it does

Sharp is a Node.js library specialized in image processing. In TRAMA, it does not decide whether an image is safe or enforce editorial rules. It is used to read and transform pixels: correct EXIF orientation, resize, crop, remove the alpha channel when the model requires it, and generate WebP files.

Sharp Is Not the Classifier

Sharp performs deterministic transformations on the image. ONNX Runtime runs the machine-learning model. Laravel then applies the policy that decides whether the image may proceed.

How safety classification is run from PHP

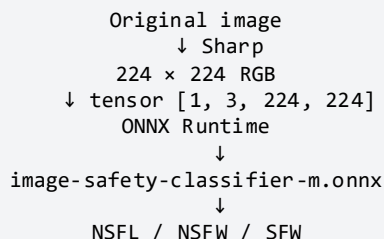
ArticleImageSafetyClassifier.php is the component that connects Laravel with scripts/images/classify-image-safety.mjs. The service obtains the Node path from config('trama.node_binary'), sets the project root as the working directory, and runs the script while passing the image's absolute path as a separate argument.

```
$result = Process::path(base_path())
->timeout(self::PROCESS_TIMEOUT_SECONDS)
->env([
    'SystemRoot' => getenv('SystemRoot'),
    'PATH' => getenv('PATH'),
])
->run([
    (string) config('trama.node_binary'),
    $scriptPath,
    $absoluteImagePath,
]);
```

Conceptually, the command is equivalent to `node scripts/images/classify-image-safety.mjs "path/to/the/image"`. The argument array avoids manually building a shell string. SystemRoot and PATH are passed explicitly because the original Windows environment requires them for Node and its dependencies to initialize correctly.

What classify-image-safety.mjs does

The script receives a physical path, prepares the image with Sharp, and runs the local model through onnxruntime-node. The image is rotated according to EXIF, resized to 224 × 224 pixels, converted to RGB without alpha, and its values are rearranged into a [1, 3, 224, 224] tensor representing one image, three channels, height, and width.



The script returns JSON through stdout containing the highest-probability category and the NSFL, NSFW, and SFW probabilities. ArticleImageSafetyClassifier converts that JSON into a PHP array and validates its structure. The accept-

or-block decision is not made in Node: ArticleImageSafetyPolicy then compares the probabilities with the limits defined by TRAMA.

```
$classification = json_decode(
    trim($result->output()),
    true,
    512,
    JSON_THROW_ON_ERROR
);
```

How final image processing is run

Once the image has been accepted, SafelImageProcessor.php starts scripts/images/process-image.mjs. In this case, the arguments include source, destination, width, height, and WebP quality.

```
$result = Process::path(base_path())
->timeout(self::PROCESS_TIMEOUT_SECONDS)
->env([
    'SystemRoot' => getenv('SystemRoot'),
    'PATH' => getenv('PATH'),
])
->run([
    (string) config('trama.node_binary'),
    $scriptPath,
    $absoluteSourcePath,
    $workingDestination,
    (string) $width,
    (string) $height,
    (string) $quality,
]);
```

process-image.mjs checks that the source exists, validates dimensions and parameters, corrects orientation, applies resize with fit=cover without enlarging small images, and writes the final WebP. It then returns JSON through stdout with the path, width, height, and format. Laravel checks that response before continuing.

Summary: classification and processing are two separate stages

Stage	Script	Technologies	Result
Security	classify-image-safety.mjs	Sharp + ONNX Runtime	NSFL / NSFW / SFW probabilities in JSON.
Decision	ArticleImageSafetyPolicy (PHP)	Laravel rules	Accepts or blocks according to the thresholds.
Final file	process-image.mjs	Sharp	WebP with the required dimensions and quality.

Step by step: Valentina uploads an accepted cover image

Step	What the person does and what happens internally
1	Valentina selects an image in Admin/Articles/Form.vue and saves the article. The file travels in the Inertia form as cover_image_file together with the rest of the editorial fields.

Step	What the person does and what happens internally
2	app/Http/Requests/Admin/ArticleRequest.php validates the request in Laravel. For the cover image, it accepts JPG, JPEG, PNG, or WebP and limits the file to 5 MB; the article's other rules depend on the status Valentina is trying to save.
3	ArticleController::store() receives the already validated ArticleRequest. If hasFile("cover_image_file") is true, it calls processCover(), which turns any processing RuntimeException into a validation error associated with the cover_image_file field itself.
4	ArticleCoverProcessor prepares the cover image destination: 1600 × 900 pixels, WebP quality 80, and an available name based on the article's stable slug. It then delegates the shared work to SafedImageProcessor.
5	SafedImageProcessor checks that the file exists and that its visual dimensions, including EXIF orientation, reach at least 1600 × 900. ArticleImageSafetyClassifier then runs scripts/images/classify-image-safety.mjs: Sharp prepares a 224 × 224 tensor and ONNX Runtime returns NSFL, NSFW, and SFW probabilities.
6	ArticleImageSafetyPolicy compares the probabilities with the configured thresholds. If the image is accepted, SafedImageProcessor runs scripts/images/process-image.mjs. Sharp corrects orientation, crops with fit=cover without enlarging a small image, and generates the final 1600 × 900 WebP at quality 80.
7	Laravel checks the JSON returned by Node.js, the dimensions, the format, and the existence of the generated file. ArticleController adds the URL and alt text to the article data, and ArticleWorkflowService::create() saves the article, synchronizes tags, and records the corresponding visual resource.
8	If everything finishes successfully, the cover image is referenced from MySQL and can be displayed in the panel and portal. If the file was generated but saving the article later fails, ArticleController removes the new cover image so no orphaned file is left behind. If processing fails earlier, the error is returned beneath the cover image field.

Step	What the person does and what happens internally
	<pre> Selected file: original-cover.jpg Size: 2400 × 1350 TRAMA checks: ✓ supported format ✓ under 5 MB ✓ sufficient dimensions ✓ allowed content Result: 1600 × 900 WebP quality 80 </pre>

What happens if someone deletes an image manually

MySQL can continue storing a path even when the physical file no longer exists. For example, `cover_image` may still point to `/images/articles/rutas-comerciales.webp` even if someone deleted `public/images/articles/rutas-comerciales.webp`.

On the portal, `Article::publicCoverImageUrl()` uses `noticia-default.webp` so a broken image is not displayed. In the panel, TRAMA detects that the path exists but the file is missing and requires the cover image to be uploaded again before completing an action that needs it. Something similar happens with an employee photo: `User::avatarUrl()` uses `avatar-default.webp`, and the edit form can require a new photo.

MySQL and files are not rolled back together

Suppose TRAMA has already generated a new cover image on disk and saving the article in MySQL then fails. MySQL can roll back its own changes, but it cannot delete that file by itself. For that reason, `ArticleController` removes the new cover image when the later save fails.

The “all-or-nothing” behavior that MySQL guarantees inside a transaction is called atomicity. The file system is outside that transaction, so TRAMA adds specific code to clean up new files or restore a previous version when appropriate.

21. Comments and Registered-User Engagement

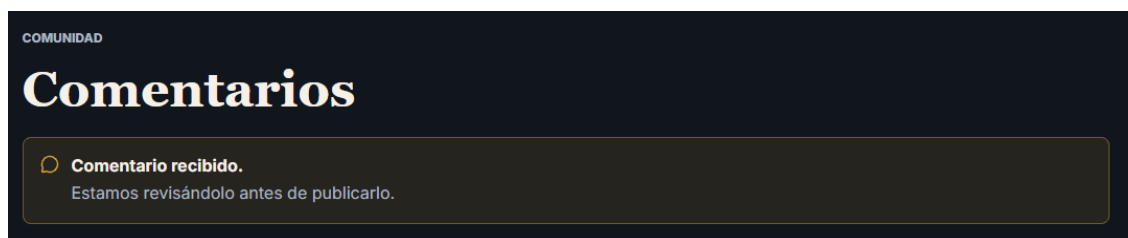
A registered user can write a top-level comment on an article and reply to comments from other users. Before appearing publicly, comments and replies go through automatic review.

Example: submitting a comment

Candela writes “Very good article; it explains the problem clearly” and clicks Submit comment. Laravel first checks the length, permissions, and commenting rules. If everything is valid, it creates a row in comments with `status=processing`. This means the comment already exists in MySQL, but is not yet displayed publicly.

Before saving, `CommentController` checks that the article is still published, normalizes the text, and validates that it contains between 8 and 1,200 characters. It also verifies that the account is active, has permission to participate, and does not already have another active top-level comment on the same article. If the comment is a reply, it also checks that the parent comment is an approved top-level comment and that the reply meets the thread rules.

For a regular reader, the row is created inside a transaction with `status=processing`, `moderation_revision=1`, `moderation_source=automatic`, and `moderation_reason=automation_queued`. Laravel then dispatches `ModerateComment` to the moderation queue and returns the review message to the browser together with the comment ID that the interface must track. Automatic analysis happens afterward; the web request does not wait for the models to finish.



Message shown to the user while the comment is being reviewed.

Status	What it means
processing / Processing	The comment has been saved, but the analysis has not finished yet.
approved / Approved	The comment appears publicly.
pending / Pending	Only its author can see it, and it is sent to the editor for manual review.
rejected / Rejected	It does not appear in the public conversation.

Conversation rules

Candela can write one active top-level comment per article. Pablo can reply to it once. Candela cannot reply to her own top-level comment. A reply cannot receive another reply: TRAMA keeps the thread to a single level of depth. If Pablo deletes his reply, he can reply to that comment again.

The processing, pending, and approved statuses count as active comments. If the top-level comment becomes rejected, the user can try to write another one.

Editing a pending comment

Only a comment that is Pending moderation can be edited. If it is still processing, the user must wait. If it has already been approved or rejected, it cannot be edited through this workflow. When a pending comment is modified, it returns to processing, `moderation_revision` increases, and it is analyzed again from scratch.

Usage limits

Action	Limit
Create a comment or reply	5 attempts every 10 minutes.
Edit or delete your own comment	20 actions per minute.
Add or remove a Like	10 actions every 5 minutes.
Report comments	10 attempts every 10 minutes.

Likes and reports

A user cannot Like their own comment, and MySQL prevents the same account from storing two identical votes on the same comment. Reporting does not automatically remove the content: it creates a case for the editor. The same account cannot report the same comment twice or report its own content.

If a user had Liked a comment and then reports that same comment, TRAMA removes the Like before saving the report so the two actions are not active at the same time.

“My Comments”: personal comment history

The `/mis-comentarios` route opens a private screen for the authenticated account. `MyCommentController` queries only that user’s comments, orders them from newest to oldest, and displays them in pages of 10. The grid shows an excerpt, article, status, Like count, and reply count.

Status	How it appears in “My Comments”
processing	Displayed to confirm that TRAMA received the comment while automatic moderation is still running.
pending	Displayed as pending and lets the user identify which comment is awaiting manual review.
approved	Displayed with a link to the article; the link includes <code>?comentario={id}</code> so Vue can locate and center that comment.
rejected	Only rejections whose <code>moderation_source</code> is editorial are retained in this grid. No public link is generated, and the details open within the screen itself.

The article link is also not generated if the article is no longer published, has not yet reached its public date, or its category is inactive. If the targeted comment falls outside the first 10 comments normally loaded by `ArticleController`, the controller includes it additionally so the jump from “My Comments” works.

Example

Candela opens “My Comments” and selects an approved comment on an article with 40 comments. The URL is built as `/noticias/{slug}?comentario=123`. `ArticleController` looks up comment 123, checks that Candela is allowed to see it, and—even if it was not in the first batch—adds it to the data sent to `ArticleShow.vue`. The interface can then take the user directly to her comment.

Step by step: Candela submits a comment and TRAMA prepares moderation

Posting a public comment looks like a simple action, but before it is displayed it passes through several account, article, and status rules.

Step	What the person does and what happens internally
1	Candela signs in, opens a published article, and writes a top-level comment.
2	Vue sends the request to the comments route. Laravel checks authentication, usage limits, and that Candela does not already have another active top-level comment on that article.
3	If the data is valid, <code>CommentController</code> saves the row in <code>comments</code> with <code>status=processing</code> , <code>moderation_revision=1</code> , and <code>automatic</code> as the source.
4	The web request does not wait for all the models to finish: Laravel dispatches <code>ModerateComment</code> to the moderation queue.
5	Candela returns to the article and sees that her comment is being reviewed instead of being published immediately.
6	A worker takes the Job and runs the automatic pipeline explained in the next chapter.
7	When the status changes, the screen can update the relevant information and show the comment as approved, pending, or rejected according to the decision.

22. Automatic Comment Moderation

When a user submits a comment, TRAMA does not force the browser to wait while every check is performed. Laravel first saves the comment and leaves the analysis as pending work. That work enters a queue, and a separate process called a worker takes it and runs moderation in the background.

How to start the moderation worker

Dispatching `ModerateComment` only leaves the work pending in the queue. For that Job to run during development, a worker must be consuming the moderation and default queues.

```
php artisan queue:work --queue=moderation,default --tries=3 --timeout=70
```

If the worker is not active, the comment remains saved with `status=processing` and the Job waits in the queue until a worker processes it. A Job, a queue, and a worker are different pieces: `ModerateComment` is the work; moderation is the queue; and `queue:work` is the process that consumes that queue.

Step	What happens
1	<code>CommentController</code> saves the comment as processing.
2	Laravel dispatches <code>ModerateComment</code> as a Job and leaves it pending in the moderation queue.
3	The browser can already display “Comment received.”
4	The worker takes the Job.
5	Node.js runs <code>scripts/comments/moderate-comment.mjs</code> .
6	Laravel receives the result and updates the same comment.

Analysis sequence

```

Language
↓
Links
↓
Threats
↓
Toxicity
↓
Spam

```

Scripts that control moderation

Script	What it does
<code>moderate-comment.mjs</code>	This is the only script Laravel runs directly. It receives the comment ID, <code>moderation_revision</code> , and text; validates the input, coordinates the pipeline, and returns a single JSON decision: approved, pending, or rejected.

Script	What it does
detect-comment-language.mjs	It determines whether the text can be treated as Spanish using linguistic rules and franc as a secondary statistical signal. A foreign language or confirmed language mixture is routed to pending. If the language remains unknown because the text is too short or insufficient, the safety checks can continue.
detect-comment-links.mjs	It is deterministic and does not use AI. It looks for standard URLs and obfuscated forms, such as domains without a protocol, hidden dots, or modified protocols. If it finds a link, the coordinator rejects the comment and stops the pipeline.
detect-comment-threats.mjs	It combines contextual and structural rules with a local ONNX model as a selective second opinion. A HIGH threat produces rejection; MEDIUM or review_recommended leaves a provisional reason for manual review and the analysis can continue.
classify-comment-toxicity.mjs	It uses a multilingual DistilBERT ONNX model together with contextual rules, normalization, and personal-attack detection. review_recommended routes the comment to review; a toxicity score of 0.70 or higher, without that recommendation, can produce automatic rejection.
classify-comment-spam.mjs	It uses a Spanish-language ONNX spam model and rules for offers, solicitation, promotions, phishing, and other signals. review_recommended routes the comment to review; a score of 0.78 or higher, without that recommendation, can produce automatic rejection.

Laravel does not start five Node.js processes. ModerateComment makes a single call through Process::run() to the moderate-comment.mjs coordinator. From there, that coordinator invokes the other modules within the same execution and returns one result to PHP.

Language and link checks are loaded at the beginning because they are the first checks. Threat, toxicity, and spam modules are imported only if the comment reaches those stages. This allows TRAMA to avoid loading heavier models when an earlier check has already resolved the case. When the files are installed, the classifiers look for their models and tokenizers in resources/models and run them locally through onnxruntime-node; this workflow does not require a moderation HTTP server.

Not every step always runs. If a prohibited link is detected, for example, the comment can be rejected at that stage without loading the later analyzers. An uncertain signal can leave the comment Pending for an editor to review.

Real source code: how Laravel starts Node.js

ModerateComment.php is where PHP actually starts the moderation coordinator. The Job does not run five separate processes: it calls moderate-comment.mjs once, and that module imports the detectors it needs during the same execution.

```
$process = Process::path(base_path())
->timeout(self::PROCESS_TIMEOUT_SECONDS)
->env([
    'SystemRoot' => getenv('SystemRoot'),
    'PATH' => getenv('PATH'),
])
->run([
```

```

        $nodeBinary,
        $scriptPath,
        (string) $comment->id,
        (string) $this->moderationRevision,
        (string) $comment->body,
    ]));

```

`Process::path(base_path())` sets the TRAMA root as the working directory. `timeout()` limits the process duration. `env()` passes the required variables to the subprocess, and `run([...])` supplies each argument separately. Conceptually, the execution is equivalent to `node scripts/comments/moderate-comment.mjs 85 3 "text..."`.

The coordinator writes a single JSON response to stdout. Laravel uses `$process->output()`, decodes it, and validates that the comment, moderation_revision, and decision correspond to the Job being processed. A nonzero exit code is treated as a technical failure and never as automatic approval.

```

$result = json_decode(
    trim($process->output()),
    true,
    512,
    JSON_THROW_ON_ERROR,
);

```

Step by step: Candela submits a normal comment and it is approved

Suppose Candela writes “Very good article; it explains the problem clearly.” If no risk signal appears, the expected path for that comment is as follows:

Step	What happens
1	CommentController validates the article, account, text, and commenting rules. It saves the same row in comments as processing, with moderation_revision=1.
2	Laravel dispatches ModerateComment to the moderation queue and responds to the browser without waiting for the analyzers.
3	The worker takes the Job. ModerateComment uses Process::run() and the executable configured in TRAMA_NODE_BINARY to start moderate-comment.mjs.
4	detect-comment-language.mjs recognizes the text as suitable to continue, and detect-comment-links.mjs finds no links.
5	The coordinator then loads the threat, toxicity, and spam detectors. None produces a rejection reason or recommends manual review.
6	moderate-comment.mjs builds JSON with decision=approved and reason=automatic_clean and writes it to stdout.
7	ModerateComment validates the JSON, checks that moderation_revision is still the same, and changes the row from processing to approved.
8	ArticleShow.vue, which polls the status while the comment is processing, detects the change and the comment can appear without reloading the entire page.

Step by step: Candela submits a comment with a clear threat

Suppose Candela writes “If you keep talking about me, I'm going to hit you” and clicks Submit comment. The purpose of this example is to follow the internal path of a direct physical threat until TRAMA decides the content must not be published.

1. Laravel receives the comment. `CommentController` checks the normal commenting rules and, if they are valid, saves the row in comments with `status=processing`. The comment already exists in MySQL, but is not public yet. `moderation_revision` identifies which version of the text is being analyzed.
2. Moderation moves to the queue. Laravel dispatches the `ModerateComment` Job to the moderation queue. The web request can finish, and the user sees that the comment was received and is being reviewed.
3. The worker takes the Job. The queue process runs `ModerateComment`, which then starts `scripts/comments/moderate-comment.mjs` to coordinate the different Node.js checks.
4. Analysis proceeds in stages. The language is checked first, followed by links. Because the example text can be analyzed normally and contains no links, the chain continues to the threat check.
5. The threat detector combines contextual rules with a local model. In this example, the signal is clear enough to reject the comment. Because a terminal decision already exists, the later toxicity and spam analyzers do not have to be loaded.
6. The result returns to Laravel. Node.js provides a structured response, and `ModerateComment` checks that `moderation_revision` still matches the text that was analyzed. If the user had created a newer revision, the older result would be discarded.
7. Laravel updates the same comments row from processing to rejected. A second comment is not created: the status of the already stored comment changes.
8. The interface detects the new status and stops displaying the review notice. The user receives the general rejection message; TRAMA does not publicly expose which detector was involved or the internal score that produced the decision.

Examples of results

Comment	Internal result	What the user sees
"Very good article, thanks for the information."	approved / automatic_clean	The review notice disappears and the comment appears.
"Look at this: https://example.com"	rejected / link_detected	"Your comment could not be published because it violates the commenting rules."
Text entirely in an unsupported language	pending / unsupported_language	"Your comment is pending moderation."
Clear threat, toxicity, or spam	rejected	The same public rejection message is used without showing the detector or its score.

Model scores

The models return a number between 0 and 1 indicating how strong the detected signal was. In the code, this value is called `score`. With the current configuration, clear toxicity can trigger automatic rejection from 0.70, and spam uses 0.78, provided that the detector itself did not recommend manual review.

What JSON, stdout, and stderr are

When Node finishes, it needs to communicate the result to Laravel. It returns a small block of data in JSON, a text format used to exchange information between programs. A normal response is written to `stdout`, the process's standard output. If an error occurs, technical details are written to `stderr`. Laravel uses this distinction to determine whether it received a valid decision or must retry the Job.

How TRAMA prevents an old result from being applied

Each comment has `moderation_revision`. Suppose Candela submits “Good article” and analysis begins with revision 1. Before it finishes, she edits the pending comment and writes “Good article, very thorough.” TRAMA increases `moderation_revision` to 2 and creates a new Job.

```
revision 1 → Job starts
Candela edits → moderation_revision = 2
revision 1 Job finishes → result discarded
revision 2 Job → only result that may be applied
```

What happens if automation fails

`ModerateComment` can make up to 3 attempts, with waits of 5, 15, and 30 seconds. If the queue cannot even receive the Job or the Job exhausts its attempts, the comment is left Pending for manual review. A technical error is never interpreted as approval.

Updating without reloading the page

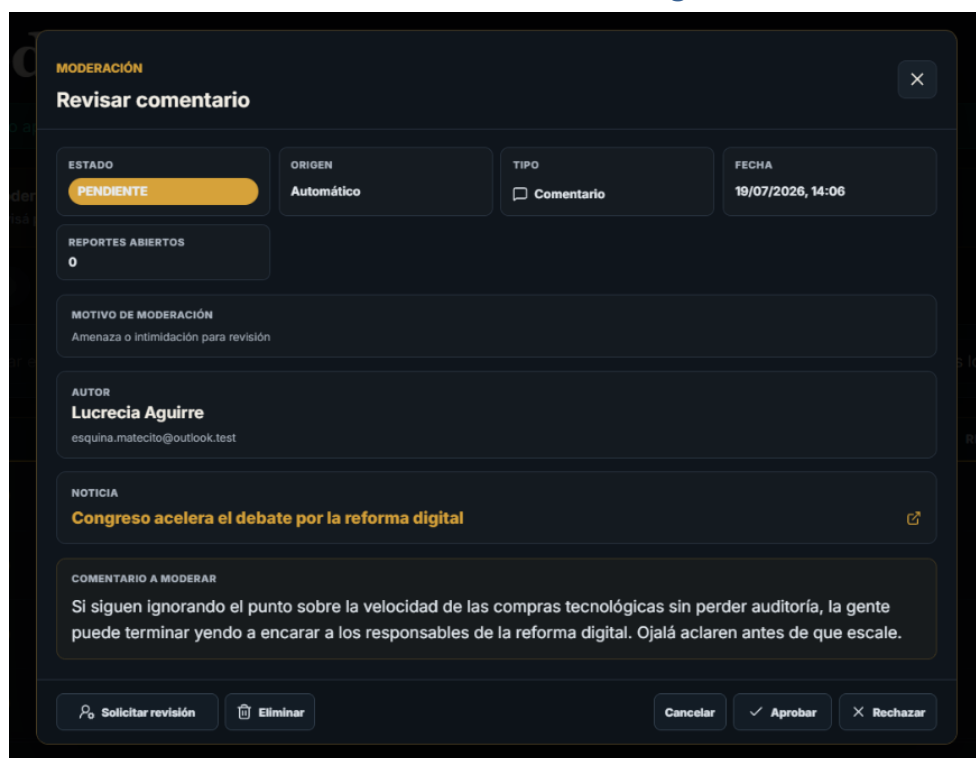
`ArticleShow.vue` polls the status every 4 seconds while the user has a comment being processed. When analysis finishes, it stops polling and updates the necessary part of the article. This is why an approved comment can appear without the user manually reloading the entire page.

23. Manual Review of Comments and Accounts

Manual review begins when automatic review does not have a sufficiently clear signal to approve or reject a comment. In that case, TRAMA leaves it Pending: the content is not shown to other users and is sent to the editor's Comments module so a person can make the decision.

Complete example: a Pending comment reaches the editor

What Mateo sees when automation leaves the case Pending



MODERACIÓN

Revisar comentario

ESTADO: **PENDIENTE**

ORIGEN: Automático

TIPO: Comentario

FECHA: 19/07/2026, 14:06

REPORTES ABIERTOS: 0

MOTIVO DE MODERACIÓN: Amenaza o intimidación para revisión

AUTOR: **Lucrecia Aguirre**
esquina_matecito@outlook.test

NOTICIA: **Congreso acelera el debate por la reforma digital**

COMENTARIO A MODERAR: Si siguen ignorando el punto sobre la velocidad de las compras tecnológicas sin perder auditoría, la gente puede terminar yendo a encarar a los responsables de la reforma digital. Ojalá aclaren antes de que escale.

Solicitar revisión Eliminar Cancelar Aprobar Rechazar

Details of a Pending comment in the Moderation module.

The screenshot helps separate two moments: automation has already finished and left a pending decision; from this point on, the editor acts on information Laravel retrieves from MySQL. Approving, rejecting, deleting, or referring the account are separate routes and each checks permissions again on the server.

Form Mateo uses to refer the account to the administrator together with the comment as evidence.

Now suppose Candela writes “Keep talking about me and I'll find you and make you pay.” The detector finds a concerning intimidation signal, but not a sufficiently clear direct physical threat to reject automatically. TRAMA changes the status from processing to pending and ends the automatic Job so the case can move to manual review.

Mateo opens the Comments module and enters the detail view. There he can see that the status is Pending, that the source was automatic, who wrote the comment, which article it appears on, the moderation reason, and the full text he needs to evaluate.

At this point, automation no longer decides. The editor reviews the context and chooses what to do with the comment.

What the editor can do

From the detail view, Mateo can Approve, Reject, Delete, or Request account review. These are different actions, and each has a specific effect on the comment or on the account that posted it.

Action	What happens
Approve	The comment becomes Approved and can be displayed publicly.
Reject	It remains Rejected, is no longer displayed, and preserves its moderation record.
Delete	The comment is permanently deleted from the moderation module.
Request review	Sends the case to the administrator with the comment as evidence. This action does not block the account by itself.

What happens when a published comment is reported

A report begins from a different situation. The comment was already Approved and visible when another user reported it. Reporting it does not automatically remove it from the portal: it creates a case so the editor can review the content and the reason for the report.

Mateo opens the detail view and can see how many reports exist and which reason each person selected. If he chooses Keep, the comment remains published. If he chooses Remove, it is no longer displayed on the portal.

Rejecting is not deleting

Rejecting changes the comment's status and leaves the case available as a moderation record. Deleting is permanent. That is why TRAMA presents the two options separately.

Team replies

A journalist can reply to comments on an article they authored, and the reply is identified as Article Author. The editor can reply as TRAMA Team. These internal replies are created directly as Approved and do not go through the automatic filters used for registered users. The administrator does not participate in the public conversation.

From a comment to an account review

Requesting a review does not automatically block the account. If Mateo believes that, in addition to resolving the comment, the user's behavior warrants administrative evaluation, he can send the case to Marina. TRAMA stores the reason, the note, the comment used as evidence, the article title, and who requested the review. The administrator then decides whether to close the case without taking action or block the account.

users keeps a summary of the account's current administrative status, while `user_moderation_reviews` stores the reviews performed while that account exists. If the registered user is deleted, those rows are deleted together with the account according to the rule defined in MySQL.

Example

Mateo rejects a comment, decides that the account's behavior needs review, and sends the case to Marina. Marina can block the account or close the review without taking action.

Where the code comes from: from Mateo's modal to the Controller

The PHP block shown later does not run in isolation. First, Mateo selects a row in `Admin/Comments/Index.vue`. The `resources/js/Components/Admin/CommentModerationModal.vue` modal loads the details on demand with an Axios request to `admin.comments.show`:

```
const response = await window.axios.get(
  route('admin.comments.show', props.commentId),
);

comment.value = response.data?.comment || null;
```

The `GET /admin/comments/{comment}` route reaches `CommentController::show()`. The controller checks `canModerateComments()` again, prevents internal team interventions from being opened through a direct URL, loads the article, author, parent context, and open reports, and transforms the result with `AdminCommentDetailResource`. Only then does the modal have the complete information Mateo needs to decide.

When Mateo clicks Approve, Reject/Remove, or Keep, the modal does not modify the local object as if the change were already valid. It sends the action to Laravel through Inertia `router.patch()`:

```
function moderate(action) {
  router.patch(
    route('admin.comments.update', comment.value.id),
    { action },
    {
      preserveState: true,
      preserveScroll: true,
      onSuccess: () => closeModalAfterSuccess(),
    },
  );
}
```

routes/web.php connects that request to `CommentController::update()`. Before entering the business logic, `CommentModerationRequest` normalizes action and allows only approve, reject, or maintain; a manipulated request with any other value is rejected before changes are applied.

```
Route::get('comments/{comment}', [AdminController::class, 'show'])
    ->name('comments.show');

Route::patch('comments/{comment}', [AdminController::class, 'update'])
    ->name('comments.update');

// CommentModerationRequest
'action' => [
    'required',
    'string',
    Rule::in(['approve', 'reject', 'maintain']),
],
```

Only then does the Controller code shown below run. `DB::transaction()` and `lockForUpdate()` guarantee that the decision is made against the row's actual state and that two editors cannot confirm incompatible actions at the same time.

Real source code: locking before an editorial decision

`CommentController::update()` processes the decision inside a transaction and reads the comment again with `lockForUpdate()` before modifying it. The following fragment shows that concurrency point: two editors cannot confirm incompatible decisions on the same row at the same time.

```
DB::transaction(
    function () use (
        $comment,
        $action,
        $editorId,
    ): void {
        $lockedComment = Comment::query()
            ->whereKey($comment->id)
            ->lockForUpdate()
            ->firstOrFail();
```

Once the row is locked, the same method applies approve, reject, or maintain according to the current status. The following table summarizes the effect of each action, including the removal of replies and resolution of reports where applicable.

What changes internally for each decision

Visible action	Request / method	Main internal result
Approve	PATCH admin.comments.update · action='approve'	Requires status=pending. Stores status=approved, moderation_source=editorial, and moderation_reason=editorial_approved.
Reject / Remove	PATCH admin.comments.update · action='reject'	Stores status=rejected. If it was a top-level comment, removes active replies and resolves open reports.
Keep	PATCH admin.comments.update · action='maintain'	Only for an approved comment with open reports. Keeps it published and marks those reports as reviewed/dismissed.

Visible action	Request / method	Main internal result
Delete	DELETE admin.comments.destroy	Opens a separate confirmation. The Controller locks the row and permanently deletes it; if the parent has replies, the database relationship applies the consequence defined for them.
Request account review	PATCH admin.comments.refer-to-admin	Stores an administrative review with a reason, note, and the comment as evidence. It does not change is_active and does not block the reader by itself.

Deleting and referring the account use separate routes because they are not moderation statuses equivalent to approve/reject/maintain. The modal calls `router.delete()` to destroy the comment and another PATCH request to refer-to-admin with reason and note.

Step by step: Mateo receives a pending comment and decides what to do

Pending means that automation has finished and left the decision in the editor's hands.

Step	What the person does and what happens internally
1	Candela submits a comment and the automatic pipeline finishes with <code>status=pending</code> . The row already exists in comments, but the content is not public.
2	Mateo opens <code>Admin/Comments/Index.vue</code> and filters by Pending. Laravel had already built the paginated grid; that row contains only the summary needed for the list.
3	Mateo opens the case. <code>CommentModerationModal.vue</code> runs <code>loadCommentDetail()</code> and makes a GET request to <code>admin.comments.show</code> using the comment ID.
4	<code>CommentController::show()</code> checks permissions, loads the article, author, parent comment, reply count, and open reports, and returns <code>AdminCommentDetailResource</code> as JSON.
5	The modal displays the full text and context. Mateo chooses Approve, and <code>moderate("approve")</code> sends a PATCH request to <code>admin.comments.update</code> with <code>{action: "approve"}</code> .
6	<code>CommentModerationRequest</code> normalizes the action and checks that it belongs to the closed list approve/reject/maintain. If it is invalid, the Controller does not apply the decision.
7	<code>CommentController::update()</code> checks <code>canModerateComments()</code> again, opens <code>DB::transaction()</code> , and rereads the row with <code>lockForUpdate()</code> . This ensures the decision is based on the current state rather than a stale copy.
8	Because the comment is still pending, Laravel updates it to approved and records <code>moderation_source=editorial</code> and <code>moderation_reason=editorial_approved</code> . Other actions execute their own specific rules.
9	The Controller returns <code>back()->with("status", ...)</code> . Inertia updates the grid and counts while preserving filters and scroll position; the modal closes after the successful operation.

Step	What the person does and what happens internally
10	If Mateo also believes Candela's behavior should be evaluated, he opens Request review. That second route records a case for Marina with the comment as evidence, but the account remains active until an administrative decision is made.

24. Blocking and Deleting Users and Employees

Blocking is not deleting

Practical example: Marina blocks an account with prior incidents

Blocking dialog: the administrator sees moderation context before deciding.

The blocking form uses a separate useForm instance called blockForm. It contains the reason and administrative note; blockForm.processing controls submission, and blockForm.errors keeps errors from that decision separate from the user edit form.

From the blocking form to the backend

For an active reader, Admin/Users/Index.vue opens a dedicated dialog and uses blockForm, created with useForm(), to store blocked_reason and blocked_note. The button does not change is_active locally: blockReader() sends a PATCH request to the real administrative route.

```
function blockReader() {
  blockForm.patch(
    route('admin.users.toggle-active', blockTarget.value.id),
    {
      preserveScroll: true,
      onSuccess: () => closeBlockAfterSuccess(),
    },
  );
}
```

routes/web.php connects PATCH /admin/users/{user}/toggle-active with UserController::toggleActive(). Before that, ToggleUserStatusRequest authorizes administrators only. When the target is an active reader, it requires an allowed reason and an internal note of 10 to 200 characters; for employees, those fields are not part of the administrative status.

```
Route::patch(
  'users/{user}/toggle-active',
  [AdminUserController::class, 'toggleActive']
)->name('users.toggle-active');

// ToggleUserStatusRequest
'blocked_reason' => [
  Rule::requiredIf($blockingReader),
  'nullable',
  Rule::in(['moderation', 'abuse', 'spam', 'security', 'terms', 'other']),
],
'blocked_note' => [
  Rule::requiredIf($blockingReader),
  'nullable', 'string', 'min:10', 'max:200',
],
```

Real source code: what UserController::toggleActive() changes

The Controller also protects rules the interface cannot guarantee by itself: Marina cannot deactivate her own account, and TRAMA does not allow the last active administrator to be blocked. Inside a transaction, it looks up the user again with lockForUpdate() and calculates the final state.

```
$locked = User::query()
  ->whereKey($user->id)
  ->lockForUpdate()
  ->firstOrFail();

$willBeActive = ! $locked->is_active;

$statusData = [
  'is_active' => $willBeActive,
  'disabled_at' => $willBeActive ? null : TramaClock::now(),
  'remember_token' => $willBeActive ? $locked->remember_token : null,
  'updated_at' => TramaClock::now(),
];

if ($locked->role === 'reader') {
  $statusData['blocked_reason'] = $willBeActive
    ? null : $request->validated('blocked_reason');
  $statusData['blocked_note'] = $willBeActive
    ? null : $request->validated('blocked_note');
}

$locked->forceFill($statusData)->save();
```

When blocking results from a pending administrative review, TRAMA also closes that review so the case and account status do not contradict each other. Reactivating reverses is_active and clears disabled_at; for readers, it also clears blocked_reason and blocked_note from the current state without deleting the history stored in user_moderation_reviews.

What happens to a session that was already open

```

Marina saves the block
↓
users.is_active = false
↓
Candela keeps a tab open
↓
Candela makes the next protected request
↓
EnsureActiveAccount checks the status again
↓
detects is_active = false
↓
invalidates that specific session
↓
Candela returns to login with the blocked-account message

```

By design, `toggleActive()` does not need to immediately delete every row in sessions. `is_active` makes the block effective on the server from the moment the transaction is committed, and `EnsureActiveAccount` signs out each open session the next time it uses the application. This also covers sessions open on other devices.

Blocking an account does not delete its data. The row remains in `users`, but `is_active` becomes false and the account can no longer sign in. If it already had an open session, `EnsureActiveAccount` invalidates it on the next request.

For a registered user, the administrator also stores a reason and an internal note. Reactivating the account restores access, but does not erase administrative decisions that were already recorded.

Employees with history

A journalist, editor, or administrator may have contributed to information TRAMA needs to preserve, such as articles, revisions, internal comments, or editorial feedback. In those cases, the internal account is not deleted: it is deactivated so historical references remain intact.

Creating, editing, and restoring employee access

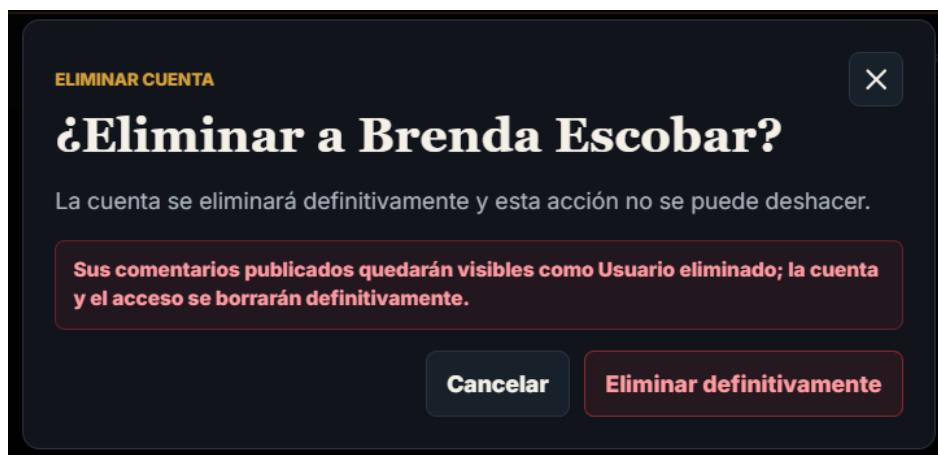
From `Employees`, the administrator can create internal administrator, editor, or journalist accounts. `StoreEmployeeRequest` validates the name, email identifier, role, position, biography where applicable, initial password, and optional photo. The email address is built with the internal `@trama.test` domain and, when the account is created, `UserController` records `email_verified_at` using `TramaClock`'s editorial time. If a photo is uploaded, it is processed as a 600 × 600 WebP before its path is stored.

Editing allows identity, role, position, biography, and avatar to be updated, while retaining administrative safeguards: an administrator cannot remove their own administrator role, and TRAMA does not allow the system to be left without another active administrator. An active account also has Reset access: `UserController::resetAccess()` creates a recovery token, sets `password_reset_required_at`, deletes open sessions, and only then sends the link by email.

Example: creating and resetting access for a journalist

Marina creates Sofía as a journalist and defines her email identifier, position, biography, initial password, and photo. Laravel creates the internal account and leaves it ready to sign in. Later, if Sofía needs to recover access, Marina uses Reset access. TRAMA invalidates previous sessions and sends a link; until that flow is completed, `password_reset_required_at` allows the system to require a password change before the CMS can be used normally again.

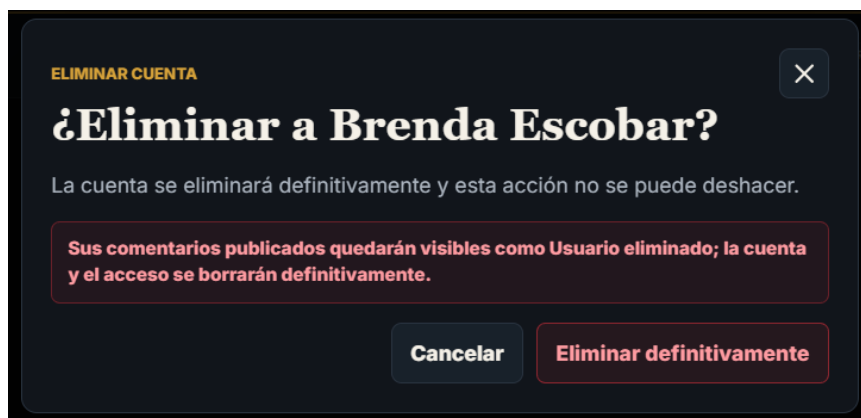
Example: deleting a registered user who has comments



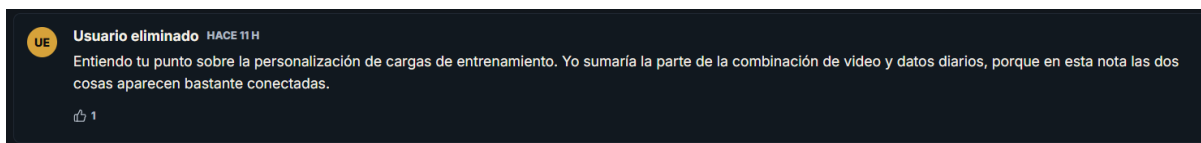
Confirmation for permanently deleting a registered account.

The visible result follows the policy explained in this section: the account disappears, but public comments that must remain part of the conversation are preserved and anonymized as “Usuario eliminado”.

Public comments do not by themselves prevent a registered account from being deleted. Before deleting users, UserController detaches those comments and removes the identifying data that belonged to the account.



Confirmation explaining what will happen before the account is deleted.



After the account is deleted, the comment remains with the author “Usuario eliminado”.

Before	After
comments.user_id = 125	comments.user_id = NULL
comments.author_name = "Brenda Escobar"	comments.author_name = "Usuario eliminado"
comments.author_email = "..."	comments.author_email = NULL
Account sessions	Deleted.
Likes from that account	Deleted.
Reports submitted by that account	Deleted.

Before	After
user_moderation_reviews for that account	Deleted.

A registered account cannot be deleted if it appears as the author of articles, the person responsible for editorial revisions, or the person responsible for returned-for-changes feedback. Those relationships are part of the editorial history, and the database prevents them from being broken.

An administrator is also not allowed to delete their own account through this action. If deletion of an active administrator is attempted, the system checks that at least one other active administrator exists.

Real source code: what is checked before an account is deleted

The DELETE /admin/users/{user} route reaches UserController::destroy(). Deleting is more restrictive than blocking: before removing the row, it locks it again with lockForUpdate() and checks whether historical content exists that must retain references to that person.

```
if ($request->user()->is($user)) {
    throw ValidationException::withMessages([
        'user' => 'No podés eliminar tu propia cuenta.',
    ]);
}

$locked = User::query()
    ->whereKey($user->id)
    ->lockForUpdate()
    ->firstOrFail();

if ($this->deletionBlockedByHistoricalContent($locked)) {
    throw ValidationException::withMessages([
        'user' => 'La cuenta conserva contenido histórico y debe desactivarse en lugar de eliminarse.',
    ]);
}
```

If the account can be deleted, destroy() deletes its sessions, first anonymizes a reader's comments, and finally deletes users. For employees with authorship, revisions, or returned feedback, the previous rule requires blocking/deactivation instead so editorial history is preserved.

Real source code: anonymizing comments before deleting a reader

When the account belongs to a registered user and there is no editorial history preventing deletion, UserController deletes its sessions and calls detachReaderCommentsBeforeDelete() before deleting users. The comments remain, but no longer identify the person:

```

DB::table('sessions')
    ->where('user_id', $locked->id)
    ->delete();

$this->detachReaderCommentsBeforeDelete($locked);
$locked->delete();

private function detachReaderCommentsBeforeDelete(User $user): void
{
    if ($user->role !== 'reader') {
        return;
    }

    Comment::query()
        ->where('user_id', $user->id)
        ->update([
            'user_id' => null,
            'author_name' => 'Usuario eliminado',
            'author_email' => null,
            'updated_at' => TramaClock::now(),
        ]);
}

```

This operation explains the “before/after” shown earlier: the comment remains part of the conversation, but `user_id` and `author_email` no longer identify the deleted account, and `author_name` becomes “Usuario eliminado”. Internal accounts with historical relationships are deactivated instead of following this deletion process.

Step by step: Marina blocks an account and the block takes effect on the next request

Blocking preserves the account and its history; deleting it is a separate, permanent decision.

Step	What the person does and what happens internally
1	Marina opens Registered Users in Admin/Users/Index.vue. The row already includes status, prior incidents, and, when present, the review referred from Moderation.
2	Marina clicks Block. Because Candela is a reader and is active, Vue opens the blockForm dialog instead of applying the change immediately.
3	Marina selects the reason and writes the note. <code>blockReader()</code> sends PATCH to <code>admin.users.toggle-active</code> with <code>blocked_reason</code> and <code>blocked_note</code> .
4	<code>ToggleUserStatusRequest</code> checks that the acting account is an administrator, normalizes the note, and requires the administrative fields because an active reader is being blocked.
5	<code>UserController::toggleActive()</code> prevents self-blocking, opens a transaction, rereads users with <code>lockForUpdate()</code> , and checks safeguards such as protection of the last active administrator when applicable.
6	Laravel stores <code>is_active=false</code> , <code>disabled_at=TramaClock::now()</code> , <code>remember_token=null</code> and, for the reader, <code>blocked_reason</code> and <code>blocked_note</code> . If there was a pending review, it resolves it as blocked.
7	The response returns to Admin/Users/Index.vue and the panel displays the account as blocked. The users row and its history continue to exist.
8	If Candela had an open session, her next request passes through <code>EnsureActiveAccount</code> . The middleware detects <code>is_active=false</code> , invalidates that session, and prevents her from continuing.

Step	What the person does and what happens internally
9	If Marina reactivates the account, the same endpoint reverses the state and restores access. If she decides to delete it, DELETE admin.users.destroy is used and the historical and anonymization rules explained separately are applied.

25. Contact and Informational Pages

TRAMA includes the public About Us, Contact, Privacy, and Terms pages. InstitutionalController delivers these screens through TramaBridge. Contact adds a real workflow that writes to MySQL and sends email, while the other pages present general and legal information about the portal.

Route	Purpose
/quienes-somos	Presents the project and its editorial approach.
/contacto	Displays the form and receives public inquiries. If a session is active, name and email are provided as initial values.
/privacidad	Presents how the portal handles data.
/terminos	Presents the general terms of use.

Contact form validation

Field	Rule	Handling
Reason	account, editorial, advertising, or other.	Must match an option accepted by PublicContactRequest.
Name	2 to 40 characters.	Leading and trailing spaces are trimmed.
Email	Valid format, maximum 50, and the StrictEmail rule.	Can belong to a guest or an authenticated account.
Message	15 to 1,200 characters.	Normalizes line breaks and reduces three or more consecutive breaks to at most one blank line between paragraphs.

Why TRAMA saves first and sends email afterward

InstitutionalController::storeContact() creates contact_messages first. That row is the source of truth for the inquiry. Only afterward does Laravel attempt to send ContactMessageReceivedMail to config('mail.contact_address'), a value supplied by the mail configuration and TRAMA_CONTACT_EMAIL. The internal message uses Reply-To with the sender's name and email address so the team can respond from its email client.

Situation	Result
MySQL stores and SMTP sends	The inquiry is recorded and the team receives the email notification.

Situation	Result
MySQL stores but SMTP fails	The inquiry remains in contact_messages; the email failure is recorded in TramaLog and the user does not need to resubmit it.
Saving to MySQL fails	OperationFailureHandler returns a safe message and the inquiry is not considered received.

Complete example

What happens in Contact.vue before reaching InstitutionalController

Public Contact form.

Contact.vue uses useForm for reason, name, email, and message. When a session exists, Laravel can send defaultName and defaultEmail as props. computed() calculates whether the form is locked, the useful message length, and whether the button can be enabled. After submission, usePage() detects the server flash message. A flash message is temporary data saved for the next navigation, useful for confirmations such as “Inquiry sent successfully.” It then leaves the entire form disabled to prevent an accidental second submission.

```
const page = usePage();
const form = useForm({
  reason: 'account',
  name: props.defaultName,
  email: props.defaultEmail,
  message: '',
});

const submitted = computed(() => Boolean(page.props.flash?.status));
const formLocked = computed(() => form.processing || submitted.value);
```

Candela selects “Problems with my account,” enters her name, email, and inquiry, and clicks Send. PublicContactRequest normalizes and validates the fields. Laravel creates the row in contact_messages. It then attempts to send the internal email with a subject related to the selected reason and Reply-To set to Candela. If SMTP does not respond, the already saved row allows the team to recover the inquiry from the database while the technical error remains available for diagnosis in the logs.

This workflow is also a concrete example of the error handling described in section 29: if the MySQL write fails, `InstitutionalController` uses `OperationFailureHandler` and the inquiry is not considered received; if MySQL has already saved the message but SMTP fails, `TramaLog` records the problem and the inquiry remains available to the team.

Step by step: Candela submits an inquiry from Contact

This example shows how an operation can save data in MySQL and then attempt an external action such as sending email.

Step	What the person does and what happens internally
1	Candela opens <code>Contact</code> , chooses a reason, and fills in her name, email, and message. If she is already signed in, some of that information may be prefilled.
2	<code>Contact.vue</code> maintains the form and displays immediate interface validations where appropriate.
3	On submission, Laravel validates every field again using the server-side rules.
4	<code>InstitutionalController</code> first saves the inquiry in the database so the message is not lost even if email delivery fails afterward.
5	Once the inquiry is persisted, TRAMA attempts to send a notification through the configured mailer.
6	If the email is sent, the screen shows the normal confirmation. If the mail service fails, the system can record the problem without necessarily undoing the inquiry that was already saved.
7	The response returns to Vue and Candela receives a useful message without seeing exceptions, internal paths, or server details.

26. Advertising: Banners, Impressions, Clicks, and Reporting

The Advertising module lets administrators create campaigns and measure how many times they were displayed and how many clicks they received. Each campaign stores a name, brand, placement, image, destination URL, start date, end date, and status.

Visible placement	Internal name	Required size
Top horizontal banner	header	1456 × 180 px
Upper article sidebar	article_sidebar_top	600 × 500 px
Lower article sidebar	article_sidebar_bottom	600 × 1200 px

Those dimensions correspond to the file TRAMA validates and stores. As a presentation reference, the CSS uses a 728 × 90 ratio for the header banner, 300 × 250 for the upper sidebar, and 300 × 600 for the lower sidebar; the width can adapt responsively to the screen. The stored file uses twice those dimensions to preserve sharpness.

Advertising images must have exactly the size required by their placement. They go through the same general image-safety check and are converted to WebP.

Where each advertisement appears on the portal

placement is not merely a name stored in the database: it determines which portal component can display the campaign. The header belongs to the public layout shared by news pages. The two sidebar placements are used within an article detail page, in an aside column separate from the main body.

```

PublicNewsLayout.vue
└─ <AdvertisementBanner
    :advertisement="visibleHeaderAdvertisement"
    variant="header"
  />

Public/ArticleShow.vue
└─ <aside class="article-ad-rail">
    <AdvertisementBanner
      :advertisement="sidebarTopAdvertisement"
      variant="sidebarTop"
    />
    <AdvertisementBanner
      :advertisement="sidebarBottomAdvertisement"
      variant="sidebarBottom"
    />
  </aside>

```

`PublicNewsLayout.vue` receives the header collection and rotates the visible campaign every 6,500 ms when more than one exists. `ArticleShow.vue` receives `sidebar_top` and `sidebar_bottom` for the loaded article. In both cases, the shared `AdvertisementBanner.vue` component applies the appropriate visual variant and handles impressions and clicks.

What an impression means

An impression is not counted merely because Laravel sent the banner data to the browser. The image must load successfully, and at least 50% of the banner must enter the visible portion of the page.

To detect that visibility, Vue uses a function available in modern browsers called `IntersectionObserver`. If the browser does not provide that function, TRAMA uses a fallback. If the image is broken, the impression is not recorded.

What a click means

When a visitor selects a banner, the browser begins navigating to the destination site. At the same time, TRAMA attempts to record the click. JavaScript uses `keepalive` to give that request a chance to finish even after navigation has begun.

Step by step: Candela clicks a banner

Advertising clicks are designed so measurement never becomes a barrier between Candela and the advertiser. The link keeps the real `target_url` as its `href`, and tracking is sent in parallel.

Step	What the person does and what happens internally
1	Candela sees an <code>AdvertisementBanner</code> whose image has already loaded. The component's link points directly to <code>advertisement.target_url</code> .
2	Candela clicks it. The <code>@click</code> event runs <code>registerClick()</code> , but the <code><a></code> keeps its normal navigation behavior and opens the destination in a new tab.
3	<code>registerClick()</code> takes <code>advertisement.click_url</code> and calls <code>window.fetch()</code> with <code>method=GET</code> , <code>cache=no-store</code> , and <code>keepalive=true</code> .
4	The <code>/ads/{advertisement}/click</code> route reaches <code>AdvertisementClickController</code> . It does not redirect to the advertiser because the browser is already handling that navigation.
5	The Controller uses <code>TramaClock::today()</code> , finds or creates the <code>advertisement_daily_metrics</code> row for that advertisement + editorial date, and increments <code>clicks_count</code> by one.
6	Laravel responds with 204 No Content. Even if tracking fails, the JavaScript catch does not replace or cancel the opening of <code>target_url</code> .

Step	What the person does and what happens internally
7	When Marina returns to the administrative report, that daily row contributes to the totals, CTR, and click series in the chart.

CTR

CTR means Click-Through Rate and represents the percentage of impressions that resulted in a click.

```

Impressions: 1,000
Clicks: 10

CTR = 10 / 1,000 × 100
CTR = 1 %

```

`advertisement_daily_metrics` stores impressions and clicks grouped by advertisement and date. The panel uses that data to show totals, CTR, active campaigns, and a daily series with Chart.js.

How the report is built and which chart TRAMA uses

The report does not draw points from the rows currently visible in the grid. `AdvertisementController` first applies the same filters and date range, obtains the IDs of the included campaigns, and sums `advertisement_daily_metrics` directly in MySQL. For the time series, it groups by date and calculates each day's impressions and clicks.

```

$dailyMetrics = AdvertisementDailyMetric::query()
    ->whereIn('advertisement_id', $advertisementIds)
    ->selectRaw('date')
    ->selectRaw('SUM(impressions_count) as impressions_total')
    ->selectRaw('SUM(clicks_count) as clicks_total')
    ->groupBy('date')
    ->orderBy('date')
    ->get();

$chart = [
    'labels' => $dailyMetrics
        ->map(fn ($metric) => $metric->date->format('d/m')),
    'impressions' => $dailyMetrics->pluck('impressions_total'),
    'clicks' => $dailyMetrics->pluck('clicks_total'),
];

```

That array travels as the chart prop to `Admin/Advertisements/Index.vue`. TRAMA does not use a Laravel plugin to draw it: it uses the JavaScript library Chart.js as the engine and `vue-chartjs` as the Vue integration. The imported component is `Line`, so the report is a line chart.

```

import {
    Chart as ChartJS,
    CategoryScale,
    LinearScale,
    LineElement,
    PointElement,
    Tooltip,
    Legend,
} from 'chart.js';
import { Line } from 'vue-chartjs';

ChartJS.register(
    CategoryScale,
    LinearScale,
    LineElement,
    PointElement,
    Tooltip,
    Legend,
);

```

`buildChartData()` creates two datasets over the same dates. Impressions uses the `yImpressions` axis and Clicks uses `yClicks`. The scales are independent because impressions are usually much larger than clicks; with a single scale, the smaller series could appear visually flattened near zero.

```
function buildChartData(chart) {
  return {
    labels: chart?.labels || [],
    datasets: [
      {
        label: 'Impresiones',
        data: chart?.impressions || [],
        yAxisID: 'yImpressions',
      },
      {
        label: 'Clicks',
        data: chart?.clicks || [],
        yAxisID: 'yClicks',
      },
    ],
  };
}

<Line :data="chartData" :options="chartOptions" />
```

`chartOptions` places `yImpressions` on the left and `yClicks` on the right. The horizontal axis represents the days returned in labels. The same panel also displays total impressions, total clicks, CTR, and the number of active campaigns, all calculated for the selected period and filters.

Step by step: Marina opens Advertising and reaches the chart

Step	What the person does and what happens internally
1	Marina opens Advertising and chooses, for example, Last 30 days. <code>AdvertisementIndexRequest</code> validates the URL filters.
2	<code>AdvertisementController</code> builds <code>filteredAdvertisementQuery()</code> and obtains the IDs of campaigns that match the search, placement, brand, status, and other filters.
3	Using <code>advertisement_daily_metrics</code> , it sums impressions and clicks for the period for the summary cards and calculates $CTR = \text{clicks} / \text{impressions} \times 100$.
4	For the chart, it queries the filtered metrics again, groups by date, and sums <code>impressions_count</code> and <code>clicks_count</code> across all included campaigns.
5	Laravel prepares <code>chart.labels</code> , <code>chart.impressions</code> , and <code>chart.clicks</code> and sends them to <code>Admin/Advertisements/Index.vue</code> through <code>Inertia</code> .
6	<code>buildChartData()</code> transforms those three arrays into two <code>Chart.js</code> datasets and associates them with <code>yImpressions</code> and <code>yClicks</code> .
7	<code>vue-chartjs</code> renders <code><Line></code> . Marina sees the daily trend and can compare impression and click patterns without confusing their scales.

How TRAMA decides which advertisement to display

`AdvertisementSelectorService` first filters by placement, `is_active`, and exact validity against `TramaClock`: `starts_at` must be less than or equal to the editorial time and `ends_at` greater than or equal to it. It then checks that the `WebP` recorded in `image_path` actually exists inside the configured folder. A campaign that is active in MySQL but whose file was manually deleted stops appearing on the portal until it has a valid image again.

The header behaves differently from the sidebars. For header, Laravel shares all currently valid campaigns and PublicNewsLayout.vue keeps the current one and rotates among them every 6,500 ms. For article_sidebar_top and article_sidebar_bottom, one valid campaign is selected at random while the article is loaded. Selecting or sending a campaign to Vue does not yet count as an impression.

End-to-end campaign example

Step by step: Marina edits an advertising campaign

Actual advertisement editing dialog.

Admin/Advertisements/Index.vue maintains createForm and editForm separately. This lets each modal preserve its own values, errors, and processing state. The interface can anticipate URL or dimension errors, but Laravel validates the campaign and image again according to the selected placement.

Visual example of an invalid URL before completing the edit.

Step	What happens
1	Marina creates a header banner with a valid image, destination URL, starts_at=19/07/2026 15:00, ends_at=19/07/2026 20:00, and is_active=true.
2	At 15:30 on TramaClock, AdvertisementSelectorService considers it active if the physical file still exists.

Step	What happens
3	Laravel sends it to the layout. If more than one header campaign is active, Vue rotates the banner every 6.5 seconds.
4	The image loads and at least 50% enters the browser's visible area (viewport). AdvertisementBanner.vue calls the impression route once for that appearance.
5	The visitor clicks. The browser opens target_url directly and, in parallel, keepalive attempts to record the click without inserting a TRAMA redirect.
6	AdvertisementDailyMetric finds or creates the row for that advertisement and editorial day and increments impressions_count or clicks_count according to the action.
7	At 20:01 the campaign no longer satisfies ends_at >= TramaClock::now() and stops being selected.

IntersectionObserver prevents banners that were sent to the browser but never became visible from being counted. If the browser does not support that API, TRAMA uses a fallback and records the impression after the image loads. A tracking error must never prevent the visitor from reading the article or opening the advertiser's site.

Advertiser demo pages

Banners in the test dataset can point to local routes such as /demo/anunciantes/{advertiser}. AdvertiserDemoController recognizes only the fictional brands defined by the project and returns 404 for an unknown slug. These pages are rendered with a separate Blade view: they do not pass through Inertia, Trama.vue, or TRAMA's public layout because they represent the fictional external site the visitor reaches after clicking.

Example: a Banco NEXO banner can use /demo/anunciantes/banco-nexo as its destination. The click is recorded through the advertising mechanism already described, and the browser opens the advertiser's fictional landing page directly. There Laravel displays the brand, message, benefits, and call to action prepared for that advertiser, without first rebuilding a screen from the news portal.

Real source code: when an impression is recorded

resources/js/Components/News/AdvertisementBanner.vue first checks that the image loaded and that the banner is not broken. It then uses IntersectionObserver with a 50% threshold:

```
impressionObserver = new IntersectionObserver(
  (entries) => {
    const isVisible = entries.some(
      (entry) => entry.isIntersecting && entry.intersectionRatio >= 0.5
    );

    if (isVisible) {
      registerImpression();
    }
  },
  {
    threshold: [0.5],
  }
);

impressionObserver.observe(element);
```

When registerImpression() runs for the first time, it marks the impression locally to avoid repeating it and sends the measurement request. If the browser does not provide IntersectionObserver, TRAMA falls back to recording after confirming that the image loaded correctly.

```

void window.fetch(impressionUrl, {
  method: 'GET',
  credentials: 'same-origin',
  cache: 'no-store',
  keepalive: true,
  headers: {
    Accept: 'application/json',
    'X-Requested-With': 'XMLHttpRequest',
  },
}).catch(() => {
  // La medición nunca debe afectar la lectura ni ocultar el banner.
});

```

keepalive allows the browser to attempt to complete the record even if the page changes state. The same principle is used when recording a click: navigation to the advertiser does not depend on the metrics endpoint finishing successfully.

Real source code: from the click to the daily counter

AdvertisementBanner.vue keeps the real destination in href and records the metric in the link event. The internal request uses click_url; the public Controller increments only clicks_count and returns an empty response.

```

<a
  :href="advertisement.target_url"
  target="_blank"
  @click="registerClick"
  @auxclick.middle="registerClick"
>

void window.fetch(clickUrl, {
  method: 'GET',
  credentials: 'same-origin',
  cache: 'no-store',
  keepalive: true,
});

// AdvertisementClickController
$dailyMetric = AdvertisementDailyMetric::query()->firstOrCreate(
  [
    'advertisement_id' => $advertisement->id,
    'date' => TramaClock::today()->toDateString(),
  ],
  [
    'impressions_count' => 0,
    'clicks_count' => 0,
  ],
);

$dailyMetric->increment('clicks_count');
return response()->noContent();

```

The impression uses the same daily model but increments impressions_count. This means one row can summarize, for example, 1,000 impressions and 10 clicks for a campaign during one day without storing one row per visitor.

27. Views, Profiles, and Search

Views and article ranking

When a public article is opened, ArticleController increments articles.views and also creates a row in article_views. If Candela opens an article and then reloads it, both page loads can add views. The metric therefore does not represent unique people.

articles.views quickly stores the total needed by the homepage and listings. article_views preserves the detail for each recorded load.

Step by step: Candela opens an article and TRAMA records the view

The view metric is easiest to understand by following a real ArticleShow load from the click through to MySQL.

Step	What the person does and what happens internally
1	Candela selects a card and the browser requests /noticias/{slug}.
2	ArticleController asks ArticleQueryService for the corresponding published article. If the slug does not belong to publicly visible content, Laravel returns 404.
3	Inside a transaction, ArticleController increments articles.views to maintain a fast counter on the article itself.
4	In the same transaction, it creates a row in article_views with article_id, the TramaClock date, and SHA-256 hashes of the IP address and user-agent, without storing those values in plain text.
5	The controller reloads the article with author, category, and tags and prepares comments, related articles, and advertising slots.
6	TramaBridge sends the props to Public/ArticleShow.vue and Candela sees the page with the counter already updated.
7	If Candela reloads the page, the process repeats and can add another view. That is why views counts recorded loads, not unique people.

Real source code: ArticleController records a view

The process above corresponds directly to app/Http/Controllers/Public/ArticleController.php. findPublishedBySlug() obtains a public article; a single transaction then keeps the fast counter and detailed record synchronized:

```
$article = $articles->findPublishedBySlug($slug);

DB::transaction(function () use ($article, $request): void {
    $article->increment('views');

    ArticleView::query()->create([
        'article_id' => $article->id,
        'ip_hash' => hash('sha256', (string) $request->ip()),
        'user_agent_hash' => hash('sha256', (string) $request->userAgent()),
        'viewed_on' => TramaClock::today()->toDateString(),
    ]);
});

$article = $article->fresh(['author', 'category', 'tags']);
```

increment("views") increments the articles counter. ArticleView::create() adds the historical row for the day with the SHA-256 fingerprints. fresh() rereads the article so Vue receives the already updated counter. If either write inside the transaction fails, one half should not remain applied without the other.

Journalist profiles

Each journalist has a public page with their visible details and published articles. The articles.author_id → users.id relationship preserves who wrote each article. If the journalist's physical photo no longer exists, User can return the default avatar so the interface does not display a broken image.

What a journalist's public profile shows

JournalistController::show() accepts only internal accounts whose role belongs to User::EDITORIAL_ROLES and works with their publicly visible publications. The profile shows name, position, biography, avatar, total number of published articles, and date of the most recent publication. It also calculates up to four main sections, ordered by article count,

and allows those sections to be used as filters within the same profile. The employee's email address is not sent to the public screen.

The journalist's article archive is ordered by `published_at` and displays 9 articles per page. If the reader selects one of the offered sections, the URL stays on the profile and adds the section filter; Laravel applies that filter only when it corresponds to one of the sections available for that journalist.

Step by step: Candela filters Valentina's public archive

Valentina has publications in several sections, and Candela wants to see only those from Economy. The following process shows the visible action and what happens internally until Vue receives the first filtered page.

Step	What the person does and what happens internally
1	Candela opens Valentina's public profile. <code>JournalistController</code> accepts only internal accounts whose role belongs to <code>User::EDITORIAL_ROLES</code> .
2	Laravel counts the profile's public articles, calculates the most recent publication date, and obtains up to four main sections ordered by article count.
3	<code>JournalistShow.vue</code> receives those values and displays the section chips. Each chip contains a URL built by Laravel for the same profile with the section parameter.
4	Candela clicks Economy. Inertia requests the profile with <code>?seccion=economia</code> while preserving state and scroll position.
5	<code>JournalistController</code> accepts the filter only if that slug actually belongs to one of Valentina's available sections, and then limits the query by category.
6	The articles are ordered by <code>published_at</code> , paginated 9 per page, and <code>withQueryString()</code> preserves the filter in subsequent links.
7	The response returns to <code>JournalistShow.vue</code> , which marks Economy as active and replaces the grid without exposing the profile's email or other private information.

Real source code: how `JournalistController` builds the profile

The profile is not built in Vue from all of the data in users either. `JournalistController` explicitly selects which information is public and prepares a paginated query of visible articles.

```

abort_unless(
    in_array($user->role, User::EDITORIAL_ROLES, true),
    404,
);

$publishedArticlesQuery = $this->publishedArticlesQuery($user);
$publishedArticlesCount = (clone $publishedArticlesQuery)->count();
$latestPublishedAt = (clone $publishedArticlesQuery)->max('published_at');
$sections = $this->sectionsWrittenByJournalist($publishedArticlesQuery);

$articles = $filteredArticlesQuery
    ->with(['author', 'category', 'tags'])
    ->latest('published_at')
    ->paginate(9)
    ->withQueryString();

```

In addition to the paginated query shown, the public response that TramaBridge delivers to Vue includes a journalist prop with id, name, slug, job_title, bio, avatar, counts, sections, and public URL; it does not include email. The section filter is accepted only when its slug belongs to the sections calculated for that journalist.

Public search

If a visitor searches for “football,” TRAMA compares that text with the title, subtitle, excerpt, category, and tags of published articles. Results can be limited to a category. It does not search the full body so that an isolated word in a long article does not generate unhelpful results.

Step by step: Candela searches for “football” on the public portal

Public search prioritizes fields that help find an article by topic without scanning the entire body.

Step	What the person does and what happens internally
1	Candela types football in Public/Search.vue. v-model keeps form.q synchronized, and computed canSearch enables the button when there is at least one real character.
2	On submission, search() trims spaces and uses router.get(route("search")) with q and the selected category.
3	SearchController receives the query parameters, normalizes the term, and calls ArticleQueryService::searchPublishedArticles().
4	The service starts from Article::publiclyVisible() and searches with LIKE in title, subtitle, and excerpt, as well as category.name and tags.name.
5	If Candela selected a category, Laravel adds that filter by slug. The full body is not included in this public search.
6	Results are ordered by published_at, paginated 9 per page, and keep q and category in subsequent links.
7	TramaBridge returns filters, categories, and results. Search.vue preserves the entered text, and PaginatedArticleGrid displays the matching articles.

Real source code: from Search.vue to the public query

Public/Search.vue creates a GET navigation with q and category. routes/web.php resolves GET /buscar to SearchController; the Controller normalizes q and delegates the actual search to ArticleQueryService::searchPublishedArticles().

```
// Search.vue
router.get(route('search'), {
  q: form.q.trim(),
  category: form.category || undefined,
});

// SearchController
$term = trim((string) $request->query('q'));
$category = $request->query('category');
$results = $articles->searchPublishedArticles($term, $category);
```

Inside the Service, the query always starts from publiclyVisible(). applySearchTerm() escapes the % and _ wildcards and builds the LIKE only over title, subtitle, excerpt, category, and tags. The body is deliberately excluded from this public search.

```

$query = Article::query()->publiclyVisible();

$like = '%'.str_replace(
    ['%', '_'],
    ['\%', '\_'],
    $term,
).'%';

$query->where(function (Builder $inner) use ($like): void {
    $inner
        ->where('title', 'like', $like)
        ->orWhere('subtitle', 'like', $like)
        ->orWhere('excerpt', 'like', $like)
        ->orWhereHas('category', fn (Builder $category) =>
            $category->where('name', 'like', $like)
        )
        ->orWhereHas('tags', fn (Builder $tag) =>
            $tag->where('name', 'like', $like)
        );
});

```

Finally, `searchPublishedArticles()` applies the optional category filter, loads author/category/tags, orders by `published_at`, and paginates 9 per page. The second fragment shows those operations and `withQueryString()`, which preserves `q` and `category` when moving between pages:

```

return $query
    ->when($categorySlug, function ($query) use ($categorySlug): void {
        $query->whereHas(
            'category',
            fn ($category) => $category
                ->publiclyVisible()
                ->where('slug', $categorySlug)
        );
    })
    ->with(['author', 'category', 'tags'])
    ->latest('published_at')
    ->paginate(9)
    ->withQueryString();

```

Search inside the panel

The editorial panel needs a broader search. For that reason, each article has an internal field called `search_text` that combines the title, subtitle, excerpt, body without HTML, category, and tags into text prepared for searching.

MySQL creates an index called `FULLTEXT` on that field. An index is a structure that allows data to be found more quickly without scanning every full row in the same way. `FULLTEXT` is designed specifically to locate words within text.

```

title + subtitle + excerpt + body + category + tags
      ↓
articles.search_text
      ↓
FULLTEXT index

```

Step by step: Mateo searches for an article from the CMS grid

Internal search is broader than public search because it needs to locate editorial work even by words that appear in the article body.

Step	What the person does and what happens internally
1	Mateo types logistics in the search box in Admin/Articles/Index.vue. The component normalizes the text and keeps the other active filters.
2	applySearch() calls visitList(), which uses router.get() to admin.articles.index with preserveState, preserveScroll, and replace.
3	Laravel validates and cleans the index filters before building the query for articles accessible to the user's role.
4	ArticleIndexQueryService extracts valid terms of three or more characters and, in MySQL, turns each one into a Boolean FULLTEXT prefix search, for example +logistica*.
5	whereFullText() queries articles.search_text. That field already contains title, subtitle, excerpt, body as plain text, category, and tags.
6	The query combines the search with status, author, category, date, or other filters Mateo may already have applied.
7	The response returns to the same grid. Vue replaces the rows and restores the scroll position the component saved before navigation.

How search_text is maintained

ArticleSearchIndexer builds search_text by combining title, subtitle, excerpt, body converted to plain text, category name, and tag names. HtmlSanitizer removes HTML and normalizes the content before concatenating it. The update is made directly to search_text so updated_at does not change and an editorial revision is not created merely because an auxiliary search value was rebuilt.

```

title + subtitle + excerpt + body without HTML + category + tags
↓
ArticleSearchIndexer::build()
↓
articles.search_text
↓
MySQL FULLTEXT index

```

When an article is saved, updated, or restored, the editorial workflow prepares this text again. If a migration, import, or maintenance task requires rebuilding it for all articles, php artisan articles:rebuild-search-index can be run. By default, the command processes batches of 500 and accepts --chunk to change that size without loading all articles into memory at once.

Search	Fields considered	Behavior
Public (/buscar)	Title, subtitle, excerpt, category, and tags.	Only publicly visible articles; can filter by category and displays 9 results per page. Does not search the full body.
Editorial (CMS)	Title, subtitle, excerpt, body without HTML, category, and tags combined in search_text.	Uses FULLTEXT for a broader and faster internal search.

Step by step: Valentina changes a tag and search_text updates without pretending there was a new edit

search_text is auxiliary search data; rebuilding it should not modify updated_at or fill the history with artificial revisions.

Step	What the person does and what happens internally
1	Valentina saves editorial changes that modify the text, category, or tags of an article.
2	After synchronizing the final relationships, ArticleSearchIndexer loads category and tags so it can work with the current names.
3	build() combines title, subtitle, excerpt, body, category.name, and each tag.name.
4	HtmlSanitizer::toPlainText() removes HTML, normalizes the text, and discards empty fragments before concatenating them.
5	index() updates only the articles.search_text column through DB::table(...)->update().
6	Because it does not save the entire model again, this auxiliary rebuild does not change updated_at or trigger an editorial revision by itself.
7	The next CMS FULLTEXT search immediately uses the rebuilt text with the current names and content.

Example: the same word in two search tools

If “lithium” appears only in the sixth paragraph of an article body, public search may not return it because it prioritizes visible identification fields. In the editorial panel, “lithium” is part of search_text and can locate that article. This difference is deliberate: readers receive more relevant results, while the internal team has a broader document search.

Related articles

When an article is opened, ArticleQueryService::related() looks for other visible publications that share the same category or at least one tag, excludes the current article, orders by publication date, and ArticleController requests three results for the “Keep Reading” block. The relationship is therefore not stored manually in a table: it is calculated from the taxonomy each article already has.

Step by step: Candela finishes an article and TRAMA chooses what to show in “Keep Reading”

Related articles are resolved on the server so the editorial criterion remains the same regardless of how Vue renders the cards.

Step	What the person does and what happens internally
1	ArticleController already has the current article and its tags loaded.
2	ArticleQueryService::related() starts from publicly visible articles and excludes the article Candela is reading.
3	The query accepts candidates from the same category or those sharing any of the current article's tags.
4	Candidates are ordered by published_at to prioritize recent content within that related set.
5	ArticleController requests three results to complete the Keep Reading row and converts them through ArticleCardResource.

Step	What the person does and what happens internally
6	TramaBridge sends related together with the main article, and ArticleShow.vue reuses the public cards to display them at the end.

What a view record stores

ArticleController increments `articles.views` and creates a row in `article_views` inside the same transaction. The row stores `article_id`, `viewed_on`, and SHA-256 hashes of the IP address and User-Agent instead of keeping those values in plain text. The hash provides a technical reference without turning `article_views` into a direct log of IP addresses or full browser strings. Even so, each valid reload can add another view: the metric does not represent unique users.

The User-Agent is a technical string the browser sends to identify itself to the server. A hash is a fingerprint derived from a value: it allows a reference to be compared or recognized without directly storing the original value. TRAMA keeps those fingerprints in `article_views` instead of storing the full IP address and User-Agent in plain text.

Real source code: CMS FULLTEXT search

ArticleIndexQueryService turns each term into a required prefix search and uses `whereFullText()` when the connection is MySQL. For example, `ciudad` becomes `+ciudad*` so words beginning that way are accepted:

```
$booleanSearch = implode(
    ',',
    array_map(
        static fn (string $term): string => '+' . $term . '*',
        $searchTerms,
    ),
);

$query->whereFullText(
    'articles.search_text',
    $booleanSearch,
    [
        'mode' => 'boolean',
    ],
);
```

The `search_text` column has a FULLTEXT index created by a migration when the driver is MySQL. During PHPUnit, in-memory SQLite is used and does not support this search; ArticleIndexQueryService therefore applies a LIKE fallback there so the general behavior can be tested without depending on a MySQL-only feature.

28. Queues, Scheduled Tasks, Transactions, and Concurrency

TRAMA uses queues, workers, scheduled tasks, transactions, and locks because not every job should run at the same time or under the same guarantees. This section explains these mechanisms from the beginning and then connects them to two real cases: automatic comment moderation and scheduled article publishing.

Why TRAMA needs work outside the HTTP request

A web request has a person waiting on the other side. When Candela submits a comment, Laravel needs to validate and save what is essential, but the browser should not be kept waiting while Node.js loads detectors and models. In other cases, nobody is clicking anything at all: a scheduled article must be published when its time arrives. Queue and Scheduler solve different execution problems.

Job, queue, and worker: three different pieces

Component	What it represents	Real example in TRAMA
Job	An object describing work that Laravel can execute later.	ModerateComment contains the comment ID and moderation_revision and knows how to run automatic moderation.
Queue	The logical place where a Job waits until it can be taken for execution.	ModerateComment uses the moderation queue.
Worker	A Laravel process that stays running, looks for available Jobs, and calls their handle() method.	<pre>php artisan queue:work -- queue=moderation,default --tries=3 -- timeout=70</pre>

They are not synonyms. ModerateComment is the work; moderation is the queue name; queue:work is the process that consumes Jobs. Dispatching a Job does not mean it is already running.

QUEUE_CONNECTION=database: where the Job waits

In the documented environment, .env defines QUEUE_CONNECTION=database. config/queue.php turns that option into the database queue connection, whose default table is jobs. The Laravel migration creates columns there such as queue, payload, attempts, reserved_at, and available_at.

```
Schema::create('jobs', function (Blueprint $table) {
    $table->id();
    $table->string('queue')->index();
    $table->longText('payload');
    $table->unsignedTinyInteger('attempts');
    $table->unsignedInteger('reserved_at')->nullable();
    $table->unsignedInteger('available_at');
    $table->unsignedInteger('created_at');
});
```

The jobs row does not replace the comment. comments stores the editorial data with status=processing; jobs stores the serialized description of the work that still needs to run. When a worker reserves the Job, it updates its attempt tracking and executes it. If it finishes successfully, the pending work leaves that queue; if it fails, Laravel can retry it according to its configuration.

Real source code: how ModerateComment is defined

app/Jobs/ModerateComment.php declares both the retry policy and the queue it uses. afterCommit() is important: if the Job is dispatched inside a transaction, Laravel must not make it available before the comment row has been committed in MySQL.

```

public int $tries = 3;
public int $timeout = 70;
public bool $failOnTimeout = true;

public function __construct(
    public readonly int $commentId,
    public readonly int $moderationRevision,
) {
    $this->onQueue('moderation');
    $this->afterCommit();
}

public function backoff(): array
{
    return [5, 15, 30];
}

public function handle(): void
{
    $this->processAutomaticModeration();
}

```

tries=3 means one initial attempt plus up to two retries. timeout=70 limits the total duration of one Job execution; the Node process has a shorter timeout of its own. backoff() sets waits of 5, 15, and 30 seconds between failed attempts. CommentController does not call handle() manually: Laravel invokes it when the worker takes the Job.

How ModerateComment reaches the queue

After saving a reader comment as processing, CommentController calls a helper method that attempts to dispatch the exact revision that must be analyzed. If the infrastructure cannot even accept the Job, the catch does not leave the row stuck in processing forever: it moves it to pending with a technical reason.

```

try {
    ModerateComment::dispatch(
        $comment->id,
        (int) $comment->moderation_revision,
    );
} catch (Throwable $exception) {
    report($exception);

    Comment::query()
        ->whereKey($comment->id)
        ->where('status', 'processing')
        ->where('moderation_revision', (int) $comment->moderation_revision)
        ->update([
            'status' => 'pending',
            'moderation_source' => 'automatic',
            'moderation_reason' => 'automation_enqueue_error',
        ]);
}

```

How to start the worker

```
php artisan queue:work --queue=moderation,default --tries=3 --timeout=70
```

This command must remain active during development if asynchronous work is expected to run. --queue=moderation,default specifies which queues to consume; --tries and --timeout set worker limits, while the Job also declares its own properties. The worker queries the database connection, takes an available Job, and Laravel runs ModerateComment::handle().

If the worker is off, the already saved comment does not disappear: it remains processing and the Job stays pending until a worker exists again. If ModerateComment exhausts its attempts, failed() records the technical error and, if that

revision is still current, changes the comment to pending with `moderation_reason=automation_error`. The rule is deliberate: a technical failure is never interpreted as approval.

Step by step: Candela submits a comment and the worker processes it

Step	What the person does and what happens internally
1	Candela clicks Submit comment. <code>CommentController</code> validates the comment and creates a row in comments with <code>status=processing</code> and <code>moderation_revision=1</code> .
2	Within the save workflow, <code>ModerateComment</code> is dispatched with the ID and revision. <code>afterCommit()</code> prevents the Job from becoming available before the transaction is committed.
3	With <code>QUEUE_CONNECTION=database</code> , Laravel serializes the work in the jobs table and assigns it to <code>queue=moderation</code> . The web request can finish without waiting for Node.js.
4	The <code>queue:work</code> process detects the Job, reserves it, and Laravel instantiates the class. Neither Vue nor <code>CommentController</code> calls <code>handle()</code> .
5	<code>handle()</code> delegates to <code>processAutomaticModeration()</code> , which looks up comments again and checks that status and <code>moderation_revision</code> still match the Job.
6	<code>ModerateComment</code> starts node scripts/comments/moderate-comment.mjs through <code>Process::run()</code> . Node coordinates language, links, threats, toxicity, and spam and returns JSON through stdout.
7	Laravel validates the result and updates the same comments row to approved, pending, or rejected. If the revision changed while the Job was running, the old result is not applied.
8	When the Job finishes successfully, it leaves the queue. <code>ArticleShow.vue</code> can detect the new status through the polling documented in section 22.

What Laravel Scheduler is and why a stored date is not enough

Scheduler solves a different problem. Saving `scheduled_at=16:40` only stores a date and time in MySQL; that column does not execute code when the clock reaches 16:40. The project needs a process that periodically checks which tasks are due. Laravel Scheduler manages that schedule and runs tasks when appropriate.

`routes/console.php` registers TRAMA's task. This declaration defines which command should run and how often, but it does not start any process by itself:

```
Schedule::command('articles:publish-scheduled')
->everyMinute()
->withoutOverlapping();
```

`everyMinute()` registers the frequency. `withoutOverlapping()` prevents a second execution of the same command from starting while the previous one is still active. When a Scheduler tick occurs, Laravel evaluates its tasks and runs `articles:publish-scheduled` when appropriate.

schedule:work in development and schedule:run in production

Environment / command	What happens
Development · <code>php artisan schedule:work</code>	Keeps a terminal process active that continuously checks Laravel's schedule. It is convenient for local development.

Environment / command	What happens
Production · php artisan schedule:run	Performs a one-time check of tasks due at that moment and exits. The operating system or platform must invoke it every minute.

TRAMA must not confuse this process with TramaClock. TramaClock defines which editorial time the project considers current; Scheduler decides when to launch tasks and can consult that time through the scheduled publishing command.

Step by step: Mateo schedules an article for 16:40

Step	What the person does and what happens internally
1	Mateo selects 16:40 in the editorial form. ArticleRequest validates the date and ArticleWorkflowService stores status=scheduled and scheduled_at=16:40.
2	The article is stored, but saving scheduled_at does not start a timer inside that row or execute the command automatically.
3	During development, Carlos keeps php artisan schedule:work running. In production, the platform invokes php artisan schedule:run every minute.
4	Laravel reads routes/console.php and finds articles:publish-scheduled with everyMinute(). At 16:39 it may run the command, but the command checks that the time has not yet arrived and does not publish that article.
5	When TramaClock reaches 16:40, a later Scheduler check runs the command again. PublishScheduledArticles searches for status=scheduled rows whose scheduled_at has already passed.
6	The command processes articles in batches and, for each candidate, opens a transaction, rereads the article with lockForUpdate(), and checks status and scheduled_at again against the actual state.
7	If the conditions still hold, it changes status to published, stores published_at, clears scheduled_at, and records published_automatically in the history.
8	The article can enter the portal according to its other public rules. withoutOverlapping() and lockForUpdate() reduce the risk of two executions acting on the same work.

Queue and Scheduler are not the same

Question	Queue / worker	Scheduler
What triggers it?	A Job has already been dispatched and is waiting to run.	A frequency or time has been reached at which a task should be checked.
TRAMA example	ModerateComment after Candela submits a comment.	articles:publish-scheduled every minute.
Where it is defined	PHP Job + queue connection + queue name.	routes/console.php through Schedule::command(...).

Question	Queue / worker	Scheduler
Development process	php artisan queue:work	php artisan schedule:work
If the process is off	Jobs remain pending and do not progress until a worker consumes them.	Periodic tasks are not triggered until the Scheduler is run again.

Queue and Scheduler can be combined in other architectures, but in these two TRAMA workflows they have separate responsibilities. From here, another problem appears: even if work runs at the correct time, several operations may try to modify the same data. Transactions, atomicity, and locks address that problem.

Transactions and atomicity

A transaction groups several MySQL changes as a single operation. If all of them finish successfully they are committed; if one fails, MySQL rolls the group back. This “all-or-nothing” property is called atomicity.

Concurrent operations

Two requests can attempt to modify the same record at almost the same time. This situation is called concurrency. TRAMA uses `lockForUpdate()` when an operation needs to read a state and update it without another request getting ahead during that interval.

Step by step: Valentina saves while `lockForUpdate()` orders the two writes

Valentina finishes a correction just as the autosave timer expires and, almost at the same time, clicks Save changes. Both requests want to write to the same article. The first to arrive opens a transaction and queries the row again with `lockForUpdate()`. MySQL keeps that row locked against the other write until the transaction finishes. The second request waits; when it acquires the lock, it rereads the article in its current state and applies its rules to the already update d data.

Step	What Valentina does and what happens internally
A	Valentina finishes a correction just as the autosave interval expires. Vue can already start the silent request for the article.
B	Almost at the same time, Valentina clicks Save changes. The second request attempts to work on the same row and must respect the lock held by whichever operation arrived first.
C	The first operation opens a transaction, rereads the article with <code>lockForUpdate()</code> , applies its validations, writes, and commits. When it finishes, it releases the lock.
D	The second request then continues, rereads the already updated article, and applies its rules to the actual state. This orders the two writes instead of allowing either one to decide based on a row that changed midway through the operation.

The same principle appears in several parts of the project: `ArticleWorkflowService`, scheduled publishing, category ordering, tag merging, moderation, and account administration. `lockForUpdate()` does not replace a transaction: the lock makes sense because it is held while the transaction protects the full read-check-write sequence.

Real source code: transaction and lock in an everyday operation

`ArticleWorkflowService::autosave()` provides a small example of the pattern TRAMA repeats in sensitive operations. It first opens a transaction, then rereads the article with `lockForUpdate()`, and only then applies the update:

```

return DB::transaction(function () use ($article, $attributes, $user): Article {
    $locked = Article::query()
        ->whereKey($article->id)
        ->lockForUpdate()
        ->firstOrFail();

    if (! in_array($locked->status, Article::JOURNALIST_EDITABLE_STATUSES, true)) {
        throw ValidationException::withMessages([
            'article' => 'La noticia ya no admite guardado automático.',
        ]);
    }

    $locked->update($attributes);

    return $locked->refresh();
}, attempts: 3);

```

The transaction defines the period during which the lock is meaningful. If another request tries to lock the same article, it waits until this operation finishes. When it acquires the lock, it rereads the actual state and may discover that the article is no longer editable. This is how `lockForUpdate()` prevents a decision from being made on data that changed while the request was in progress.

29. Error Handling, Logging, and Fallback Pages

TRAMA distinguishes between errors expected during normal operation and unexpected technical failures. If a person leaves a required field empty, tries to enter a screen without permission, or requests a URL that does not exist, Laravel already has a specific response for that situation. By contrast, if an unexpected exception appears during an operation, the system needs to preserve enough technical information to investigate it without showing those internal details to the user.

Validation, authorization, and exceptions are not the same

Validation responds to a known rule. For example, an article without a category cannot be submitted for review: `ArticleRequest` detects the problem and returns a message associated with the form. Authorization answers another known question: whether the current account has permission to perform an action. A technical exception is different: it represents a problem that occurred while the code was trying to complete an operation that should normally have succeeded.

PHP represents these technical problems with exception objects. In TRAMA they are usually received in a variable called `$exception`. That object can expose, among other details, the error class, its technical message, the file, and the line where it occurred. This information can be useful for diagnosis, but it must not be sent unchanged to the browser.

What a log is and what logging means

A log is a technical record of what happened while an application was running. It can store the date and time, severity level, a message, and contextual data that help reconstruct a failure. Logging is the process of creating and writing those records. In TRAMA, important unexpected errors are separated from Laravel's general log and end up in dedicated daily files, by default under `storage/logs`.

The purpose of a log is not to display a message to the visitor. Its audience is the person maintaining the project. It may therefore contain technical information that should never appear in the interface, while the user receives only a short, safe message that lets them retry or continue.

Example 1: a captured failure while saving a Contact inquiry

Section 25 explained that `Contact` first saves the inquiry in MySQL. `InstitutionalController::storeContact()`, located in `app/Http/Controllers/Public/InstitutionalController.php`, surrounds that write with try and catch. try defines the code that may fail. If the insertion finishes successfully, catch does not run. If a `Throwable` occurs, PHP interrupts the block and provides the problem object in `$exception`.

```
try {
    $contactMessage = ContactMessage::query()->create($request->validated());
} catch (Throwable $exception) {
    $this->operationFailures->fail(
        $exception,
        'public_contact',
        'store_message',
        'No pudimos enviar tu consulta en este momento. Intentá nuevamente.',
        [
            'email' => $request->validated('email'),
            'reason' => $request->validated('reason'),
        ],
    );
}
```

Throwable is the general type that can capture PHP errors and exceptions. If `ContactMessage::create()` works, execution continues normally. If it fails, `catch` does not try to build the log itself: it passes the exception to a helper class called `OperationFailureHandler`.

What OperationFailureHandler is and why it exists

`OperationFailureHandler` is a helper class located at `app/Support/OperationFailureHandler.php`. Its purpose is to keep each controller from repeating the same procedure whenever an operation fails unexpectedly. The controller gives it five things: the received exception, the system area, the operation name, the safe message the user can see, and, when needed, additional contextual data.

The `fail()` method first checks whether the problem belongs to a situation Laravel already knows how to handle specifically. Validation, authentication, permissions, HTTP errors Laravel already knows how to handle—for example, a 404 for a missing resource—and a complete MySQL outage are not converted into a generic message: the exception is rethrown so its corresponding treatment is preserved. Only when the failure is unexpected does it record the technical detail and then return the safe message specified by the controller to the interface.

```
if (
    $exception instanceof ValidationException
    || $exception instanceof AuthenticationException
    || $exception instanceof AuthorizationException
    || $exception instanceof ModelNotFoundException
    || $exception instanceof HttpExceptionInterface
    || $this->isDatabaseUnavailable($exception)
) {
    throw $exception;
}

TramaLog::error(
    'Falló una operación.',
    [
        ...$context,
        'area' => $area,
        'operation' => $operation,
        'exception_class' => $exception::class,
        'exception_message' => $exception->getMessage(),
        'exception_file' => $exception->getFile(),
        'exception_line' => $exception->getLine(),
    ]
);

throw ValidationException::withMessages([
    'operation' => $userMessage,
]);
```

The two parts of the method have different audiences. `TramaLog::error()` preserves the information needed to diagnose the failure. The final `ValidationException` contains only `$userMessage`; Inertia can display that text in a form, and an Axios request receives it as a 422 response. The browser does not receive `exception_message`, `exception_file`, or `exception_line`.

What TramaLog is and where it is located

TramaLog is another project-specific helper class, located at `app/Support/TramaLog.php`. It acts as the common entry point for recording TRAMA's technical errors. Thanks to this class, controllers and services do not need to know the log filename, how it is written, or the internal configuration Laravel uses: they call TramaLog and the logging infrastructure handles the rest.

The class has three main pieces. `error()` records a message and optional context; `mailFailureMessage()` turns certain long mail-system errors into shorter technical messages; and `formatContext()` removes null values and prepares the context as readable JSON before adding it to the record. The first two methods are public because other parts of the project call them. `formatContext()` is private because it is used only inside the class itself.

```
TramaLog
├── error()
│   records the message and technical context
├── mailFailureMessage()
│   summarizes certain mailer errors
└── formatContext()
    cleans the context and converts it to readable JSON
```

How the code records an error with TramaLog

Recording an error does not mean opening a file manually. The code calls `TramaLog::error()`. That method receives the main message and a context array and uses Laravel's Log facade to select TRAMA's dedicated channel named `trama_unexpected`.

```
public static function error(string $message, array $context = []): void
{
    Log::channel('trama_unexpected')->error(
        $message.self::formatContext($context)
    );
}
```

Log is the facade Laravel provides for working with its logging system. `channel('trama_unexpected')` selects a specific configuration, and `error(...)` records the message at ERROR level. Before sending it, `formatContext()` adds a JSON block below the message when useful data exists.

```
private static function formatContext(array $context): string
{
    $context = array_filter(
        $context,
        static fn (mixed $value): bool => $value !== null
    );

    if ($context === []) {
        return '';
    }

    return "\n".json_encode(
        $context,
        JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES
    );
}
```

If the context is empty, `formatContext()` returns an empty string and the record keeps only the message. If data exists, `JSON_PRETTY_PRINT` places each field on a separate line, while the other options keep characters and paths readable.

What a logging channel is and how TRAMA's channel is configured

A logging channel is a configuration that tells Laravel how to handle a group of records: what destination type to use, where to write, what minimum level to accept, how long to retain files, and which customizations to apply. TRAMA defines its unexpected-error channel in `config/logging.php` under the name `trama_unexpected`.

```
'trama_unexpected' => [
    'driver' => 'daily',
    'path' => $tramaLogPath,
    'level' => 'error',
    'days' => 30,
    'tap' => [App\Logging\TramaLogFormatter::class],
    'replace_placeholders' => true,
],
```

driver=daily means a different file is used each day. path contains the base file; if TRAMA_LOG_PATH is not defined, the project uses storage/logs/trama-backend.log and Laravel adds the date to the final name, for example trama-backend-2026-09-15.log. level=error sets ERROR as the minimum level, so more severe levels are also accepted; days=30 retains up to thirty daily files. replace_placeholders=true allows Laravel to replace placeholders such as {id} or {email} when the message uses them and those values exist in the context.

The tap option is especially important: it connects the channel with App\Logging\TramaLogFormatter. No controller calls this class manually. When Laravel prepares the trama_unexpected channel, it runs the class specified in tap so TRAMA can customize the logger before records begin to be written.

Monolog: the logging engine Laravel uses underneath

Laravel provides a convenient interface through Log and the channels defined in config/logging.php, but the internal logging work relies on a PHP library called Monolog. Monolog is not a process that must be started in another terminal, nor a tool that TRAMA controllers call directly. It is part of the infrastructure Laravel uses whenever a message is logged.

The relationship can be read this way: TRAMA code calls Log; Laravel checks the channel configuration; underneath, Monolog manages the components that ultimately send the record to its destination. TRAMA uses Monolog classes directly only in TramaLogFormatter, because that is where it needs to customize how each entry is presented.

Handler and formatter: destination and presentation of a log record

Within Monolog, a handler is the component responsible for sending or writing a record to a destination. That destination could be a file, the console, or an external service. In TRAMA's daily channel, Laravel prepares the handler needed to write to the file for the current day.

A formatter has a different role: it decides how the record is represented before the handler writes it. It does not decide which errors enter the channel or where they are stored. It only controls presentation: date order, level, message, line breaks, and other formatting details.

What TramaLogFormatter is and how it enters the flow

TramaLogFormatter is a TRAMA-specific class located at app/Logging/TramaLogFormatter.php. Its responsibility is to customize how records from the trama_unexpected channel are presented so the file remains concise and readable. The class does not catch exceptions, decide what should be logged, or choose the file path: those decisions have already been made by the code and config/logging.php.

Laravel runs TramaLogFormatter automatically because the channel declared it in tap. To do so, it calls the special __invoke() method. PHP allows an object that defines __invoke() to be executed as if it were a function; Laravel uses that mechanism to pass the newly prepared logger to the class.

```

public function __invoke(LaravelLogger $logger): void
{
    $monolog = $logger->getLogger();

    if (! $monolog instanceof MonologLogger) {
        return;
    }

    $formatter = new LineFormatter(
        "[%datetime%] %level_name%: %message%\n",
        "Y-m-d H:i:s",
        true,
        true
    );

    foreach ($monolog->getHandlers() as $handler) {
        if (! $handler instanceof FormattableHandlerInterface) {
            continue;
        }

        $handler->setFormatter($formatter);
    }
}

```

`$logger->getLogger()` provides access to the internal logger Laravel prepared. The instanceof `MonologLogger` check prevents Monolog-specific methods from being used if the internal logger were ever different. A `LineFormatter` is then created—that is, a formatter that builds each entry as text according to a template.

In that template, `%datetime%` is replaced by the date and time, `%level_name%` by the level—for example `ERROR`—and `%message%` by the message already prepared by `TramaLog`, including the context JSON when present. The `foreach` iterates over Monolog's handlers. Only handlers that implement `FormattableHandlerInterface` can be assigned a formatter through `setFormatter()`; the others are left unchanged.

That is why `TramaLog`, `TramaLogFormatter`, and `Monolog` do not do the same thing: `TramaLog` gives TRAMA one consistent way to record errors; Laravel selects and configures the channel; `Monolog` is the logging engine used by Laravel; the handler writes the record to its destination; and `TramaLogFormatter` customizes how the final entry looks.

Complete flow: from the exception to the file

```

Controller / Service
↓
try
↓
an exception occurs
↓
catch (Throwable $exception)
↓
OperationFailureHandler::fail()
↓
TramaLog::error()
↓
Laravel Log
↓
trama_unexpected channel
↓
config/logging.php
↓
tap runs TramaLogFormatter
↓
TramaLogFormatter assigns LineFormatter
↓
Monolog + daily handler
↓
the handler formats and writes
↓
storage/logs/trama-backend-YYYY-MM-DD.log
↓
OperationFailureHandler generates the safe message
↓
interface

```

The diagram shows the path used when a controller catches a failure that can be converted into a normal response for the interface. TramaLogFormatter helps prepare the logger and changes the presentation the handler will use; it does not receive the controller exception directly.

Example 2: a real record produced during Artisan

```

[2026-09-15 10:03:04] ERROR: Unable to locate file in Vite manifest: resources/js/Pages/Trama.vue. (View: C:\Users\PC\Desktop\project\TRAMA\resources\views\app.blade.php)
{
  "controller": "Console",
  "method": "artisan",
  "status": 500
}

```

Real example of an error recorded during a local TRAMA execution.

The screenshot shows a real project record. The first line preserves the date and time, ERROR level, and technical message: Vite could not locate resources/js/Pages/Trama.vue in the manifest. Below it appears structured context with controller=Console, method=artisan, and status=500.

This second example does not correspond to the Contact try/catch shown at the beginning. The error occurred during an Artisan execution and reached Laravel's global handling. It demonstrates that different parts of the project can use the same logging infrastructure and end up in the same daily channel.

Uncaught errors can also reach TramaLog

OperationFailureHandler is used when a controller or service can catch a failure and return a safe message. However, some exceptions occur outside those try/catch blocks, for example during Artisan or before reaching a controller. bootstrap/app.php also records these unexpected internal errors and adds general context before calling TramaLog.

```

if (app()->runningInConsole()) {
    $controller = 'Console';
    $method = 'artisan';
}

TramaLog::error($logMessage, [
    'controller' => $controller,
    'method' => $method,
    'status' => $isDatabaseUnavailable ? 503 : 500,
    'user_id' => optional($request->user())->id,
    'email' => $request->input('email'),
    'slug' => $request->route('slug') ?? $request->input('slug'),
    'article_id' => $articleId ?? $request->input('article_id'),
]);

```

That code explains why the previous screenshot contains Console, artisan, and 500 even though the error did not pass through `OperationFailureHandler`. The entry point changes, but `TramaLog` selects the same channel again and the record ends up using the same configuration and format.

Email errors: logging does not always mean rolling back

How a failure is handled depends on which part of the operation has already been committed. In `Contact`, MySQL is the source of truth: `contact_messages` is saved first, and only then is the SMTP notification attempted. If email fails, the inquiry is not deleted because it was already received successfully; the project records the notification problem and allows the team to recover the message from the database.

```

try {
    Mail::to((string) config('mail.contact_address'))
        ->send(new ContactMessageReceivedMail($contactMessage));
} catch (Throwable $exception) {
    TramaLog::error(TramaLog::mailFailureMessage($exception), [
        'controller' => self::class,
        'method' => 'storeContact',
        'contact_message_id' => $contactMessage->id,
        'recipient' => config('mail.contact_address'),
        'sender_email' => $contactMessage->email,
    ]);
}

```

Here the controller calls `TramaLog` directly because it does not need to turn the failure into a form error: the inquiry has already been saved. `mailFailureMessage()` reduces certain long mailer messages to shorter technical text, such as SMTP Error: Could not authenticate. or SMTP Error: Could not connect to SMTP host.

The policy is different during public registration: if the verification email fails, `RegisteredUserController` records the problem and deletes the newly created account because registration cannot be completed. The logging infrastructure is the same, but the recovery decision depends on the workflow.

What happens if MySQL stops responding

A complete MySQL outage requires global handling because it can occur even before the session can be read, the user obtained, or a normal `Inertia` page prepared. `OperationFailureHandler` therefore does not turn it into a form message: it rethrows that `QueryException` and `bootstrap/app.php` decides the final response.

```

if ($request->header('X-Inertia')) {
    $location = $request->headers->get('Referer') ? url('/') : null;
    return Inertia::location($location);
}

if ($request->expectsJson()) {
    return response()->json([
        'success' => false,
        'code' => 'DATABASE_UNAVAILABLE',
        'message' => 'La página no está disponible en este momento.',
    ], 503);
}

if (view()->exists('errors.database')) {
    return response()->view('errors.database', [], 503);
}

```

An Inertia navigation forces a full page load to leave Vue's internal flow. A request expecting JSON receives HTTP 503 and a safe structure. A normal navigation receives resources/views/errors/database.blade.php, an independent Blade page designed to render without querying MySQL again.



Page shown when TRAMA cannot access MySQL.

Request type	Response
Normal navigation	HTTP 503 and the errors/database.blade.php page.
Navigation performed by Inertia	Inertia::location() forces a full page load; if MySQL is still unavailable, that new load ends on the fallback page.
Request expecting JSON	HTTP 503 with code=DATABASE_UNAVAILABLE and a public message without MySQL details.

Error.vue and other HTTP errors

When Laravel can still build a normal Inertia response, resources/js/Pages/Error.vue presents errors such as 403, 404, 419, 500, and some 503 responses with understandable text. A 403 indicates missing permission; a 404 means the resource does not exist; a 419 can indicate that the session expired; and a 500 represents an internal error that the interface should not expose in technical detail.

General rule

The user should receive enough information to correct the form, try again, or continue. Details needed to diagnose a failure—exception class, technical message, file, line, controller, operation, and other contextual data—remain on the server side. TRAMA thus separates two responsibilities: the interface communicates a problem safely, while the logging system retains the information needed by whoever maintains the application.

Step by step: Candela encounters an error and Carlos investigates what TRAMA recorded

Not all errors mean the same thing. This walkthrough distinguishes invalid data, a denied action, and an unexpected exception.

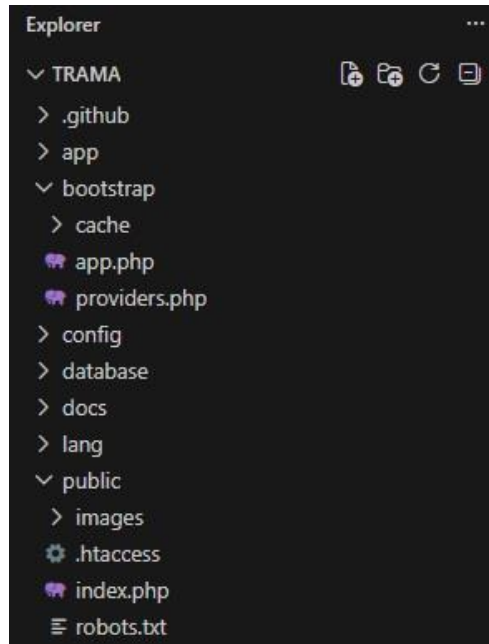
Step	What Candela, Valentina, and Carlos do and what happens internally
1	Candela submits a form with invalid data. A Form Request detects the problem before the operation runs and returns validation errors to the same screen.
2	Valentina attempts an action she is not allowed to perform. Middleware or an authorization check stops the request before data is modified.
3	In another case, a valid operation reaches the controller but an unexpected exception occurs while accessing a dependency or internal process.
4	OperationFailureHandler and TramaLog make it possible to record useful technical context for diagnosing the failure without showing it directly to the user.
5	Monolog sends the record to the configured channel, and TramaLogFormatter determines the final presentation of each line.
6	The interface receives an appropriate response or fallback page, for example a controlled HTTP error or a dedicated screen when the database is unavailable.
7	Carlos reviews storage/logs and the recorded context to investigate the actual cause, while the user sees only the information needed to continue or try again.

30. Local Installation of TRAMA

This section starts from the real scenario of a developer cloning or downloading TRAMA from GitHub onto a new computer. The repository contains the source code and files needed to rebuild the installation, but it does not include installed dependencies, the .env file, or artifacts generated by Vite or SSR. The goal is to end with TRAMA configured, connected to MySQL, loaded with its test data, and with the SSR bundle prepared. Day-to-day startup is explained separately in section 31.

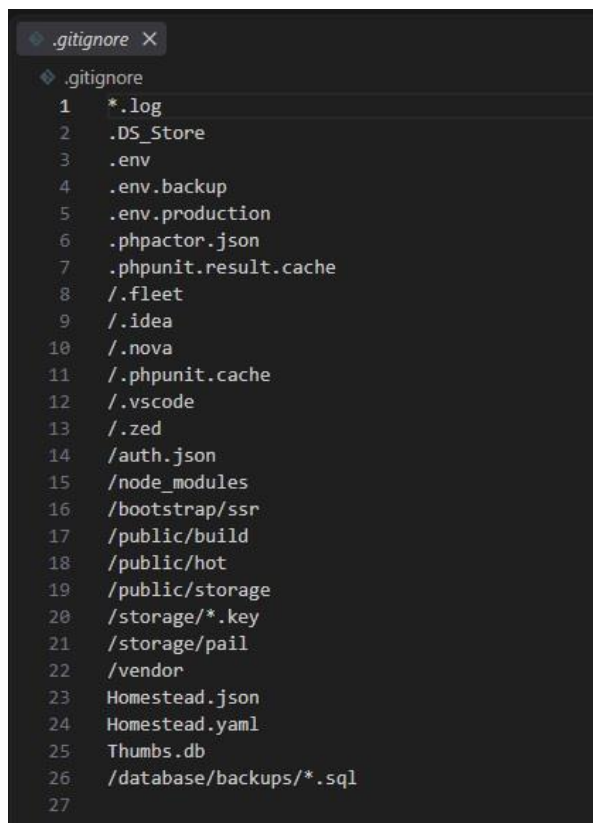
30.1 What comes from GitHub and what is rebuilt

There is no need to create a new Laravel project or a new Vue project. app/, config/, database/, resources/, routes/, scripts/, and the rest of the code are already part of the repository. What gets rebuilt on each computer are the installed dependencies and local configuration.



TRAMA immediately after being downloaded from GitHub, before installing dependencies or generating local files.

Item	Expected in GitHub?	What happens in a new installation
app/, config/, database/, resources/, routes/, scripts/	Yes	They contain the source code and already exist after cloning or downloading TRAMA.
composer.json + composer.lock	Yes	They describe and lock the PHP dependency tree Composer will use to rebuild vendor/.
package.json + package-lock.json	Yes	They describe and lock the JavaScript dependency tree NPM will use to rebuild node_modules/.
vendor/	No	Created locally with composer install.
node_modules/	No	Created locally with npm install.
.env.example	Yes	It is the template for the required variables.
.env	No	Created locally and may contain keys, passwords, and machine-specific paths.
bootstrap/ssr/	No	Contains the generated SSR bundle; created when SSR is built.
public/build/	No	Contains the production client build generated by Vite.



```

1  *.log
2  .DS_Store
3  .env
4  .env.backup
5  .env.production
6  .phpactor.json
7  .phpunit.result.cache
8  /.fleet
9  /.idea
10 /.nova
11 /.phpunit.cache
12 /.vscode
13 /.zed
14 /auth.json
15 /node_modules
16 /bootstrap/ssr
17 /public/build
18 /public/hot
19 /public/storage
20 /storage/*.key
21 /storage/pail
22 /vendor
23 Homestead.json
24 Homestead.yaml
25 Thumbs.db
26 /database/backups/*.sql
27

```

The .gitignore file confirms which dependencies, secrets, and generated artifacts are not versioned.

Do not upload vendor or node_modules to GitHub

Composer and NPM can rebuild those folders from composer.lock and package-lock.json. The .env file must not be versioned either, and the generated builds in bootstrap/ssr and public/build remain outside the repository.

30.2 Required programs and services

Component	Use in TRAMA
Visual Studio Code	Editor used in this guide to open the project and work with integrated terminals.
PHP 8.5.8 recommended	composer.json declares ^8.3, but the current composer.lock contains dependencies that require PHP 8.4.1 or later. The documented environment was verified with PHP 8.5.8.
Composer	Installs Laravel and the PHP dependencies declared in composer.json and composer.lock.
Node.js + NPM	Install Vue, Vite, Sharp, ONNX Runtime, and the other JavaScript dependencies; Node also runs SSR and auxiliary scripts.
MySQL	Database server used by TRAMA.
MySQL Workbench	Graphical client used in this guide to create and inspect the database.
resources/models	Expected location for the models and auxiliary files required by automatic moderation and local image classification.

Before installing dependencies, check which PHP version the terminal is actually using:

```
php -v
composer --version
node --version
npm --version
```

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ php -v
PHP 8.5.8 (cli) (built: Jul 1 2026 04:03:04) (NTS Visual C++ 2022 x64)
Copyright (c) The PHP Group
Built by The PHP Group
Zend Engine v4.5.8, Copyright (c) Zend Technologies
with Zend OPcache v8.5.8, Copyright (c), by Zend Technologies
```

PHP 8.5.8 check in the terminal used to install TRAMA.

Do not run composer update to fix an incorrect PHP version

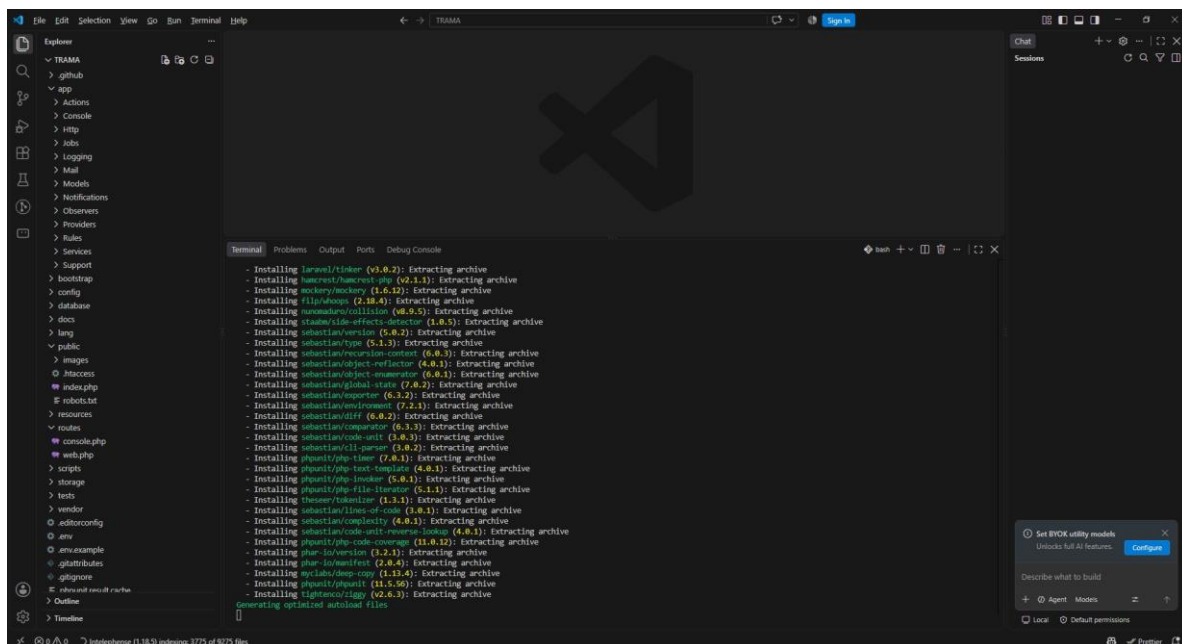
If Composer reports that the platform does not meet requirements, first correct the PHP version or the terminal's PATH. composer update would change versions in the dependency tree and is not the fix for this case.

30.3 Install PHP dependencies

Open a terminal in the TRAMA root—the folder that contains artisan, composer.json, and package.json—and run:

```
composer install
```

Composer reads composer.json and composer.lock, downloads the locked versions, and creates vendor/. That folder contains Laravel and the installed PHP libraries, so it does not need to be stored in GitHub.



Composer installing the project's PHP dependencies; vendor/ is rebuilt locally.

30.4 Install JavaScript and Node.js dependencies

From the same project root, run:

```
npm install
```

NPM reads package.json and package-lock.json and creates node_modules/. It contains Vite, Vue, Sharp, ONNX Runtime, and the other dependencies used by the frontend and Node.js processes. npm install prepares the installation; it is not the command that keeps Vite running during development.

```

PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ npm install
npm warn deprecated boolean@3.2.0: Package no longer supported.

added 247 packages, and audited 248 packages in 56s

68 packages are looking for funding
  run `npm fund` for details

4 vulnerabilities (2 moderate, 2 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.

```

Result of npm install: the Node.js dependencies are installed in the local copy.

npm warnings

NPM may report deprecated packages or known vulnerabilities. That report alone does not mean the installation failed: it must be distinguished from an error that interrupts the command. Any dependency update should be evaluated separately so the project is not changed without control.

30.5 Create .env and generate APP_KEY

.env.example is part of the repository and serves as the template. The .env file is the one actually used by that installation and is not versioned. It can be created with the command appropriate for the terminal:

Terminal	Create .env from .env.example
Git Bash	<code>cp .env.example .env</code>
PowerShell	<code>Copy-Item .env.example .env</code>
CMD	<code>copy .env.example .env</code>

```

.env.example X
.env.example
1 APP_NAME=TRAMA
2 APP_ENV=local
3 APP_KEY=
4 APP_DEBUG=true
5 APP_URL=http://127.0.0.1:8000
6 APP_TIMEZONE=America/Argentina/Buenos_Aires
7
8 # Fecha fija de referencia para que campañas, métricas y reportes no
9 TRAMA_REFERENCE_DATE=2026-07-19
10 # Hora inicial del reloj editorial demo. Solo se usa como punto de pa
11 # después TRAMA continúa desde el último instante guardado entre sesi
12 TRAMA_REFERENCE_TIME=15:30:00

```

.env.example contains the variable names and reference values; APP_KEY remains empty until the local key is generated.

After creating `.env`, generate Laravel's cryptographic key:

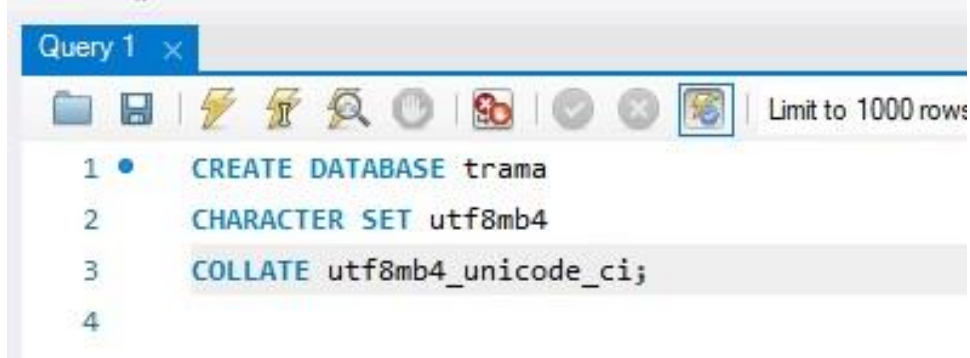
```
php artisan key:generate
```

`key:generate` creates `APP_KEY` and writes it to `.env`. The `.env` file may contain MySQL credentials, email settings, paths, and other private configuration; it must not be uploaded to GitHub or shown with real secrets in public screenshots.

30.6 Create the trama database and configure MySQL

Start the MySQL server TRAMA will use and open MySQL Workbench. In an SQL tab, create an empty database named `trama`:

```
CREATE DATABASE trama
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```



Creating the trama database from MySQL Workbench.

Then fill in `.env` with the actual connection settings. `DB_DATABASE` must point to `trama`; the port, username, and password depend on each computer's MySQL server.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=<mysql-port>
DB_DATABASE=trama
DB_USERNAME=<mysql-username>
DB_PASSWORD=<mysql-password>
```

Do not copy credentials from another computer

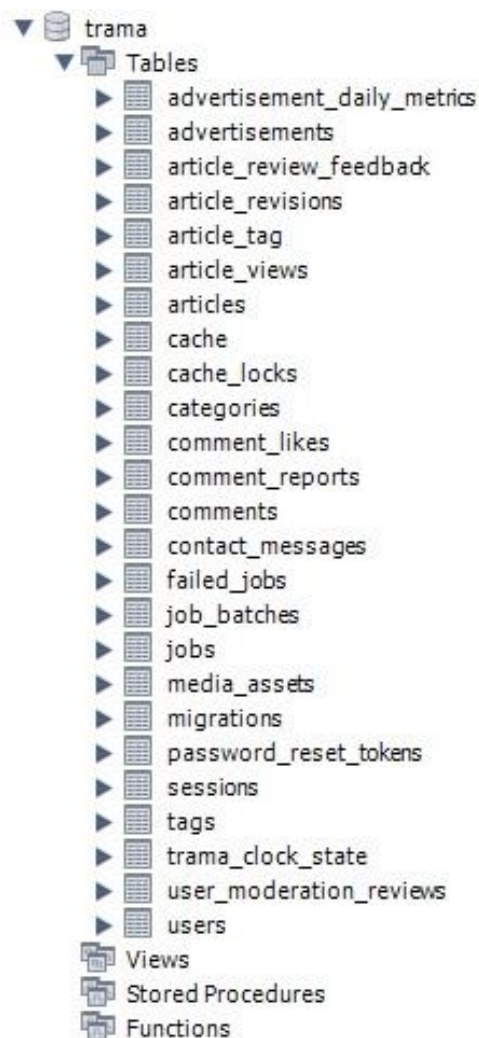
`DB_PORT`, `DB_USERNAME`, and `DB_PASSWORD` must match the local installation. The documentation deliberately uses placeholders and does not publish real passwords.

30.7 Create the tables with migrations

With the empty database created and `.env` configured, Laravel can build the current schema:

```
php artisan migrate
```

Migrations create tables, columns, indexes, foreign keys, and Laravel's internal tables. When they finish, SCHEMAS > trama > Tables can be refreshed in Workbench.



TRAMA tables created in MySQL after running the migrations.

30.8 Load test data

Migrations create the structure but do not load test data. To load employees, registered users, articles, comments, revisions, advertising, and the other data prepared by TRAMA, run:

```
php artisan db:seed
```

DatabaseSeeder coordinates the project's seeders, and TramaIntegritySeeder checks several counts and consistency rules in the final scenario. In a freshly seeded database, some useful values for quickly checking the result are:

Item	Expected result
Users	2,506 (2,500 registered users + 6 employees)
Articles	90
Comments and replies	1,395
Advertisements	15
Article views	64,212
Banner impressions / clicks	35,298 / 145

When you deliberately want to destroy a local environment and rebuild it completely in one operation, you can use:

```
php artisan migrate:fresh --seed
```

Be careful with migrate:fresh

migrate:fresh deletes all tables in the configured database before running the migrations and seeders again. It must not be used on a database whose data needs to be preserved.

30.9 Prepare the initial SSR bundle

The repository does not include bootstrap/ssr because that folder is excluded by .gitignore. However, php artisan inertia:start-ssr requires the compiled SSR bundle to exist. In a clean copy, after npm install, generate that bundle once with:

```
npm run build:ssr
```

This command compiles resources/js/ssr.js and normally generates bootstrap/ssr/ssr.js. It does not start the SSR server. It can also be generated as part of npm run build, which builds both client and SSR; for initial local setup, build:ssr is enough when only the server bundle is missing.

The command that starts SSR is different

npm run build:ssr generates the file that Node will execute. The SSR server is started while TRAMA is in use with php artisan inertia:start-ssr, as explained in section 31.

30.10 Checking a completed installation

Item	Must exist or respond
PHP dependencies	vendor/
Node dependencies	node_modules/
Local configuration	.env with correct APP_KEY and DB_* values
MySQL database	trama with its tables
Test data	Seeders completed without errors
SSR bundle	bootstrap/ssr/ssr.js

At this point TRAMA is installed. composer install, npm install, migrations, and seeders are not repeated every time the project is opened. The next section explains only which processes need to be started for day-to-day work.

Step by step: Carlos installs TRAMA from a clean copy

This sequence combines into one walkthrough what happens the first time the project is prepared on a new PC.

Step	What the person does and what happens internally
1	Carlos downloads or clones the repository and checks that PHP, Composer, Node.js/NPM, and MySQL are available.
2	He runs composer install. Composer rebuilds vendor/ from composer.lock.
3	He runs npm install. NPM rebuilds node_modules/ from package.json and package-lock.json.
4	He copies .env.example as .env and generates APP_KEY with php artisan key:generate.
5	He manually creates the trama MySQL database and fills in the host, port, username, password, and DB_DATABASE in .env.

Step	What the person does and what happens internally
6	He runs <code>php artisan migrate</code> . Laravel creates tables, indexes, foreign keys, and internal structures.
7	He runs <code>php artisan db:seed</code> to load the test-data scenario and allow <code>TramaIntegritySeeder</code> to verify its consistency.
8	He generates the initial SSR bundle with <code>npm run build:ssr</code> to create <code>bootstrap/ssr/ssr.js</code> , which is not versioned in the repository.
9	He checks that storage and application-managed folders are writable and that the required external dependencies are available.
10	At this point, the initial installation is complete. On following days, he does not repeat all these steps: he only starts the development processes described in chapter 31.

31. Running TRAMA in Development

Once the local installation from section 30 is complete, TRAMA already has its dependencies, configuration, database, test data, and SSR bundle. From this point on, initial setup is separate from day-to-day startup: normally only the processes that keep the application running while work is being done need to be started.

31.1 Processes TRAMA needs

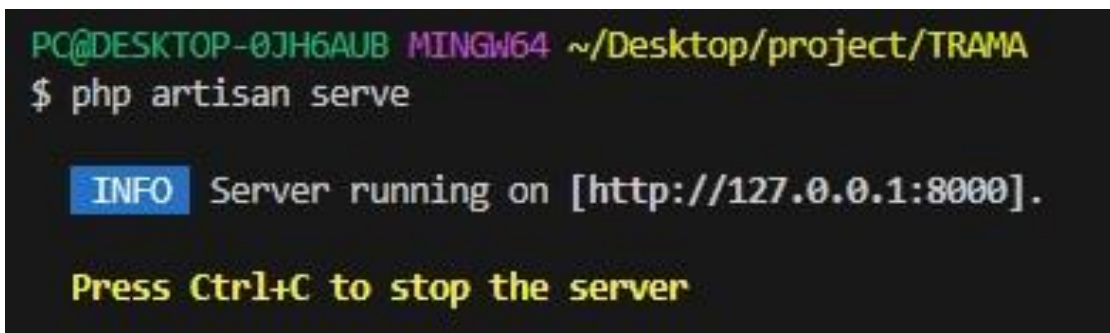
Process	Development responsibility
Laravel	Handles HTTP requests through <code>php artisan serve</code> .
Vite	Serves development JavaScript and CSS through <code>npm run dev</code> .
SSR	Generates the initial HTML for Public/Home through <code>php artisan inertia:start-ssr</code> .
Queue worker	Processes Jobs from moderation and default.
Scheduler	Checks scheduled tasks through <code>php artisan schedule:work</code> .

These processes are not duplicate servers. Each covers a different responsibility and can be running while another fails. For learning and diagnosing the project, this guide therefore starts them in separate terminals.

31.2 Terminal 1 - Laravel server

```
php artisan serve
```

Laravel starts its development server and displays the HTTP URL, normally `http://127.0.0.1:8000`. This terminal must remain open.



```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ php artisan serve

INFO Server running on [http://127.0.0.1:8000].

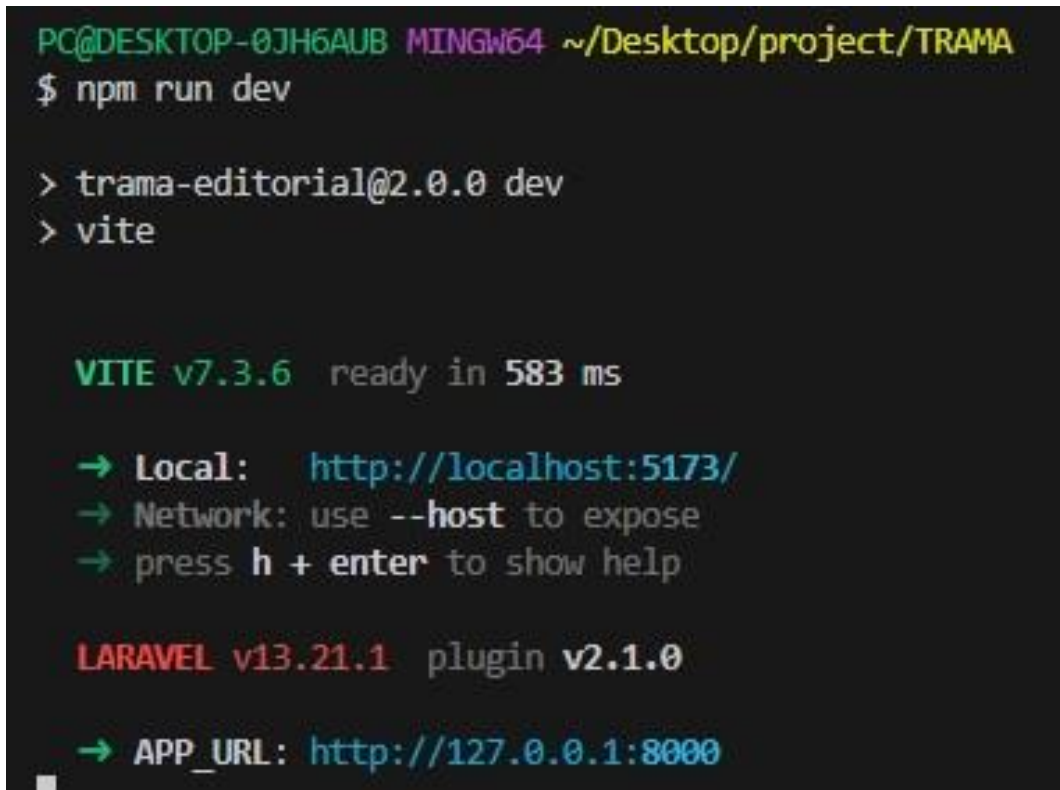
Press Ctrl+C to stop the server
```

Laravel development server running at `http://127.0.0.1:8000`.

31.3 Terminal 2 - Vite

```
npm run dev
```

Vite serves the frontend assets and keeps Vue and CSS in development mode. Port 5173 belongs to Vite; the TRAMA site is opened through the Laravel URL, not by navigating directly to Vite.



```

PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ npm run dev

> trama-editorial@2.0.0 dev
> vite

VITE v7.3.6 ready in 583 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

LARAVEL v13.21.1 plugin v2.1.0

→ APP_URL: http://127.0.0.1:8000
  
```

Vite running and connected to the Laravel application configured in APP_URL.

31.4 Terminal 3 - SSR server

```
php artisan inertia:start-ssr
```

This is the command that actually starts SSR. `INERTIA_SSR_ENABLED=true` only enables the feature; it does not start Node by itself. The process uses `bootstrap/ssr/ssr.js` and must remain open while pre-rendering for Public/Home is desired. If code used by SSR changes, `npm run build:ssr` must be run and this process restarted; `npm run dev` does not automatically rebuild the SSR bundle.



```

PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ php artisan inertia:start-ssr
Starting SSR server on port 13714...
Inertia SSR server started.
  
```

Inertia SSR server started successfully on port 13714.

The aliases defined in `composer.json` allow `composer ssr:start`, `composer ssr:check`, and `composer ssr:stop` to be run, but `php artisan inertia:start-ssr` makes it explicit which process is being started.

31.5 Terminal 4 - Queue worker

```
php artisan queue:work --queue=moderation,default --tries=3 --timeout=70
```

The worker takes pending Jobs. In particular, when a user submits a comment, `ModerateComment` remains in the moderation queue and does not run the Node pipeline until a worker consumes it.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ php artisan queue:work --queue=moderation,default --tries=3 --timeout=70

INFO Processing jobs from the [moderation,default] queues.
```

Laravel worker processing the moderation and default queues.

31.6 Terminal 5 - Local Scheduler

```
php artisan schedule:work
```

`schedule:work` keeps the Scheduler active during development. In TRAMA, it lets Laravel check tasks such as `articles:publish-scheduled` every minute. Scheduler and TramaClock are different mechanisms: one triggers tasks; the other defines the scenario's editorial date and time.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ php artisan schedule:work

INFO Running scheduled tasks.
```

Local Scheduler running and checking scheduled tasks.

31.7 Open TRAMA in the browser

With Laravel and Vite running, the application can be opened. To also test SSR, automatic moderation, and scheduled publishing, the corresponding processes must be active. The main URL in the documented environment is:

```
http://127.0.0.1:8000
```



TRAMA homepage running in the local development environment.

31.8 Step by step: Carlos starts TRAMA each day

Step	What Carlos does
1	Carlos opens the TRAMA root and checks that the configured MySQL server is running.
2	Carlos opens Terminal 1 and runs <code>php artisan serve</code> .
3	Carlos opens Terminal 2 and runs <code>npm run dev</code> .
4	Carlos opens Terminal 3 and runs <code>php artisan inertia:start-ssr</code> .
5	Carlos opens Terminal 4 and runs <code>php artisan queue:work --queue=moderation,default --tries=3 --timeout=70</code> .
6	Carlos opens Terminal 5 and runs <code>php artisan schedule:work</code> .
7	With the required processes running, Carlos opens <code>http://127.0.0.1:8000</code> in the browser.

31.9 The composer dev shortcut and Pail's limitation on Windows

TRAMA also includes a development script in the `composer.json` file that allows all the processes required to work with the application to be started in a single execution, avoiding the need to manually open and run the five terminals described in the previous subsection.

```
composer dev
```

Before starting the processes, the script runs:

```
npm run build:ssr
```

to generate the bundle used by Inertia SSR. It then starts the following processes simultaneously:

Proceso	Comando
Laravel	<code>php artisan serve</code>
Job queue	<code>php artisan queue:listen --queue=moderation,default --tries=3 --timeout=70</code>
Vite	<code>npm run dev</code>
Inertia SSR	<code>php artisan inertia:start-ssr</code>
Scheduler	<code>php artisan schedule:work</code>

31.10 Quick diagnostics

Test	What it confirms	If it fails, check first
Open <code>http://127.0.0.1:8000</code>	Laravel responds and the application can handle HTTP.	<code>php artisan serve</code> , <code>.env</code> , and MySQL connection.
The interface loads styles and JavaScript	Vite is serving the development assets.	<code>npm run dev</code> and the Vite console.
<code>php artisan inertia:check-ssr</code>	The SSR server responds.	<code>bootstrap/ssr/ssr.js</code> bundle and <code>inertia:start-ssr</code> .
Submit a comment	The Job enters moderation and can be processed.	<code>queue:work</code> , <code>TRAMA_NODE_BINARY</code> , and models.
Schedule an article for the near future	The Scheduler can run scheduled publishing.	<code>schedule:work</code> and <code>routes/console.php</code> .

32. Deploying TRAMA to a Server

Deploying TRAMA means moving from the local development environment to a server accessible from the Internet without exposing private files and while keeping the processes the application needs running. This documentation does not assume a specific provider: it describes the requirements any deployment must satisfy.

32.1 What the server must provide

Requirement	Why it is required
Compatible PHP	The current <code>composer.lock</code> requires at least PHP 8.4.1; the documented environment was verified with PHP 8.5.8.
Composer	Installs the PHP production dependencies.
Node.js + NPM	Generate the build and remain necessary at runtime for SSR, Sharp, ONNX Runtime, and moderation.
MySQL	Stores persistent data.
HTTP server with HTTPS	Must expose only <code>public/</code> as the web root.
Persistent processes	Queue worker and SSR must be kept running and restarted if they stop.
Task scheduler	Must invoke <code>php artisan schedule:run</code> every minute.
SMTP and write access to <code>storage/</code> and <code>public/images/</code>	Required for email, logs, and persistent files generated or replaced by the CMS.

32.2 Prepare the production `.env`

The production instance needs its own `.env`. `APP_ENV` must be `production`, `APP_DEBUG` must remain `false`, and `APP_KEY` must contain a valid key. Real credentials and the application key are not published in the repository.

```

APP_ENV=production
APP_DEBUG=false
APP_URL=https://<real-domain>
APP_KEY=

DB_CONNECTION=mysql
DB_HOST=<mysql-host>
DB_PORT=<mysql-port>
DB_DATABASE=<production-database>
DB_USERNAME=<username>
DB_PASSWORD=<password>

```

On a new installation, after creating `.env`, `php artisan key:generate` must be run once to populate `APP_KEY`. An existing key must not be regenerated on an already operating production system unless the impact on encrypted data, cookies, and sessions is understood. `SMTP`, `timezone`, `TRAMA_NODE_BINARY`, and the `INERTIA_SSR_*` variables must also be configured for the real server.

32.3 Install production dependencies

```

composer install --no-dev --optimize-autoloader
npm ci

```

Composer installs only the PHP dependencies required to run the application and optimizes the autoloader. `npm ci` uses `package-lock.json` to reproducibly rebuild the JavaScript tree. In TRAMA's current structure, `Vue`, `Inertia`, `Axios`, and other components imported by the SSR bundle are in `devDependencies`; therefore `npm ci --omit=dev` must not be used, and `devDependencies` must not be removed after the build while SSR still depends on them.

32.4 Build the client and SSR bundle

```

npm run build

```

In TRAMA, the build script runs the client build and then the SSR build. The result includes `public/build/` for browser assets and `bootstrap/ssr/ssr.js` for the SSR process. Unlike development, Vite does not need to remain running after the production assets have been generated.

32.5 Apply migrations without destroying data

```

php artisan migrate --force

```

`--force` allows migrations to run in an environment marked as production. This command updates the existing structure; it must not be replaced with `migrate:fresh` when the database contains information that must be preserved. Test seeders should also not be run automatically against a real database unless there is an explicit decision to populate it with test data.

32.6 Configure `public/` as the web root

The complete project must exist on the server because Laravel needs `app/`, `vendor/`, `config/`, `resources/`, `storage/`, and the other internal folders. Only `public/` should be exposed over HTTP. In Apache, Nginx, or the chosen server, `DocumentRoot` or the web root must point to TRAMA's `public/` folder.

```

/path/to/project/TRAMA/
├─ app/
├─ config/
├─ resources/
├─ storage/
├─ vendor/
├─ .env
└─ public/   ← domain web root

```

This does not mean copying only `public/` to the server

Laravel needs the complete project. The rule is that `public/` must be the only folder directly accessible from the web; `.env`, `app/`, `vendor/`, and `resources/` must remain outside the public root.

32.7 Keep the queue worker running

```
php artisan queue:work --queue=moderation,default --tries=3 --timeout=70
```

In production, the worker must be maintained as a persistent process using the supervision mechanism available on the server. If it stops, Jobs may continue accumulating and comments can remain in processing until it runs again.

32.8 Keep the SSR server running

```
php artisan inertia:start-ssr
```

If selective SSR for Public/Home is to be preserved, this process must also remain active. The production environment should supervise it and restart it if it exits; it must not depend on a terminal left open manually.

32.9 Run the Scheduler every minute

```
php artisan schedule:run
```

In production, schedule:work is not kept running as it is in development. The operating system or platform must invoke schedule:run every minute. routes/console.php registers articles:publish-scheduled with everyMinute() and withoutOverlapping().

32.10 Permissions, models, logs, backups, and HTTPS

- storage/, bootstrap/cache/, and the folders managed by the CMS inside public/images/—articles/, categories/, team/, and banners/—must be writable by the Laravel/Node processes that create or replace those files.
- resources/models must retain the ONNX models and auxiliary files used by comments and images.
- TRAMA_NODE_BINARY must point to a compatible Node executable available on the server.
- Logs must be writable, and a rotation and diagnostics policy should be defined.
- The MySQL database and persistent files, including images managed inside public/images/, require a backup policy.
- The public site must use HTTPS and APP_DEBUG must remain disabled.

32.11 Post-deployment checks

Check	Expected result
Open the domain	The homepage responds over HTTPS.
Authentication	Registration/login and sessions work with the production database.
SSR	php artisan inertia:check-ssr confirms that the SSR server is responding.
Comments	A comment leaves processing when the worker processes ModerateComment.
Scheduling	A scheduled article can be published through Scheduler.
Images	Sharp/ONNX work with TRAMA_NODE_BINARY and resources/models.
Email	Verification, recovery, and Contact use the production SMTP configuration.
Logs	Technical errors are recorded without being exposed to visitors.

Step by step: Carlos deploys a TRAMA release to production

A correct deployment is not just a matter of copying files: it must also leave Laravel, Node, and operating-system background processes running.

Step	What the person does and what happens internally
1	Carlos copies the project to a server that meets the required versions and extensions and keeps private files outside public access.
2	He creates the production <code>.env</code> with <code>APP_ENV=production</code> , <code>APP_DEBUG=false</code> , a valid <code>APP_KEY</code> , and real MySQL, mail, Node, and SSR credentials/configuration.
3	He runs <code>composer install</code> with the production options and <code>npm ci</code> to reconstruct the JavaScript dependencies exactly from the lockfile.
4	He runs <code>npm run build</code> . Vite generates <code>public/build/</code> and the SSR bundle Node will use in production.
5	He runs <code>php artisan migrate --force</code> to update the structure without destroying existing data. He does not use <code>migrate:fresh</code> on a real database.
6	He configures the web server so only <code>public/</code> is exposed as the HTTP root.
7	He leaves a supervised queue worker running so Jobs such as <code>ModerateComment</code> do not accumulate.
8	He keeps the SSR server running and supervised if he wants to preserve homepage pre-rendering.
9	He configures a system task that runs <code>php artisan schedule:run</code> every minute for periodic tasks.
10	He checks permissions, HTTPS, logs, backups, local models, and a complete portal navigation before considering the deployment finished.

33. Test Data and Seeders

A freshly migrated installation has the structure required to run, but its tables may be empty. That would require manually creating users, articles, comments, editorial statuses, and advertisements before the project could be explored. TRAMA avoids that work with a prepared test dataset designed to reconstruct a complete, repeatable local scenario.

TRAMA uses several seeders to automatically rebuild that test scenario. This section first explains what a seeder is and then, from that idea, how `DatabaseSeeder` coordinates the PHP classes and JSON files that ultimately load records into MySQL.

Seeders do not create the database structure. That task belongs to migrations, which create or modify tables, columns, indexes, and relationships. Seeders work afterward on that structure and load the required records. Running migrations is therefore not the same as loading test data: they are two separate steps.

What a seeder is and what it is used for

A seeder is a Laravel class whose purpose is to insert data automatically into tables that already exist. Its main benefit is avoiding manual data creation and quickly, reproducibly loading the records needed to develop, test, or demonstrate an application without having to create them one by one.

In TRAMA, seeders make it possible to recreate in a controlled way the data needed to test the system: users, employees, categories, tags, articles, comments, moderation, Contact inquiries, and advertising. If the database is prepared again from scratch, the same seeders can automatically load that data again.

The important idea is that a seeder works on the structure created by migrations. Tables and columns must exist first; the seeder then inserts rows into them. Section 34 introduces factories, which serve a different purpose: generating only the records required by a specific test.

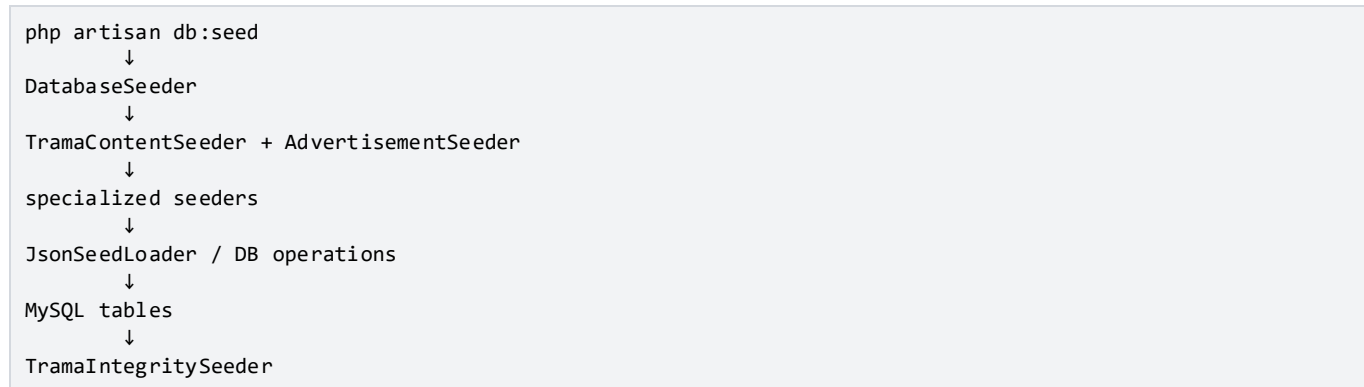
Concept	What it is used for
Migration	Creates or modifies the database structure: tables, columns, indexes, and relationships.
Seeder	Loads a prepared dataset into a structure that already exists.
Factory	Generates test records on demand, usually for a specific automated scenario.

Main idea of this section

Migrations prepare where information will be stored. Seeders prepare which test information will be stored. TRAMA also separates large datasets into JSON files and uses PHP classes to decide how and in what order they reach MySQL.

33.1 From an empty database to a test scenario

The usual command for seeding a database whose structure already exists is `php artisan db:seed`. Laravel begins with `DatabaseSeeder`, which acts as the entry point for the seeding process. From there, the remaining seeders run until users, taxonomies, articles, comments, moderation, contact, and advertising data have been loaded.



The final step is important: `TramaIntegritySeeder` does not add content. It reviews the result to detect inconsistent test data before the load is considered complete.

33.2 Where the data lives: JSON files

JSON is a text format that represents information through field names and values. TRAMA uses it to keep most test records separate from the PHP code that organizes the load. Those files are located under `database/seeders/data/`.

For example, `categories.json` contains the categories that will later appear in the categories table. One of its rows is represented as follows:

```
{
  "id": 1,
  "name": "Política",
  "slug": "politica",
  "description": "Poder, Congreso, gobiernos y decisiones públicas.",
  "accent_color": "#D8323C",
  "cover_image": "/images/categories/politica.webp",
  "sort_order": 1,
  "is_active": true,
  "created_at": "2026-06-05 08:00:00",
  "updated_at": "2026-06-05 08:00:00"
}
```

The file does not create a MySQL row by itself. It is only a representation of the data. A PHP class must read it, check that it can be decoded correctly, and perform the corresponding insertion; in TRAMA, that shared task is centralized in `JsonSeedLoader`.

33.3 DatabaseSeeder: entry point and transaction

`DatabaseSeeder` is the class Laravel runs when the general load begins. Its `run()` method does not contain thousands of `INSERT` statements: it delegates work to other seeders and wraps the entire process in a transaction.

```
public function run(): void
{
    DB::disableQueryLog();

    DB::transaction(function (): void {
        $this->call([
            TramaContentSeeder::class,
            AdvertisementSeeder::class,
            TramaIntegritySeeder::class,
        ]);
    }, 3);
}
```

`run()` is the method Laravel executes in that seeder. `DB::disableQueryLog()` avoids retaining a record of every query in memory while thousands of rows are loaded. `$this->call()` specifies which seeders should run next.

`DB::transaction()` groups database operations into a single unit. If part of the load throws an exception before completion, the transaction can roll back its changes instead of leaving a partially loaded scenario. The second argument, 3, allows the transaction to be retried up to three times for certain concurrency conflicts, such as a deadlock; it does not mean every error is executed three times.

33.4 TramaContentSeeder: why order matters

`TramaContentSeeder` organizes the main part of the scenario. The order is not decorative: some records require others to already exist before their relationships can be stored correctly.

```
public function run(): void
{
    $this->call([
        TramaStaffSeeder::class,
        TramaTaxonomySeeder::class,
        TramaRegisteredUsersSeeder::class,
        TramaArticlesSeeder::class,
        TramaEditorialHistorySeeder::class,
        TramaEngagementSeeder::class,
        TramaModerationSeeder::class,
        TramaContactSeeder::class,
    ]);
}
```

Employees, categories, tags, and users are prepared first; articles and the relationships that depend on them can then be loaded. History, comments, and moderation data are added later. Contact messages are loaded last.

A concrete example appears in `TramaEngagementSeeder`. Top-level comments are inserted before replies because a reply can store another comment's identifier in `parent_id`. That parent comment must exist before the reply is inserted.

```
$comments = JsonSeedLoader::rows('comments.json');
$main = array_values(array_filter(
    $comments,
    static fn (array $row): bool => $row['parent_id'] === null,
));
$replies = array_values(array_filter(
    $comments,
    static fn (array $row): bool => $row['parent_id'] !== null,
));

foreach (array_chunk($main, 400) as $chunk) {
    DB::table('comments')->insert($chunk);
}
foreach (array_chunk($replies, 400) as $chunk) {
    DB::table('comments')->insert($chunk);
}
```

33.5 JsonSeedLoader: how JSON reaches MySQL

Many seeders need to perform exactly the same work: locate a JSON file, read it, check that it can be decoded correctly and contains a list, and save its rows. `JsonSeedLoader` exists to handle that shared logic and avoid repeating it in every class.

The `rows()` method first handles reading and validating the file:

```
public static function rows(string $filename): array
{
    $path = base_path('database/seeder/data/'.$filename);

    if (! is_file($path)) {
        throw new RuntimeException("No se encontró el archivo de seed: {$path}");
    }

    try {
        $rows = json_decode(
            (string) file_get_contents($path),
            true,
            512,
            JSON_THROW_ON_ERROR,
        );
    } catch (JsonException $exception) {
        throw new RuntimeException(
            "JSON inválido en {$filename}: {$exception->getMessage()}",
            previous: $exception,
        );
    }

    if (! is_array($rows)) {
        throw new RuntimeException("El archivo {$filename} no contiene una lista válida.");
    }

    return $rows;
}
```

`base_path()` builds the location within the project. `is_file()` prevents the process from continuing if the file does not exist. `file_get_contents()` reads its contents, and `json_decode()` converts them into PHP arrays. `JSON_THROW_ON_ERROR` makes malformed JSON produce a catchable error; the catch turns it into an exception with a message that identifies the file.

The `insert()` method uses those rows and saves them to the specified table:

```

public static function insert(
    string $table,
    string $filename,
    int $chunkSize = 500,
    ?callable $transform = null,
): void {
    $rows = self::rows($filename);

    if ($transform !== null) {
        $rows = array_map($transform, $rows);
    }

    foreach (array_chunk($rows, $chunkSize) as $chunk) {
        if ($chunk !== []) {
            DB::table($table)->insert($chunk);
        }
    }
}

```

`$table` specifies the destination table and `$filename` the JSON file to read. `$chunkSize` determines how many rows are grouped in each insertion. `array_chunk()` divides a large dataset into smaller batches, and each batch is inserted with `DB::table(...)->insert(...)`.

`$transform` is optional. When a seeder needs to modify a row before saving it, it can supply a transformation function. If it does not need one, `JsonSeedLoader` inserts the data exactly as read.

33.6 Simple example: TramaArticlesSeeder

Once `JsonSeedLoader` is understood, a simple seeder is reduced to specifying which file feeds which table and what batch size it uses. `TramaArticlesSeeder` contains:

```

public function run(): void
{
    JsonSeedLoader::insert('articles', 'articles.json', 250);
    JsonSeedLoader::insert('article_tag', 'article_tag.json', 500);
    JsonSeedLoader::insert('media_assets', 'media_assets.json', 250);
}

```

The first line can be read literally as: read `articles.json` and insert its rows into the `articles` table in groups of up to 250. The other two lines do the same for the `article_tag` pivot table and for `media_assets`.

```

articles.json → JsonSeedLoader → articles
article_tag.json → JsonSeedLoader → article_tag
media_assets.json → JsonSeedLoader → media_assets

```

Not all seeders are this direct. `TramaRegisteredUsersSeeder`, for example, applies a transformation before inserting users: it temporarily leaves several moderation-summary fields as null because their relationships with comments or users cannot yet be resolved, and replaces password with a shared test hash. After those relationships exist, `TramaModerationSeeder` restores the moderation summary in users. This process again shows why load order is part of the project's logic.

A password hash is a non-readable representation of the original password. Test accounts can authenticate with password, but the password column does not store that word in plain text.

33.7 TramaIntegritySeeder: checking that the data makes sense

Loading every row does not by itself guarantee that the scenario is correct. `TramaIntegritySeeder` runs at the end to check counts as well as relationships between tables.

A simple validation checks, for example, that exactly 90 articles exist:

```
$this->assertSame(
    90,
    DB::table('articles')->count(),
    'noticias',
);
```

Other checks do more than count. The following fragment looks for comments associated with articles that are neither published nor archived. The expected result is zero:

```
$nonPublicComments = DB::table('comments')
->join('articles', 'articles.id', '=', 'comments.article_id')
->whereNotIn('articles.status', ['published', 'archived'])
->count();

$this->assertSame(
    0,
    $nonPublicComments,
    'comentarios en noticias no públicas',
);
```

`assertSame()` is a private method of the seeder itself. It receives the expected value, the obtained value, and a label identifying the check. If the two values do not match, it throws a `RuntimeException`:

```
private function assertSame(int $expected, int $actual, string $label): void
{
    if ($expected !== $actual) {
        throw new RuntimeException(
            "Integridad TRAMA: {$label}. Esperado {$expected}; obtenido {$actual}.",
        );
    }
}
```

An exception interrupts normal execution. Because `DatabaseSeeder` performs the load inside a transaction, an inconsistency detected at the end can cause that load to be rolled back instead of remaining partially applied.

Among the rules `TramaIntegritySeeder` checks in this version are:

- 2,500 registered users and 6 employees;
- 90 articles and their exact distribution by editorial status;
- 6 categories, 12 active tags, 15 advertisements, and 285 advertising-metric rows;
- no views or public comments on articles that are not yet public;
- a match between `articles.views` and the actual number of `article_views` rows;
- no comments from unverified public accounts;
- and an administrative review with a blocked resolution for every inactive public account.

33.8 Results of a Fresh Seed Run and Verification in MySQL

Current Source of Test Data

`database/seeder/data/seed_stats.json` records 64,212 article views, 35,298 impressions, and 145 advertising clicks. `RESULTADO_GENERADO.md` still contains 19,340 views, 35,293 impressions, and 140 clicks. This document treats `seed_stats.json` as the current result of the dataset.

After a fresh seed run, several totals provide a quick way to confirm that the main scenario is available. These numbers describe the test data prepared by the project; they are not TRAMA limits.

Item	Expected result
Registered users	2,500
Employees	6
Articles	90

Item	Expected result
Article views	64,212
Comments and replies	1,395
Advertisements	15
Daily advertising-metric records	285

The 90 articles cover different points in the editorial workflow so the panel can be tested without manually creating every case:

Status	Count
Published	60
Archived	8
Scheduled	5
Under Review	7
Returned for Changes	5
Drafts	5

The same records can be checked directly in MySQL. The following screenshot comes from a local TRAMA database that has already been used and shows COUNT(*) queries on users, articles, and comments.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'trama' selected. The main window shows a query with five lines:

```
1 • USE trama;
2
3 • SELECT COUNT(*) FROM users;
4 • SELECT COUNT(*) FROM articles;
5 • SELECT COUNT(*) FROM comments;
```

The 'Result Grid' shows the output of the query:

COUNT(*)
1397

The bottom panel shows the 'Action Output' with the following details:

#	Time	Action	Message	Duration / Fetch
1	09:39:49	USE trama	0 row(s) affected	0.000 sec
2	09:39:49	SELECT COUNT(*) FROM users LIMIT 0, 1000	1 row(s) returned	0.062 sec / 0.000 sec
3	09:39:49	SELECT COUNT(*) FROM articles LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
4	09:39:49	SELECT COUNT(*) FROM comments LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec

Verification queries on a local TRAMA database that has already been used.

This block provides a closer look at the query and the result grid. The query is:

```
1 • USE trama;
2
3 • SELECT COUNT(*) FROM users;
4 • SELECT COUNT(*) FROM articles;
5 • SELECT COUNT(*) FROM comments;
```

The 'Result Grid' shows the output:

COUNT(*)
1397

In the selected tab in the screenshot, comments returns 1,397. A fresh seed run contains 1,395 comments and replies. The screenshot therefore shows two more comments than a fresh seed run, which is expected in a local database that has already been used. A database in use can therefore differ from the seeder's deterministic totals without creating a contradiction.

Rebuild a disposable database from scratch

`php artisan migrate:fresh --seed` deletes the tables from the configured database, runs the migrations again, and then runs the seeders. It should only be used when that data can be deleted. If the structure already exists and only the test data need to be seeded, use `php artisan db:seed`.

33.9 Test accounts and their relationship with TramaClock

The six internal accounts loaded by the seeders make it possible to enter the CMS with all available roles without registering employees manually. They all use the password `password` only within the local test environment.

Name	Email	Role
Marina Ibarra	admin@trama.test	Administrator
Mateo Ledesma	mateo.ledesma@trama.test	Editor
Valentina Ríos	valentina.rios@trama.test	Journalist
Lucía Ferreyra	lucia.ferreyra@trama.test	Journalist
Bruno Salvatierra	bruno.salvatierra@trama.test	Journalist
Camila Torres	camila.torres@trama.test	Journalist

Public test accounts use the same password and addresses ending in `.test` domains, so the data has a valid email format without pointing to real mailboxes. These values exist for the test environment and do not represent a password policy for real users.

The five scheduled articles in the test data include dates after 19/07/2026. They are useful for checking that the panel saves, lists, and filters articles with `status=scheduled`. However, TramaClock has the time limit explained in section 8 and does not automatically advance beyond 19/07/2026. As a result, not all of them can reach their publication date simply by letting that clock advance.

Step by step: Carlos runs db:seed and a JSON file becomes MySQL rows

The example connects the data files with the PHP classes that organize the load.

Step	What the person does and what happens internally
1	Carlos starts from an already migrated database and runs <code>php artisan db:seed</code> .
2	Laravel enters through <code>DatabaseSeeder::run()</code> , disables the query log so thousands of queries do not accumulate in memory, and opens the general load inside a transaction.
3	<code>DatabaseSeeder</code> calls <code>TramaContentSeeder</code> and the other seeders in an order designed to respect dependencies among users, categories, articles, comments, and other data.
4	A specific seeder requests rows from <code>JsonSeedLoader</code> by specifying a file under <code>database/seeders/data/</code> .
5	<code>JsonSeedLoader</code> locates the JSON file, reads the text, and converts it to PHP arrays. If the file does not exist or the JSON is malformed, it stops the load with an identifiable error.

Step	What the person does and what happens internally
6	The loader inserts the rows, normally in batches, into the corresponding table. Later seeders can use those identifiers to create relationships.
7	At the end, TramaIntegritySeeder does not add content: it checks counts and consistency rules to verify that the scenario is complete.
8	If something fails inside the transaction before completion, the load can be rolled back instead of leaving a database only partially populated.

34. Automated Tests and Technical Checks

The test data from the previous section and the automated tests in this section serve different purposes. Seeders prepare a populated database for exploring the project. An automated test, by contrast, is code that prepares a specific situation, performs an action, and checks whether the result matches the expectation.

TRAMA uses PHPUnit to run tests written in PHP. The complete set of tests is called the suite. These tests help detect regressions in behaviors covered by the test code. A regression occurs when a function that previously worked correctly stops doing so after a change. Automated tests do not replace checks of external processes such as Vite, SSR, the worker, Scheduler, or local models.

An assertion is a check that states what result a test expects. For example, `assertRedirect()` states that the response must be a redirect, and `assertDatabaseHas()` states that the test database must contain a row with specific values. If reality does not match that expectation, PHPUnit marks the test as failed.

What a factory is and what it is used for in tests

A factory is a class that quickly generates test records. Instead of depending on the entire database loaded by seeders, a test can request exactly the account, article, or relationship it needs to build its scenario.

For example, `User::factory()->create()` creates an account inside PHPUnit's temporary database. That row exists for the test currently being run and does not represent a real account in TRAMA's local environment. `RefreshDatabase` helps ensure that the next scenario does not depend on what the previous test left behind.

Do not confuse them: a seeder prepares a broad, repeatable scenario for exploring the project; a factory creates individual records on demand for a test. Both insert data, but they solve different needs.

34.1 Feature and Unit: two levels of testing

Laravel commonly organizes tests under `tests/Feature` and `tests/Unit`. A Feature test can exercise several parts of the system together: for example, authenticate an account, call a route, pass through middleware and controllers, use Eloquent, and verify what was saved in the test database.

A Unit test aims to isolate a small piece of logic. In TRAMA, most useful coverage is in Feature tests because many behaviors depend on permissions, editorial statuses, routes, and relationships between tables.

The main groups present in the project are:

File or group	Part of the project it exercises
<code>tests/Feature/Auth/*</code>	Registration, sign-in and sign-out, email verification, and password recovery.
<code>ArticleIndexQueryServiceTest.php</code>	Role-based visibility, filters, search, sorting, tags, counts, and pagination in the article list.

File or group	Part of the project it exercises
EditorialWorkflowTest.php	Public and editorial workflows: comments, article creation, slugs, cover images, revisions, scheduling, and other rules.
PublicCommentStageTwoTest.php	Reports, Likes, permissions, and limits in the comment and reply system.
TramaClockTest.php	Start, progression, pause, continuity, and limits of TramaClock.
tests/Feature/ExampleTest.php	Minimal check of an application response.
tests/Unit/ExampleTest.php	Minimal unit test of the PHPUnit environment.

The existence of a test file does not by itself prove that the entire application is correct. The suite must be run after changes to verify that its expectations still match the code's current behavior.

34.2 The environment used by PHPUnit

Tests do not necessarily use the same configuration used to open TRAMA during everyday development. `phpunit.xml` defines a specific environment for PHPUnit. Its variables include:

```
<env name="APP_ENV" value="testing"/>
<env name="DB_CONNECTION" value="sqlite"/>
<env name="DB_DATABASE" value=":memory:"/>
<env name="MAIL_MAILER" value="array"/>
<env name="QUEUE_CONNECTION" value="sync"/>
<env name="SESSION_DRIVER" value="array"/>
```

`APP_ENV=testing` identifies the run as a test environment. `DB_CONNECTION=sqlite` and `DB_DATABASE=:memory:` make PHPUnit use a temporary in-memory SQLite database, separate from the MySQL database normally used by the application. `MAIL_MAILER=array` prevents real email from being sent, and `SESSION_DRIVER=array` keeps test sessions in memory.

`QUEUE_CONNECTION=sync` means that, unless a test replaces the queue with a fake, Jobs configured for that connection execute synchronously within the test run itself.

34.3 RefreshDatabase and preparing each scenario

Many Feature tests use `RefreshDatabase`. It is a Laravel feature that prepares the test database so one scenario does not depend on rows left behind by another. Each test can therefore create only the users, articles, or relationships it needs.

```
use RefreshDatabase;
```

In that context, methods such as `assertDatabaseHas()` query PHPUnit's temporary database, not the MySQL database used when browsing TRAMA normally.

34.4 Real example: saving a comment for moderation

`EditorialWorkflowTest.php` contains an example combining authentication, an HTTP request, the database, and the moderation queue. Before each test in the file, `setUp()` runs:

```
protected function setUp(): void
{
    parent::setUp();

    Queue::fake([
        ModerateComment::class,
    ]);
}
```

setUp() runs before every test method in that class. Queue::fake() temporarily replaces the real execution of ModerateComment. This allows the test to check the state Laravel leaves immediately after receiving the comment without starting Node or the moderation models.

Inside the method, the user and article are prepared first and the request is then sent:

```
$reader = User::factory()->create(['name' => 'Sofia Herrera', 'email' => 'sofia@example.com']);
$article = $this->publishedArticle();

$response = $this->actingAs($reader)->post(route('comments.store', $article), [
    'body' => 'Comentario con cuerpo suficiente.',
]);

$response->assertRedirect();
```

User::factory()->create() generates an account only for that scenario. publishedArticle() is a helper method in the same file that prepares a published article. actingAs(\$reader) makes the request behave as if that account had signed in. route('comments.store', \$article) generates the route URL, and post() simulates sending an HTTP request to that URL. \$response stores the response returned by Laravel, and assertRedirect() checks that it is a redirect.

Next, the test checks what was stored in PHPUnit's temporary database:

```
$this->assertDatabaseHas('comments', [
    'article_id' => $article->id,
    'user_id' => $reader->id,
    'author_name' => 'Sofia Herrera',
    'author_email' => 'sofia@example.com',
    'status' => 'processing',
    'moderation_revision' => 1,
    'moderation_source' => 'automatic',
]);
```

assertDatabaseHas() verifies that a comment exists with those values. In this case it confirms, among other things, that the initial status is processing, moderation_revision is 1, and moderation_source is automatic.

This test does not check the final output of the moderation models because ModerateComment was replaced by Queue::fake(). Real Job execution belongs to an operational check with the worker and moderation resources available.

34.5 How to run the tests

The complete suite is started from the project root with:

```
php artisan test
```

composer.json also defines the composer test shortcut, which first clears the configuration cache and then runs PHPUnit:

```
composer test
```

When working on a specific area, an individual file or method can be run:

```
php artisan test tests/Feature/EditorialWorkflowTest.php

php artisan test \
    --filter=test_authenticated_registered_user_comment_is_saved_as_processing
```

Running a specific test helps isolate work during development; the full suite is what later verifies whether a change affected other covered behaviors.

34.6 Operational environment checks

An automated test can verify code without proving that every required process is active in a real installation. TRAMA therefore also needs operational checks: actions intended to confirm that each external process is working in the environment where the application runs.

Check	Expected result	What it confirms
<code>php artisan serve + open /</code>	The homepage responds at 127.0.0.1:8000.	Laravel, routes, configuration, and access to the database used by the application.
<code>npm run dev</code>	Vite reports ready and the interface loads development JavaScript and CSS.	Node.js, NPM, and Vite.
<code>php artisan inertia:check-ssr</code>	The SSR server responds.	SSR bundle and inertia:start-ssr process.
Submit a comment with the worker running	The comment leaves processing when ModerateComment finishes.	Queue, worker, Node script, and moderation resources.
Schedule an article with <code>schedule:work</code> running	The Scheduler checks the task and can publish it when appropriate.	Scheduler and the scheduled-publishing command.
Upload a valid cover image	The image is processed and stored in the expected format.	TRAMA_NODE_BINARY, Sharp, and the image-processing chain.

Checks that require inference can be completed only if resources/models contains the ONNX and auxiliary files expected by the scripts. Those resources must be installed before running an operational check that depends on the models.

34.7 Production build and final review

The build checks another part of the project: whether Vite can generate the assets intended for production. The command defined by TRAMA builds the client and SSR bundle:

```
npm run build
```

When it finishes successfully, the result includes the compiled browser files under `public/build/` and the bundle used by SSR. This check does not replace automated tests or a functional review of the published application.

After installing or deploying TRAMA, it is advisable to check at least the following:

- open the homepage, a category, and an article and verify public navigation;
- sign in with internal accounts from different roles and confirm that each panel respects its permissions;
- create or edit an article and verify that the change persists in the database used by the application;
- submit a comment and verify its workflow with the worker running;
- schedule an article and confirm that Scheduler processes it;
- test an image upload, email, and log writing in the corresponding environment;
- and, in production, confirm HTTPS, `APP_DEBUG=false`, the required write permissions, and that `public/` is the only exposed web root.

34.8 Technical notes and project consistency

Static checks can review the syntax of PHP files and JavaScript/MJS scripts. In addition, there are some mismatches among documentation, tests, and executable code that should be known before interpreting a complete run as total validation of the project.

Finding	Current technical status
ONNX models	The four folders under resources/models exist, but they are empty. The scripts are implemented; real inference requires installing the expected models and auxiliary files.

Finding	Current technical status
RESULTADO_GENERADO.md	Contains older figures: 19,340 views, 35,293 impressions, and 140 clicks. seed_stats.json contains 64,212 views, 35,298 impressions, and 145 clicks.
docs/empleados-acceso.md	Lists only five employees and outdated data. staff.json contains six accounts: Marina Ibarra, Mateo Ledesma, Valentina Ríos, Lucía Ferreyra, Bruno Salvatierra, and Camila Torres.
PublicCommentStageTwoTest.php	The test test_like_limit_is_thirty_actions_per_minute still expects 30 actions per minute. CommentController currently applies 10 Like actions every 5 minutes. The functional documentation follows the current controller.
Inline body-image uploads	The admin/media route remains commented out and RichTextEditor does not provide inline image uploads. Cover images and the other images managed by the project continue to use their specific workflows.
Installed dependencies	vendor and node_modules are generated during installation through composer install and npm install, respectively, according to the documented workflow.

Documentation consistency criterion

When an auxiliary repository note and the executable code disagree, this documentation describes the behavior implemented by the current routes, components, Requests, controllers, Services, models, and datasets; the discrepancy is noted instead of hidden.

Step by step: Carlos runs a test and understands what it is checking

An automated test prepares its own scenario, performs a specific action, and compares the result with an expectation.

Step	What the person does and what happens internally
1	Carlos runs php artisan test or a specific file. PHPUnit loads phpunit.xml and switches the application to the testing environment.
2	The configuration uses temporary SQLite, an in-memory mailer, and other isolated options so the everyday MySQL database is not modified and real emails are not sent.
3	RefreshDatabase prepares the database for each scenario, and factories create only the rows that test needs.
4	A Feature test can use actingAs() to simulate a session and post(), patch(), or get() to pass through routes, middleware, and controllers.
5	assertDatabaseHas() checks the result in the temporary database, and other assertions verify redirects, responses, permissions, or dispatched Jobs.
6	When Queue::fake() is active, the test can verify that a Job was dispatched without actually running Node or the moderation models.
7	If an expectation fails, PHPUnit reports which test and which assertion did not match the current behavior.
8	Carlos corrects the code or the test as appropriate, then reruns the affected case first and the full suite afterward to detect regressions.